

Derivación de Programas

Notas de clase de Algoritmos y Estructuras de Datos I
(borrador)

Franco Luque

Índice

[Índice](#)

[Introducción](#)

[Un poco de historia](#)

[Ingeniería del Software: Programación a gran escala](#)

[Algoritmos I: Programación a pequeña escala \("in the small"\)](#)

[El proceso de construcción de programas](#)

[Roles básicos en un proyecto de software](#)

[Estructura del curso](#)

[Preliminares](#)

[Cálculo proposicional \(lógica de orden cero\)](#)

[Cálculo de predicados \(lógica de primer orden\)](#)

[Tipos básicos](#)

[Listas: funciones y propiedades](#)

[Demostraciones](#)

[Equivalencia entre dos predicados](#)

[Igualdad entre dos expresiones](#)

[Implicaciones](#)

[Análisis por casos](#)

[Diferencia entre igualdad \(\$=\$ \) y equivalencia \(\$\equiv\$ \)](#)

[Cuantificación General](#)

[Introducción](#)

[Sintaxis](#)

[Lectura Operacional](#)

[Ejemplos](#)

[Ejemplos con dos variables cuantificadas](#)

[Axiomas y Teoremas de Cuantificadores](#)

[Digesto](#)

[A1: Rango vacío](#)

[A2: Rango unitario](#)

[A3: Partición de rango](#)

[A4: Regla del término](#)

[A5: Término constante](#)

[A6: Distributividad a derecha](#)

[A6': Distributividad a izquierda](#)

[A7: Anidado](#)

[T1: Cambio de variable](#)

[Cambio de variable con listas](#)

[Ejercicios](#)

[T2: Eliminación de variable](#)

[T3: Rango unitario y condición](#)

[T11: Leibniz 2](#)

[Conteo](#)

[A8: Definición de conteo](#)

[Digesto](#)

[Reglas adicionales para el conteo](#)

[Ejercicio 15b](#)

[Ejercicios](#)

[Programación funcional](#)

[Introducción](#)

[Problemas](#)

[Especificaciones](#)

[Ejemplos](#)

[Observaciones](#)

[Predicados](#)

[Programas](#)

[Ejemplos](#)

[Demostración](#)

[Derivación](#)

[Testing / Ejecución](#)

[Resumen: El lenguaje de programación funcional](#)

[Modularización](#)

[Ejemplo: Promedio de una lista](#)

[Ejemplo: Suma de potencias](#)

[Resumen: ¿Qué hicimos?](#)

[Inducción](#)

[Esquemas de inducción](#)

[Inducciones con naturales](#)

[Inducciones con listas](#)

[Inducciones combinadas: Sub-inducción](#)

[Generalización](#)

[Ejemplo: Función psum](#)

[Resumen: ¿Qué hicimos?](#)

[Más ejemplos](#)

[Generalización con dos parámetros nuevos](#)

[Segmentos de lista](#)

[Definiciones](#)

[Derivaciones con segmentos iniciales](#)

[Derivaciones con segmentos finales](#)

[Derivaciones con segmentos arbitrarios](#)

[Rangos con pares de elementos de lista](#)

[Rangos con análisis por casos](#)

[Resumen de técnicas de derivación](#)

[Temas complementarios](#)

[Técnicas de autocorrección](#)

[Traducción a Haskell](#)

[Problemas Exponenciales](#)

[Técnica de Tuplas](#)

[Recursión final](#)

[Ejercicios](#)

[Programación Imperativa](#)

[Introducción](#)

[Antes: Modelo computacional de la programación funcional](#)

[Ahora: Modelo computacional de la programación imperativa](#)

[Sintaxis vs. Semántica](#)

[El lenguaje de programación Imperativa](#)

[Asignación](#)

[Condicional \(if\) y skip](#)

[Repetición / ciclo \(do\)](#)

[Cosas que NO tiene el lenguaje de programación](#)

[Ejemplo: contar múltiplos de 6](#)

[Otro ejemplo: factorial](#)

[Arreglos](#)

[Declaración \(tipo\)](#)

[Consulta / acceso \(expresión\)](#)

[Asignación para arreglos \(sentencia\)](#)

[Ejemplo: Sumar elementos de un arreglo](#)

[Testing / Ejecución](#)

[Lenguaje imperativo completo](#)

[Declaración de constantes y variables](#)

[Sentencias](#)

[Skip](#)

[Asignación \(:=\)](#)

[Expresiones](#)

[Secuenciación \(;\)](#)

[Condicional \(if\)](#)

[Repetición, ciclo o bucle \(do\)](#)

[Anotaciones de programa](#)

[Precondición y postcondición](#)

[Especificaciones](#)

[Ejemplos](#)

[Ternas de Hoare](#)

[Introducción](#)

[Variables de especificación](#)

[Definición](#)

[Observaciones](#)

[Ejemplos](#)

[Derivación vs demostración](#)

[Precondición más débil](#)

[Fortaleza y debilidad de predicados](#)

[Weakest Precondition \(WP\)](#)

[Relación entre la WP y la Terna de Hoare](#)

[WP de la asignación](#)

[WP del skip](#)

[Derivación de programas imperativos](#)

[Digesto](#)

[Skip](#)

[El skip en derivaciones](#)

[Asignación](#)

[La asignación en derivaciones](#)

[Condicional \(if\)](#)

[El condicional en derivaciones](#)

[Secuenciación \(;\)](#)

[La secuenciación en derivaciones](#)

[Ciclo / Repetición \(do\)](#)

[La repetición en derivaciones](#)

[Derivación de ciclos: Técnicas para encontrar invariantes](#)

[Idea general](#)

[1ra técnica: Tomar términos de una conjunción](#)

[2da técnica: Reemplazo de constantes por variables](#)

[Ejemplo: Suma de los elementos de un arreglo:](#)

[Otro ejemplo: Exponenciación: Dados \$X > 0\$, e \$Y \geq 0\$, calcular \$XY\$.](#)

[Ejemplo: Factorial de un número N: Dado \$N \geq 0\$ quiero calcular el factorial de N.](#)

[Ejemplo: De nuevo suma de los elementos de un arreglo:](#)

[Ejemplo: Promedio de los elementos de un arreglo:](#)

[Ciclos anidados](#)

[Ejemplo](#)

[Otro ejemplo](#)

[Fortalecimiento de invariantes](#)

[Ejemplo](#)

[Análisis de complejidad](#)

[Ejemplo \(fibonacci\)](#)

[Terminación de ciclos: Función de cota](#)

[Dedución de la cota](#)

[Demostración formal de la cota](#)
[Problemas de bordes](#)
[Ejercicio con segmentos iniciales](#)
[¿Qué pasa si fortalecemos mal?](#)
[Ejercicio con segmentos arbitrarios](#)
[Segmento de suma máxima](#)
[Segmento de suma máxima sin Segmentos vacíos](#)
[Especificaciones con segmentos de arreglo](#)
[Temas complementarios](#)
[Terminación anticipada de ciclos](#)
[Traducción a C](#)
[Ejercicios](#)
[Resoluciones de ejercicios](#)
[Ejercicio de final con varias cosas interesantes](#)
[Referencias](#)
[Apéndice](#)
[Ejercicios](#)
[Digestos](#)
[Digesto de Funciones de Listas y Propiedades](#)
[Digesto de Cuantificadores y Cálculo Proposicional](#)
[Cuantificador de Conteo N](#)
[Digesto para la Programación Imperativa](#)
[Videos](#)
[Contenido extra I: Clases prácticas y consultas](#)
[Funcional / Cuantificación general \(2021\)](#)
[Imperativo \(2021\)](#)
[Clases de consulta \(2021\)](#)
[Clases de consulta \(2023\)](#)
[Clases de consulta \(2024\)](#)
[Clases de consulta \(2025\)](#)
[Exámenes parciales \(2016 – 2019\)](#)
[Solución 1er parcial turno tarde \(2024\)](#)
[Solución 2do parcial turno tarde \(2024\)](#)
[Solución examen final 17/12/2024](#)
[Ejercicio 1:](#)
[Ejercicio 2:](#)
[Calendario sugerido](#)

Introducción

En este documento se encuentra todo el contenido teórico y práctico de la materia Algoritmos y Estructuras de Datos I de la [FAMAF](#), UNC ([Universidad Nacional de Córdoba](#)), Argentina.

Todo este contenido fue creado con el apoyo de la **Universidad Pública y Gratuita** a través de la UNC, y del **Sistema Nacional de Ciencia y Tecnología** a través del [CONICET](#).

Un poco de historia

En los inicios de las computadoras, la programación era una tarea subestimada, y los programas eran escritos con el “**método**” de prueba y error (**code & fix**). No existían metodologías y procesos estructurados para el desarrollo de software. En cambio, los programadores iban directo a escribir el código que debía resolver su problema o necesidad. Una vez que el programa estaba escrito, era cuestión de probarlo y corregirlo hasta que funcione de la manera esperada.

Esta manera de programar resultaba muy efectiva ya que las computadoras no eran en ese momento muy poderosas, y por lo tanto no eran muy complejos los problemas que podían resolver.

Con el aumento de la potencia de las computadoras y por ende de la complejidad de los programas, empezó a ser evidente que el *code & fix* no era suficiente para el desarrollo de proyectos exitosos. El mundo entró en lo que se denominó la [Crisis del software](#), ya que muchos grandes proyectos empezaron a fracasar, provocando pérdidas millonarias y hasta vidas humanas.

La crisis del software dio lugar a la creación de la **Ingeniería del Software** como una nueva disciplina que estudia el **proceso de creación y mantenimiento del software**. En este marco se desarrollaron investigaciones y experiencias que llevaron a la creación de numerosas metodologías para todos los aspectos que comprende el proceso de desarrollo del software.

Ingeniería del Software: Programación a gran escala

La Ingeniería del Software trata del **proceso completo de desarrollo del software** desde el mismo surgimiento de la necesidad o problema a resolver hasta las últimas etapas de gestión y mantenimiento del software ya terminado y en uso. Por lo tanto, no abarca solamente cuestiones relacionadas con la técnica, las computadoras y la programación, sino que también abarca aspectos sociales como la comunicación con el cliente y el manejo de equipos enteros de personas.

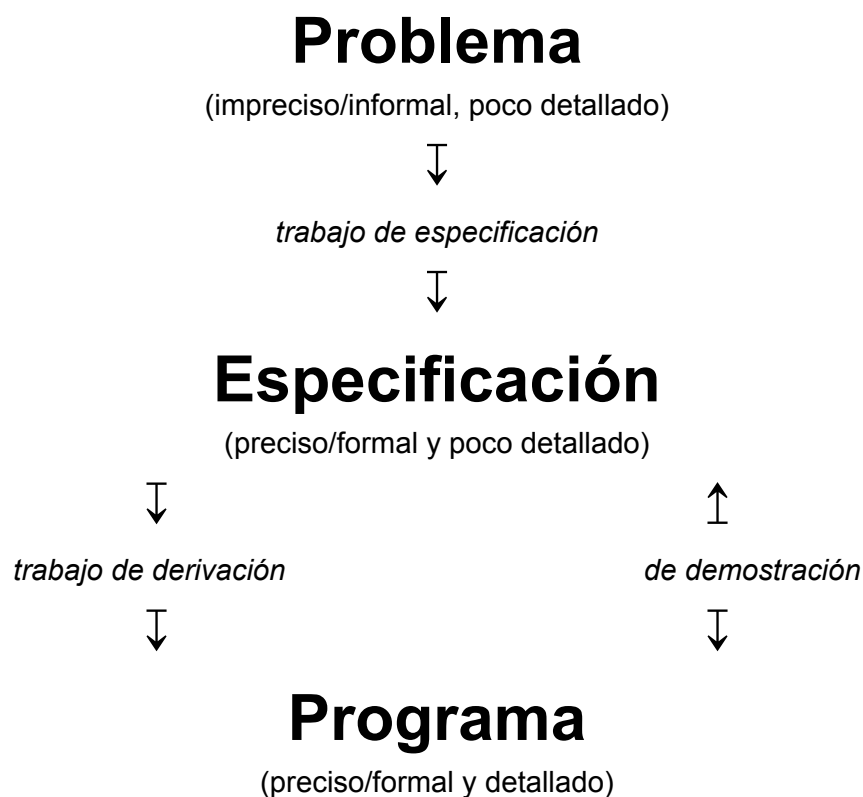
Hay infinidad de libros y artículos publicados sobre Ingeniería del Software, y existen procesos de todo tipo para ser aplicados en proyectos de diferentes características.

La aplicación cuidadosa de un proceso de desarrollo se vuelve fundamental sobre todo en la gestión de proyectos grandes, ambiciosos, complejos y/o de alto riesgo, lo que podríamos llamar “**programación a gran escala**”¹. En contraste, podemos llamar “**programación a pequeña escala**” a la creación de pequeñas piezas de software o programas, a veces para resolver problemas sencillos y otras veces como parte de un proceso más grande de desarrollo.

Algoritmos I: Programación a pequeña escala (“in the small”)

En esta materia nos vamos a ocupar de aprender programación a pequeña escala, esto es, la escritura de pequeños algoritmos para la resolución de problemas relativamente simples. Estos problemas serán siempre problemas de cálculo de uno o más resultados a partir de unos datos de entrada. **No habrá entonces ningún tipo de interacción o interactividad de nuestro programa.**

El proceso de construcción de programas



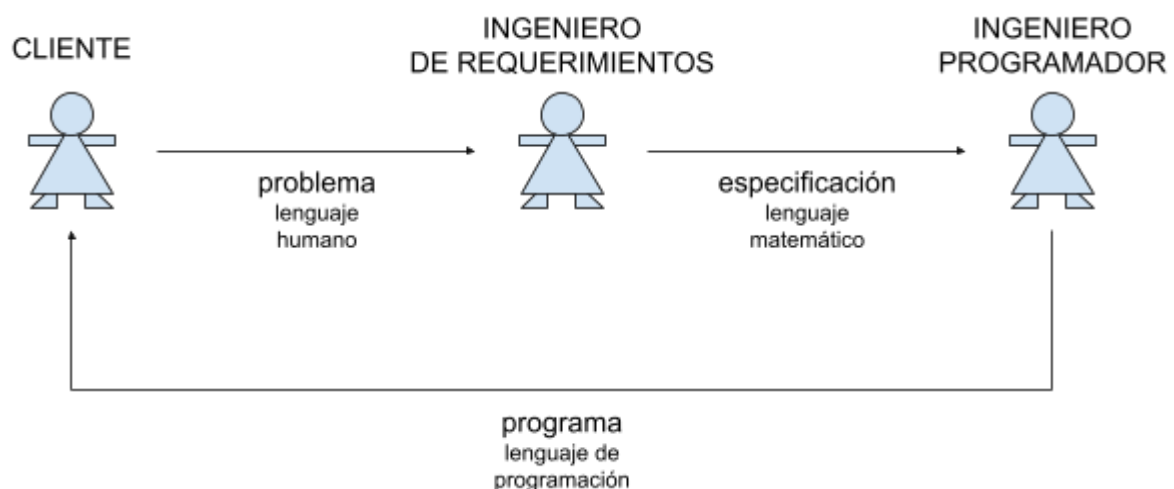
¹ https://en.wikipedia.org/wiki/Programming_in_the_large_and_programming_in_the_small

Problemas: Son tareas que se desean resolver usando computadoras. En el marco de esta materia, expresaremos los problemas en **lenguaje humano escrito**, y los problemas van a tratar de obtener ciertos valores de resultado que se quieren obtener a partir de ciertos datos iniciales o de entrada.

Especificaciones: Son formas precisas y formales de expresar qué debería cumplir un programa para ser solución del problema que se desea solucionar. Para especificar se usan herramientas matemáticas que permiten expresar muchas “fórmulas” o “cuentas” de manera simple y breve. Una especificación siempre va a mencionar el programa solución diciendo **qué hace el programa, pero nunca cómo lo hace**. En general no será posible que una computadora entienda o ejecute una especificación.

Programas: Los programas son “textos” que una computadora puede entender y ejecutar, por lo que vamos a usarlos para solucionar problemas. Un programa está escrito en un lenguaje de programación que tiene una sintaxis y una semántica. La sintaxis es la forma de escribir los programas o, dicho de otra manera, el conjunto de reglas que define qué textos son programas válidos y cuáles no. La semántica define qué significan los programas, esto es, qué sucede cuando se ejecutan, y cómo las distintas partes del texto se traducen en acciones de la computadora.

Roles básicos en un proyecto de software



Cliente:

- persona común y corriente (tu tía comerciante)
- tiene un problema recurrente que quiere resolver

Ingeniero de requerimientos:

- genio de las matemáticas
- hábil comunicador
- (casi) no sabe programar

Ingeniero programador:

- genio de las computadoras y de la programación
- no tantas habilidades sociales

Estructura del curso

La materia está dividida en tres partes:

1. Cuantificación general
2. Programación funcional
3. Programación imperativa

Cuantificación general: En esta parte presentamos una herramienta matemática muy útil para escribir **especificaciones** precisas y formales de muchos problemas. Esta herramienta se suma a otras herramientas ya conocidas como la lógica proposicional, la lógica de primer orden ($\forall$, $\exists$) y muchos otros recursos matemáticos que podrán ser usados a la hora de especificar.

Programación funcional: En esta parte presentamos un lenguaje de programación de tipo funcional, esto es, centrado en la definición de funciones y cuyo modelo computacional es el del cálculo/reducción de expresiones. Además, presentamos una forma de especificar problemas de manera funcional, y técnicas de derivación que permiten obtener programas funcionales a partir de especificaciones.

Programación imperativa: En esta parte presentamos un lenguaje de programación de tipo imperativo, cuyo modelo computacional es la definición de un estado y la modificación del mismo a través de la ejecución de sentencias. Presentamos también una forma de especificar problemas de manera imperativa, y técnicas de derivación que permiten obtener programas imperativos a partir de especificaciones.

Preliminares

En esta sección se mencionan brevemente contenidos previos que son necesarios para comprender la materia. Todos estos contenidos corresponden a materias previamente cursadas, especialmente a Introducción a los Algoritmos.

Cálculo proposicional (lógica de orden cero)

Repasar, tomando las cosas que más nos sirven (en particular: Leibniz 2).

Ver sección de “Cálculo Proposicional” del [digesto.pdf](#)

Qué cosas de Intro no vamos a usar:

- **Regla dorada. ¡No sirve para nada!**

Cálculo de predicados (lógica de primer orden)

Repasar para todo ($\forall$) y existe ($\exists$).

Tipos básicos

- Booleanos: Tipo **Bool**. Valores: True y False. Operaciones: ...
- Enteros: Tipo **Int**.
- Naturales: Tipo **Nat**.
- Decimales: Tipo **Float**.
- Números en general: Tipo **Num**. Lo usamos para decir que pueden ser **Int**, **Nat** o **Float**.
- Caracteres: Tipo **Char**. Valores: 'a', '6', '#', etc.

Listas: funciones y propiedades

[listas.pdf](#)

Qué cosas de Intro no vamos a usar para nada:

- Función “pertenece” para listas

Demostraciones

Escribiremos demostraciones usando una notación que nos permite explicar qué estamos haciendo en cada paso: qué propiedades, axiomas, teoremas y/o hipótesis estamos aplicando.

Equivalencia entre dos predicados

Para demostrar $P \equiv Q$:

$$\begin{array}{l} P \\ \equiv \{ \text{paso 1} \} \\ P' \\ \equiv \{ \text{paso 2} \} \\ \dots \\ \equiv \{ \text{paso n} \} \\ Q \end{array}$$

Igualdad entre dos expresiones

Cuando tenemos expresiones que no son predicados usamos “=” en lugar de “ $\equiv$ ”.

Para demostrar $E = F$:

$$\begin{array}{l} E \\ = \{ \text{paso 1} \} \\ E' \\ = \{ \text{paso 2} \} \\ \dots \\ = \{ \text{paso n} \} \\ F \end{array}$$

Implicaciones

Para demostrar $P \Rightarrow Q$ en general vamos a **suponer P como hipótesis** y vamos a demostrar que vale Q:

$$\begin{array}{l} Q \\ \equiv \{ \text{paso 1 (se puede usar hipótesis P)} \} \\ Q' \\ \equiv \{ \text{paso 2 (se puede usar hipótesis P)} \} \\ \dots \\ \equiv \{ \text{paso n} \} \\ \text{True} \end{array}$$

Otra forma (menos común) de demostrar la implicación es partiendo de P y llegando a Q donde algunos pasos se conectan con $\equiv$ y otros con $\Rightarrow$:

P
 $\equiv \{ \text{paso 1} \}$
P'
 $\Rightarrow \{ \text{paso 2: en este paso vale } P' \Rightarrow P'' \}$
P''
 $\equiv \{ \text{paso 3} \}$
...
 $\equiv \{ \text{paso n} \}$
Q

Análisis por casos

A veces necesitamos dividir la demostración en varios casos posibles. Cada caso es una rama de la demostración.

Por ejemplo, para $P \equiv Q$:

P
 $\equiv \{ \text{paso 1} \}$
P'

- Caso 1: vale condición C_1
 $\equiv \{ \text{paso 1.2 (puedo usar nueva hipótesis } C_1) \}$
...
 $\equiv \{ \text{paso 1.n} \}$
Q
- Caso 2: vale condición C_2
 $\equiv \{ \text{paso 2.2 (puedo usar nueva hipótesis } C_2) \}$
...
 $\equiv \{ \text{paso 2.n} \}$
Q

Diferencia entre igualdad (=) y equivalencia ($\equiv$)

Equivalencia ($\equiv$): Se usa solamente para **predicados**, es decir, **expresiones booleanas**. Solamente se puede escribir " $A \equiv B$ " si tanto A como B son predicados.

Igualdad (=): Se puede usar en **cualquier caso**. " $A = B$ " se puede escribir siempre que A y B sean expresiones del mismo tipo. Sin embargo, si sabemos que A y B son predicados preferimos usar la equivalencia ($\equiv$) ya que se habilitan las propiedades específicas de la equivalencia.

Cuantificación General

Introducción

TODO:

- sumatoria y productoria como en álgebra I

Sintaxis

$$\langle \oplus i : R.i : T.i \rangle$$

- $\oplus$ representa la operación que está siendo cuantificada.

Observaciones:

1. Siempre se trata de operaciones binarias: **Tipo A $\rightarrow$ A $\rightarrow$ A.** (toma dos cosas de tipo A y devuelve una de tipo A).
2. La operación debe ser asociativa y conmutativa.

Ejemplos: suma (+ ó Σ), producto (* ó $\prod$), conjunción ($\wedge$ ó $\forall$), disyunción ($\vee$ ó $\exists$), máximo, mínimo, etc.

Ejemplos que no: negación (no es binaria), resta (no es conmutativa), división (no es conmutativa).

- **i:** Nombres de las **variables cuantificadas**. (ponemos sólo “i” pero pueden ser varias). Cada variable va a tener un tipo asociado. En general no vamos a escribir los tipos explícitamente sino que se van a poder inferir por el contexto en el que se usan las variables.
- **R.i:** **El rango**. Es un predicado (tipo Bool) sobre las variables cuantificadas. Define el conjunto de valores posibles que van a tomar las variables al ser aplicadas al término.
- **T.i:** **El término**. Es una expresión de tipo A (a donde A es el tipo de la operación). Define las expresiones que luego serán aplicadas a los valores definidos por el rango para luego ser “juntadas” (operadas) con la operación cuantificada $\oplus$, para finalmente obtener un único valor de tipo A.

Lectura Operacional

¿Cómo calcular el resultado de una expresión cuantificada?

0. Identificar los tipos de las variables y el **universo de cuantificación**.
La variable cuantificada i es de tipo A.
1. Calcular el conjunto de valores posibles para las variables cuantificadas de **acuerdo al rango R**.
Los valores posibles para i son $\{v_1, v_2, \dots, v_n\}$
2. Aplicar el término T a todos los elementos del conjunto de valores posibles.
 $T.v_1$

$T.v_2$

$\dots$

$T.v_n$

3. “Juntar todo” con la operación cuantificada:

$$T.v_1 \oplus T.v_2 \oplus \dots \oplus T.v_n$$

Ejemplos

Ejemplo:

$$\langle \prod i : 0 \leq i < 5 : i * 2 + 1 \rangle$$

0. El tipo de i es Int (y también el universo de cuantificación es Int).

1. El rango es: $i \in \{0, 1, 2, 3, 4\}$.
2. Los términos aplicados son:

$$0 * 2 + 1$$

$$1 * 2 + 1$$

$$2 * 2 + 1$$

$$3 * 2 + 1$$

$$4 * 2 + 1$$

3. Al operar (“juntar todo”), nos queda:

$$(0 * 2 + 1) * (1 * 2 + 1) * (2 * 2 + 1) * (3 * 2 + 1) * (4 * 2 + 1)$$

Otro ejemplo:

$$\langle \prod i : 0 \leq i < 5 \wedge 2 < i \leq 10 : 37 \rangle$$

1. El rango es: $i \in \{3, 4\}$
2. Los términos aplicados son:

$$37$$

$$37$$

3. Al operar (“juntar todo”), nos queda:

$$37 * 37$$

Ejemplos con dos variables cuantificadas

Se usan tuplas para definir el conjunto de valores posibles.

Ejemplo: “Todos los elementos de la lista xs son distintos.”

$$\langle \forall i, j : 0 \leq i < \#xs \wedge 0 \leq j < \#xs \wedge i \neq j : xs[i] \neq xs[j] \rangle$$

Lectura operacional con $xs = [2, 7, -1, 7]$:

1. El rango es: $(i, j) \in \{(0, 1), (0, 2), (0, 3), (1, 0), (1, 2), (1, 3), (2, 0), (2, 1), (2, 3), (3, 0), (3, 1), (3, 2)\}$

2. Son 12 términos (ejercicio: aplicarlos y resolver)

$i \setminus j$	0	1	2	3
0	(0, 0)	(0, 1)	(0, 2)	(0, 3)
1	(1, 0)	(1, 1)	(1, 2)	(1, 3)
2	(2, 0)	(2, 1)	(2, 2)	(2, 3)
3	(3, 0)	(3, 1)	(3, 2)	(3, 3)

Hay muchos términos repetidos: (0,1) y (1,0) dicen lo mismo.

Otra forma quitando términos repetidos: Ponemos $i < j$ en lugar de $i \neq j$.

$$\langle \forall i, j : 0 \leq i < \#xs \wedge 0 \leq j < \#xs \wedge \underline{i < j} : xs[i] \neq xs[j] \rangle$$

Equivalente, más corto y más bonito:

$$\langle \forall i, j : 0 \leq i < j < \#xs : xs[i] \neq xs[j] \rangle$$

$i \setminus j$	0	1	2	3
0	(0, 0)	(0, 1)	(0, 2)	(0, 3)
1	(1, 0)	(1, 1)	(1, 2)	(1, 3)
2	(2, 0)	(2, 1)	(2, 2)	(2, 3)
3	(3, 0)	(3, 1)	(3, 2)	(3, 3)

Quedan 6 términos.

Otro ejemplo:

$$\langle \exists i, j : 0 \leq i < 5 \wedge i \bmod 2 = 0 \wedge |j - i| = 1 : i + j \rangle$$

0. Ambas variables son Int, el universo de cuantificación es Int x Int (o sea pares de enteros).

1. El rango es: $(i, j) \in \{ (0, -1), (0, 1), (2, 1), (2, 3), (4, 3), (4, 5) \}$ (6 elementos)
2. Los términos aplicados son: **ejercicio!!**
3. Resultado: **Ejercicio!!**

Axiomas y Teoremas de Cuantificadores

Digesto

[digesto.pdf](#)

A1: Rango vacío

$$\langle \oplus i : \text{False} : T.i \rangle = e$$

- donde e es el neutro $\oplus (e \oplus X = X)$

¿Qué pasa cuando no hay ningún valor posible para las variables cuantificadas que satisfaga el rango?

En la lectura operacional: El rango es el conjunto vacío.

Ejemplo:

$\langle \text{Max } i : i \bmod 2 = 0 \wedge \underline{i > 10 \wedge i < 12} : i * 2 \rangle$
= { propiedad de los enteros que me dice que $i > 10 \wedge i < 12$ es lo mismo que $i = 11$ }
 $\langle \text{Max } i : \underline{i \bmod 2 = 0} \wedge i = 11 : i * 2 \rangle$
= { reemplazo de iguales por iguales en la conjunción (Leibniz 2) }
 $\langle \text{Max } i : \underline{11 \bmod 2 = 0} \wedge i = 11 : i * 2 \rangle$
= { $11 \bmod 2$ es 1 }
 $\langle \text{Max } i : \underline{1 = 0} \wedge i = 11 : i * 2 \rangle$
= { lógica }
 $\langle \text{Max } i : \text{False} \wedge i = 11 : i * 2 \rangle$
= { lógica (absorvente de $\wedge$) }
 $\langle \text{Max } i : \text{False} : i * 2 \rangle$
= { rango vacío (A1), ya que - infinito es el neutro de max }
- infinito

A2: Rango unitario

$$\langle \oplus i : i = C : T.i \rangle = T.C$$

¿Qué pasa cuando la variable cuantificada tiene sólo un valor posible?

En la lectura operacional: El rango es un conjunto de un solo elemento.

Ejemplo (casi igual al anterior, pero con i impar):

$\langle \text{Max } i : \underline{i \bmod 2 = 1} \wedge i = 11 : i * 2 \rangle$
= { reemplazo de iguales por iguales en la conjunción (Leibniz) }
 $\langle \text{Max } i : \underline{11 \bmod 2 = 1} \wedge i = 11 : i * 2 \rangle$
= { $11 \bmod 2$ es 1 }

$$\begin{aligned}
& \langle \text{Max } i : \underline{1 = 1} \wedge i = 11 : i * 2 \rangle \\
& = \{ \text{lógica} \} \\
& \langle \text{Max } i : \text{True} \wedge i = 11 : i * 2 \rangle \\
& = \{ \text{lógica (neutro de } \wedge) \} \\
& \langle \text{Max } i : i = 11 : i * 2 \rangle \\
& = \{ \text{rango unitario (A2)} \} \\
& \quad 11 * 2
\end{aligned}$$

Ejemplo:

$$\langle \sum i, j, k : i + j = 10 \wedge j + 2 * k = 10 \wedge i + k = 9 : i * (j + 1) * (k - 1) \rangle$$

Lectura operacional:

0. El universo de cuantificación es $\text{Int} \times \text{Int} \times \text{Int}$ (tripas o ternas o tuplas de 3 elementos).

1. El rango es: $(i, j, k) \in \{ (6, 4, 3) \}$ (ejercicio: chequear! es un sistema de tres ecuaciones con tres incógnitas)
2. Aplicar el término una sola vez:

$$6 * (4 + 1) * (3 - 1)$$

3. Juntar todo (hay una sola cosa):

$$6 * (4 + 1) * (3 - 1)$$

Demostración:

$$\begin{aligned}
& \langle \sum i, j, k : \underline{i + j = 10 \wedge j + 2 * k = 10 \wedge i + k = 9} : i * (j + 1) * (k - 1) \rangle \\
& = \{ \text{resuelvo el sistema de ecuaciones, reemplazo predicados equivalentes} \} \\
& \langle \sum i, j, k : i = 6 \wedge j = 4 \wedge k = 3 : i * (j + 1) * (k - 1) \rangle \\
& = \{ \text{armo la tupla para que tenga la forma de rango unitario} \} \\
& \langle \sum i, j, k : (i, j, k) = (6, 4, 3) : i * (j + 1) * (k - 1) \rangle \\
& = \{ \text{rango unitario} \} \\
& \quad 6 * (4 + 1) * (3 - 1) \\
& = \{ \text{aritmética} \} \\
& \quad 60
\end{aligned}$$

A3: Partición de rango

$$\begin{aligned}
& \langle \oplus i : R.i \vee S.i : T.i \rangle \\
& = \langle \oplus i : R.i : T.i \rangle \oplus \langle \oplus i : S.i : T.i \rangle
\end{aligned}$$

siempre que suceda la menos una de estas dos cosas:

- $\oplus$ es idempotente
- R y S son disjuntos

Ejemplo:

$$\langle \forall i : \underline{(-1 \leq i < 1) \vee (8 \leq i < 11)} : T.i \rangle \quad (A)$$

¿es esto igual que

$$\langle \forall i : (-1 \leq i < 1) : T.i \rangle \wedge \langle \forall i : (8 \leq i < 11) : T.i \rangle \quad (B)$$

? ¡Sí! Veamos la lectura operacional:

(A): El rango es $\{-1, 0, 8, 9, 10\}$. El resultado es:

$$(T.(-1) \wedge T.0 \wedge T.8 \wedge T.9 \wedge T.10)$$

que es lo mismo que:

$$(T.(-1) \wedge T.0) \wedge (T.8 \wedge T.9 \wedge T.10)$$

(B): Tengo dos expresiones cuantificadas conjugadas entre sí.

La primera es $(T.(-1) \wedge T.0)$

La segunda es $(T.8 \wedge T.9 \wedge T.10)$

Luego me queda exactamente lo mismo que (A):

$$(T.(-1) \wedge T.0) \wedge (T.8 \wedge T.9 \wedge T.10)$$

Otro ejemplo:

$$\langle \forall i : (-1 \leq i < 2) \vee (0 \leq i < 4) : T.i \rangle \quad (A)$$

¿es esto igual que

$$\langle \forall i : (-1 \leq i < 2) : T.i \rangle \wedge \langle \forall i : (0 \leq i < 4) : T.i \rangle \quad (B)$$

? ¡Sí! Veamos:

Acá, para (A) el rango es $\{-1, 0, 1, 2, 3\}$. El resultado es:

$$(T.(-1) \wedge T.0 \wedge T.1 \wedge T.2 \wedge T.3)$$

Para (B) tengo dos rangos: $\{-1, 0, 1\}$ y $\{0, 1, 2, 3\}$. Luego. el resultado de toda la expresión es:

$$(T.(-1) \wedge T.0 \wedge T.1) \wedge (T.0 \wedge T.1 \wedge T.2 \wedge T.3)$$

¿Es esto lo mismo que A? Sí, pero sólo gracias a que la conjunción es idempotente. Me queda:

$$(T.(-1) \wedge T.0 \wedge T.1 \wedge T.2 \wedge T.3)$$

Otro ejemplo:

$$\langle \sum i : (-1 \leq i < 2) \vee (0 \leq i < 4) : T.i \rangle \quad (A)$$

¿es esto igual que

$$\langle \sum i : (-1 \leq i < 2) : T.i \rangle + \langle \sum i : (0 \leq i < 4) : T.i \rangle \quad (B)$$

? ¡No! Ya que (A) es

$$(T.(-1) + T.0 + T.1 + T.2 + T.3)$$

Y (B) es

$$(T.(-1) + T.0 + T.1) + (T.0 + T.1 + T.2 + T.3)$$

$T.0$ y $T.1$ aparecen repetidos pero no se pueden eliminar porque la suma no es idempotente.

¿En qué caso sí se podría aplicar la partición de rango si el operador no es idempotente?

En el caso en el que no aparecen términos repetidos, es decir que los predicados R y S son disjuntos entre sí (no hay ningún valor que satisfaga ambos al mismo tiempo).

A4: Regla del término

$$\langle \oplus i : R.i : T.i \oplus U.i \rangle = \langle \oplus i : R.i : T.i \rangle \oplus \langle \oplus i : R.i : U.i \rangle$$

En el término tengo dos sub-términos operados entre sí.

En lectura operacional:

La expresión de la izquierda es:

$$(T.v_1 \oplus U.v_1) \oplus \dots \oplus (T.v_n \oplus U.v_n)$$

La expresión de la derecha es:

$$(T.v_1 \oplus \dots \oplus T.v_n) \oplus (U.v_1 \oplus \dots \oplus U.v_n)$$

¿Son equivalentes estas dos expresiones? Sí. ¿Por qué? Porque es sólo un reacomodo de los términos usando asociatividad y conmutatividad.

Ejemplo:

$$\langle \prod i : i \bmod 2 = 1 \wedge |i - 10| \leq 3 : i * (10 - i) \rangle$$

= { regla del término (A4) con $T.i = i$, $U.i = 10 - i$ }

$$\langle \prod i : i \bmod 2 = 1 \wedge |i - 10| \leq 3 : i \rangle * \langle \prod i : i \bmod 2 = 1 \wedge |i - 10| \leq 3 : (10 - i) \rangle$$

Lectura operacional:

1. Rango: $i \in \{7, 9, 11, 13\}$

2. (y 3.). Aplicamos término y operamos:

$$(7 * (10 - 7)) * (9 * (10 - 9)) * (11 * (10 - 11)) * (13 * (10 - 13))$$

$$= 7 * 3 * 9 * 1 * 11 * (-1) * 13 * (-3)$$

$$= \dots$$

Ejercicio: hacer la lectura operacional de la expresión que queda luego de aplicar el axioma.

A5: Término constante

$$\langle \oplus i : R.i : C \rangle = C$$

El término de la expresión cuantificada no menciona a las variables cuantificadas.

¿Cuándo vale esta propiedad? **Sólo vale cuando el operador $\oplus$ es idempotente en el valor C.**

Además, **el rango debe ser no vacío**. Si no, quedaría el elemento neutro.

En lectura operacional: Si el rango es $\{v_1, \dots, v_n\}$, tengo los siguientes n términos:

$$C \oplus C \oplus \dots \oplus C$$

(n veces)

¿Cuándo sucede que esto es lo mismo que C? Confirmamos: cuando $\oplus$ es idempotente en ese valor específico C.

Ejemplo 1:

$$\langle \forall xs : \#xs = 2 \wedge (xs!0) * (xs!1) = 1 : \underline{xs = [] \vee xs \neq []} \rangle$$

$\equiv$ { tercero excluido }

$$\langle \forall xs : \#xs = 2 \wedge (xs!0) * (xs!1) = 1 : \text{True} \rangle$$

$\equiv$ { término constante }

True

Lectura operacional: sólo para seguir jugando con los rangos.

0. Universo de cuantificación: Es el de las listas de números enteros.

1. Rango: $xs \in \{ [1, 1], [-1, -1] \}$
2. (y 3.) Aplicamos término y operamos: $\text{True} \wedge \text{True}$

Ejemplo 2 (ejemplo que no funciona):

$\langle \prod xs : \#xs = 2 \wedge (xs!0) * (xs!1) = 1 : \#xs \rangle$

= { sustitución: por el rango, sé que $\#xs = 2$ }

$\langle \prod xs : \#xs = 2 \wedge (xs!0) * (xs!1) = 1 : 2 \rangle$

= { si valiera término constante, podría decir: }

2

Pero no ya que da (haciendo lectura operacional): $2 * 2 = 4$.

Si hubiera tenido:

$\langle \prod xs : \#xs = 2 \wedge (xs!0) * (xs!1) = 1 : 1 \rangle$

Ahí sí se puede aplicar término constante ya que $*$ es idempotente en el valor 1.

Quedaría: 1.

Ejemplo 3 (otro que no funciona):

$\langle \sum i : 3 \leq i \leq 6 : 24 \rangle$

¿Cuánto da?

Lectura operacional:

1. Rango: $i \in \{ 3, 4, 5, 6 \}$.
2. Aplico el término a los valores del rango:

24

24

24

24

(lo que estoy haciendo es sustituir en la expresión del término cada mención de i por su valor) (lo que pasa es que i no se menciona en el término, entonces queda siempre lo mismo).

3. Juntamos todo con la suma:

$24 + 24 + 24 + 24 = 96$

Luego, **no vale la regla de término constante (no dió 24)**.

A6: Distributividad a derecha

$\langle \oplus i : R.i : T.i \otimes C \rangle = \langle \oplus i : R.i : T.i \rangle \otimes C$

Requisitos:

- C es constante (las variables cuantificadas no aparecen mencionadas en C)
- $\oplus$ y $\otimes$ son dos operaciones de tipo $\mathbf{A} \rightarrow \mathbf{A} \rightarrow \mathbf{A}$.
- $\otimes$ distribuye con $\oplus$ a derecha: $(x \otimes y) \oplus (z \otimes y) = (x \oplus z) \otimes y$
- R es no vacío, o bien el neutro de $\oplus$ es absorbente para $\otimes$.

Lectura operacional:

La expresión de la izquierda es: si el rango es $\{v_1, \dots, v_n\}$:

$$(T.v_1 \otimes C) \oplus \dots \oplus (T.v_n \otimes C)$$

La expresión de la derecha es:

$$(T.v_1 \oplus \dots \oplus T.v_n) \otimes C$$

¿Cuándo sucede que estas dos expresiones son equivalentes? Confirmamos que debe darse la siguiente **propiedad de distributividad**:

$$(x \otimes y) \oplus (z \otimes y) = (x \oplus z) \otimes y \quad \text{"saco factor común y"}$$

Formalmente, estamos diciendo que $\otimes$ distribuye con $\oplus$ a derecha.

¿Qué pasa si tengo rango vacío?

La expresión de la izquierda me queda el elemento neutro de la operación $\oplus$ (llamémoslo $e_\oplus$):

$$e_\oplus$$

La expresión de la derecha me queda:

$$e_\oplus \otimes C$$

¿qué debe suceder para que estas dos cosas sean iguales? $e_\oplus = e_\oplus \otimes C$

Si pasa esto, quiere decir que el neutro de $\oplus$ ($e_\oplus$) es absorbente para la operación $\otimes$.

Conclusión: si tengo rango vacío, para poder aplicar distributividad debe suceder que el neutro de $\oplus$ es absorbente para $\otimes$.

Ejemplo (ejercicio 12c):

$$\langle \forall i: i=0 \vee 4 > i \geq 1 : \neg f.i \vee \neg f.n \rangle$$

$\equiv \{ \oplus \text{ es } \wedge, \otimes \text{ es } \vee, C \text{ es } \neg f.n, \text{ luego puedo aplicar distributividad ya que se cumple todos los requisitos} \}$

$$\langle \forall i: i=0 \vee 4 > i \geq 1 : \neg f.i \rangle \vee \neg f.n$$

Ejemplo (ejercicio 12d):

$$\langle \text{Max } i: 0 \leq i < \#xs : \underline{k + xs[i]} \rangle$$

$= \{ \text{conmutatividad} \}$

$$\langle \text{Max } i: 0 \leq i < \#xs : xs[i] + k \rangle$$

$= \{ \oplus \text{ es max}, \otimes \text{ es } +, C \text{ es } k, \text{ ¿distribuyen?} \}$

$$(x + y) \text{ max } (z + y) = (x \text{ max } z) + y. \text{ Sí vale, siempre.}$$

¿es el rango no vacío? no sabemos, si $xs = []$, el rango es vacío.

luego, debe suceder que el neutro de max (-infinito) es absorbente para + (la suma), o sea que **(-infinito) + x = - infinito**.

En esta materia vamos a asumir que esto vale, así que vamos a permitir distribuir.

}

$$\langle \text{Max } i: 0 \leq i < \#xs : xs[i] \rangle + k$$

Observación importante acerca de los rangos: Si xs es vacío, podemos ver que:

$$0 \leq i < \#xs$$

$\equiv \{ \text{sustituyo } xs \text{ por } [] \}$

$$0 \leq i < \#[]$$

$\equiv \{ \text{def de } \# \}$

$$0 \leq i < 0$$

$\equiv \{ \text{lógica (no existe } i \text{ tal que es al mismo tiempo } \geq 0 \text{ y } < 0) \}$
False

MUCHO OJO CON LAS DESIGUALDADES: MENOR/MAYOR Estricto (<, >) Y MENOR/MAYOR IGUAL ($\leq$, $\geq$) SON COSAS MUY DISTINTAS, NUNCA “DA LO MISMO” USAR UNA U OTRA.

A6': Distributividad a izquierda

Igual que A6 pero con la distributividad al revés:

$$\langle \oplus i : R.i : C \otimes T.i \rangle = C \otimes \langle \oplus i : R.i : T.i \rangle$$

A7: Anidado

$$\langle \oplus i, j : R.i \wedge S.i.j : T.i.j \rangle = \langle \oplus i : R.i : \langle \oplus j : S.i.j : T.i.j \rangle \rangle$$

Aclaraciones:

- R.i no menciona a “j”, puede mencionar a “i” pero no obligatoriamente
- S.i.j puede mencionar a “i” y “j”, pero no obligatoriamente.

Ejemplo:

$$\begin{aligned} & \langle \sum i, j : 7 \leq i < 10 \wedge i \bmod j = 0 \wedge j > 0 : T.i.j \rangle \\ &= \{ R.i \text{ es } 7 \leq i < 10, S.i.j \text{ es } i \bmod j = 0 \wedge j > 0, \text{ aplicamos anidado} \} \\ & \langle \sum i : 7 \leq i < 10 : \langle \sum j : i \bmod j = 0 \wedge j > 0 : T.i.j \rangle \rangle \end{aligned}$$

Lectura operacional del ejemplo:

El lado izquierdo:

- El rango es: $(i, j) \in \{ (7, 1), (7, 7), (8, 8), (8, 1), (8, 2), (8, 4), (9, 1), (9, 9), (9, 3) \}$
- La expresión queda:

$$T.7.1 + T.7.7 + T.8.8 + T.8.1 + \dots + T.9.3$$

El lado derecho:

- El rango de la sumatoria de afuera ($7 \leq i < 10$): $i \in \{ 7, 8, 9 \}$
- Luego, debo calcular el término para cada uno de estos valores posibles:

$$1. \langle \sum j : 7 \bmod j = 0 \wedge j > 0 : T.7.j \rangle +$$

$$2. \langle \sum j : 8 \bmod j = 0 \wedge j > 0 : \underline{T.8.j} \rangle +$$

$$3. \langle \sum j : 9 \bmod j = 0 \wedge j > 0 : T.9.j \rangle$$

1. El rango es: $j \in \{ 1, 7 \}$. Me queda:

$$T.7.1 + T.7.7$$

2. El rango es: $j \in \{ 1, 2, 4, 8 \}$. Me queda:

$$T.8.1 + T.8.2 + T.8.4 + T.8.8$$

3. El rango es: $j \in \{ 1, 3, 9 \}$. Me queda:

$$T.9.1 + T.9.3 + T.9.9$$

- Resultado del lado derecho:

$$(T.7.1 + T.7.7) + (T.8.1 + T.8.2 + T.8.4 + T.8.8) + (T.9.1 + T.9.3 + T.9.9)$$

¿Quedó igual que el lado izquierdo?

Sí, sólo reorganizando los términos. Se verifica la regla.

Caso particular: Supongamos que en el rango tengo: $0 \leq i < j < 10 \wedge i \bmod j = 0$.

Ambas partes mencionan a i y j. Pero puedo tomar:

- $R.i = \text{True}$
- $S.i.j = 0 \leq i < j < 10 \wedge i \bmod j = 0$

Y aplicar anidado de esta manera.

T1: Cambio de variable

$$\langle \oplus i : R.i : T.i \rangle = \langle \oplus j : R.(f.j) : T.(f.j) \rangle$$

Requisitos:

- f es una función **biyectiva** cuya imagen es el conjunto definido por el rango R.i.
- j no aparece mencionada en R ni en T.

Ejemplo:

$$\begin{aligned} & \langle \sum i : 24 < i \leq 29 : i * 2 - 10 \rangle \quad (A) \\ &= \{ \text{propongo usar cambio de variable con la función } f.j = j - 11 \} \\ & \langle \sum j : 24 < (f.j) \leq 29 : (f.j) * 2 - 10 \rangle \\ &= \{ \text{sustituyo f por lo que es} \} \\ & \langle \sum j : 24 < j - 11 \leq 29 : (j - 11) * 2 - 10 \rangle \\ &= \{ \text{aritmética} \} \\ & \langle \sum j : 35 < j \leq 40 : (j - 11) * 2 - 10 \rangle \quad (B) \end{aligned}$$

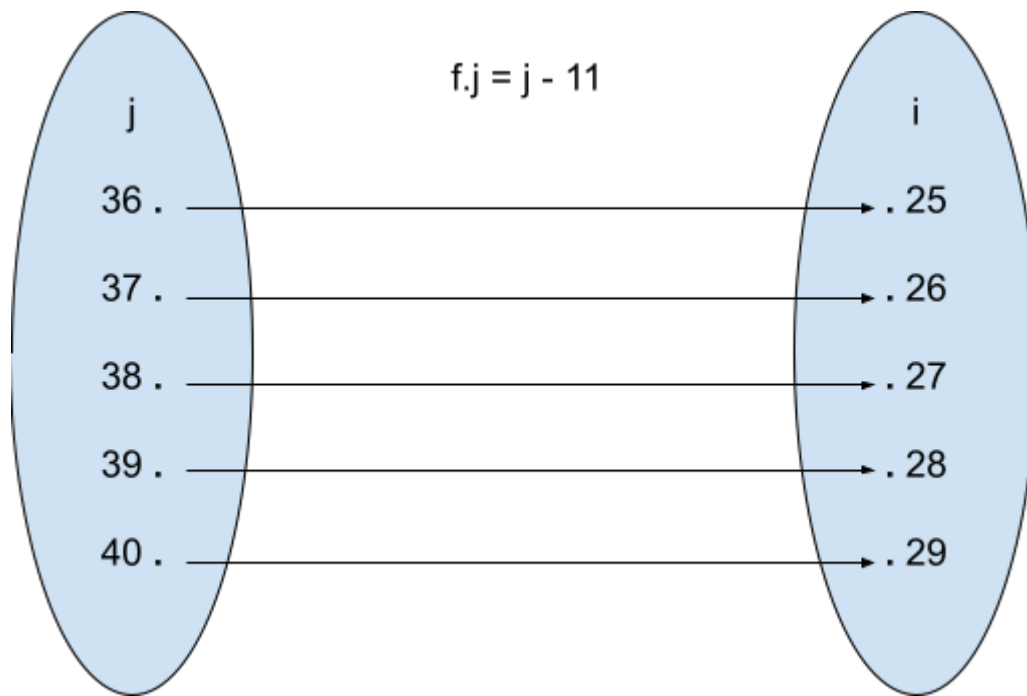
¿es igual lo de arriba de todo (A) a lo de abajo de todo (B)? Sí.

Veamos la lectura operacional:

- (A) El rango es: $i \in \{25, 26, 27, 28, 29\}$. La expresión queda:
- $$(25 * 2 - 10) + (26 * 2 - 10) + (27 * 2 - 10) + (28 * 2 - 10) + (29 * 2 - 10)$$
- (B) El rango es: $j \in \{36, 37, 38, 39, 40\}$. La expresión queda:
- $$((36 - 11) * 2 - 10) + ((37 - 11) * 2 - 10) + ((38 - 11) * 2 - 10) + ((39 - 11) * 2 - 10) + ((40 - 11) * 2 - 10)$$
- o sea:
- $$(25 * 2 - 10) + (26 * 2 - 10) + (27 * 2 - 10) + (28 * 2 - 10) + (29 * 2 - 10)$$
- que es lo mismo que (A).

¿Qué pasó? Apliqué una función **cuya imagen** (el conjunto de llegada) es conjunto definido por el rango original R.i:

- Al aplicarla en el rango, cambio el conjunto original por un nuevo conjunto que es el del dominio de la función, ya no la imagen (en este ejemplo, $\{36, 37, 38, 39, 40\}$).
- Al aplicarla en el término, estoy haciendo que cada punto vuelva a tener el valor correcto que espero usar en el término (vuelvo al rango original).



¿Cuándo se rompe esto?

Ejemplo con función no inyectiva y no sobreyectiva:

$$f.j = j * j$$

Lado izquierdo: $\langle \sum i : 24 < i \leq 29 : i * 2 - 10 \rangle$ (A)

Lado derecho: $\langle \sum j : 24 < j * j \leq 29 : (j * j) * 2 - 10 \rangle$ (B')

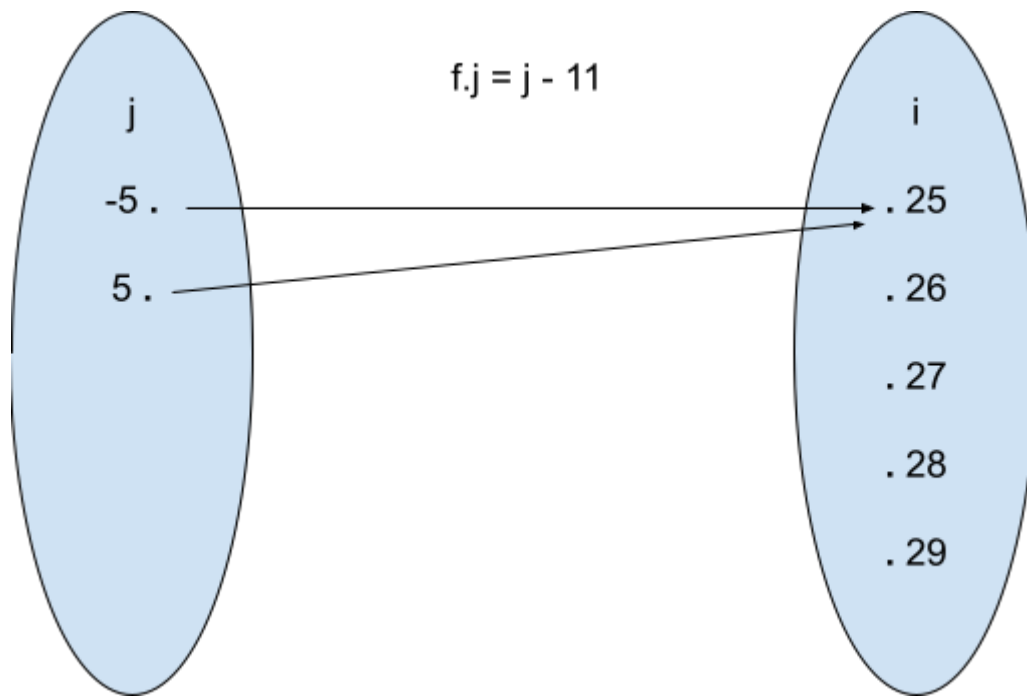
Lectura operacional para A:

$$(25 * 2 - 10) + (26 * 2 - 10) + (27 * 2 - 10) + (28 * 2 - 10) + (29 * 2 - 10)$$

Lectura operacional para B':

- El rango es $j \in \{5, -5\}$.
- El resultado es: $((5 * 5) * 2 - 10) + (((-5) * (-5)) * 2 - 10)$
O sea: $(25 * 2 - 10) + (25 * 2 - 10)$

Luego, se rompió todo, porque tengo un término repetido (por culpa de la no inyectividad), y tengo términos que faltan (por culpa de la no sobreyectividad).



Luego, para que la regla de cambio de variable funcione, la función que yo aplique debe ser biyectiva considerando a el conjunto definido por el rango R.i como la **imagen de la función**.

Ejemplo con función inyectiva pero no sobreyectiva:

$$\langle \sum i : 24 < i \leq 29 : i * 2 - 10 \rangle (A)$$

Cambio de variable con $f: j = 2 * j$ quedaría:

$$\langle \sum i : 24 < 2 * j \leq 29 : (2 * j) * 2 - 10 \rangle (B'')$$

Lectura operacional para A:

$$(25 * 2 - 10) + (26 * 2 - 10) + (27 * 2 - 10) + (28 * 2 - 10) + (29 * 2 - 10)$$

Lectura operacional para B'':

- El rango es $j \in \{13, 14\}$.
- El resultado es: $((2 * 13) * 2 - 10) + ((2 * 14) * 2 - 10)$
 $= (26 * 2 - 10) + (28 * 2 - 10)$

¡¡Faltan tres términos!!

Ejemplo con función sobreyectiva pero no inyectiva:

$$\langle \sum i : 24 < i \leq 29 : i * 2 - 10 \rangle (A)$$

Cambio de variable con $f: j = j \text{ div } 2$ quedaría:

$$\langle \sum i : 24 < j \text{ div } 2 \leq 29 : (j \text{ div } 2) * 2 - 10 \rangle (B''')$$

Lectura operacional para B'':

El rango es $j \in \{ 50, 51, 52, 53, 54, 55, 56, 57, 58, 59 \}$.

$$\begin{aligned} \text{El resultado es: } & ((50 \text{ div } 2) * 2 - 10) + ((51 \text{ div } 2) * 2 - 10) + \\ & ((52 \text{ div } 2) * 2 - 10) + ((53 \text{ div } 2) * 2 - 10) + \dots \\ & = (25 * 2 - 10) + (25 * 2 - 10) + (26 * 2 - 10) + (26 * 2 - 10) \end{aligned}$$

¡¡Todos los términos aparecen repetidos!!

Conclusión:

- **Inyectiva:** Me asegura de que no se repiten términos al aplicar el cambio.
- **Sobreyectiva:** Me asegura de que no se pierden términos al aplicar el cambio.

Veamos el 2do requisito:

- j no aparece en R ni en T.

$$\langle \sum i : 24 < i \leq 29 : i * 2 - 10 + j \rangle$$

Acá, j es variable libre y su valor no depende de la cuantificación.

Si aplico cambio de variable con $f.j = j - 11$, me quedaría:

$$\langle \sum j : 24 < j - 11 \leq 29 : (j - 11) * 2 - 10 + j \rangle$$

Tengo una **colisión** de variables, se mezclaron dos cosas distintas.

Siempre tendremos cuidado de no pisar nombres de variables.

Usualmente, volveremos a usar el mismo nombre de variable que teníamos antes (i):

$$\langle \sum i : 24 < i \leq 29 : i * 2 - 10 + j \rangle$$

= { cambio de variable con $f.i = i - 11$, (también solemos escribir $i \leftarrow i - 11$) }

$$\langle \sum i : 24 < i - 11 \leq 29 : (i - 11) * 2 - 10 + j \rangle$$

Notación:

- Usualmente, volveremos a usar el mismo nombre de variable que teníamos antes (cambiamos "i" por "i - 11" y no por "j - 11").
- Muchas veces escribiremos la función de manera implícita:
Diremos " $i \leftarrow i - 11$ " en lugar de " $f.i = i - 11$ "

Cambio de variable con listas

Un cambio de variable que usaremos mucho será el siguiente:

$$f.(b, bs) = b \blacktriangleright bs$$

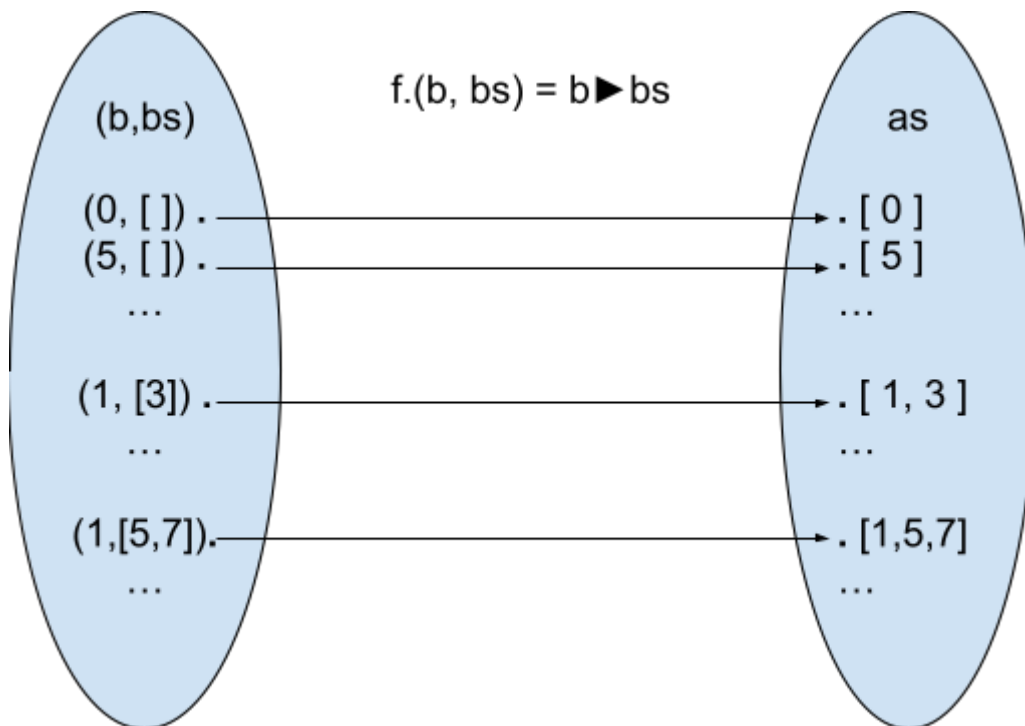
Este cambio se aplica a una variable cuantificada de tipo lista "as" y **se reemplaza por dos variables cuantificadas "b" y "bs"**, una para el primer elemento de la lista y otra para la lista de los elementos restantes. Para que la función sea sobreyectiva, el rango original

debe forzar que “ $as \neq []$ ” (“**as**” **no es la lista vacía**), ya que no es posible llegar a ese elemento con f .

Ejemplo:

$$\langle \text{Max } as : as \neq [] \wedge \#as \leq 3 : \#as \rangle$$

Acá el rango claramente excluye la lista vacía. En la siguiente figura podemos ver que la función es biyectiva (los conjuntos son infinitos!):



$\langle \text{Max } as : as \neq [] \wedge \#as \leq 3 : \#as \rangle$
 = { cambio de variable con $f.(b, bs) = b \blacktriangleright bs$ }
 $\langle \text{Max } b, bs : b \blacktriangleright bs \neq [] \wedge \#(b \blacktriangleright bs) \leq 3 : \#(b \blacktriangleright bs) \rangle$
 = { (**hacemos más pasos por diversión:**) propiedad de listas y lógica }
 $\langle \text{Max } b, bs : \#(b \blacktriangleright bs) \leq 3 : \#(b \blacktriangleright bs) \rangle$
 = { def. de # }
 $\langle \text{Max } b, bs : \#bs + 1 \leq 3 : \#bs + 1 \rangle$
 = { distrib. de + con Max }
 $\langle \text{Max } b, bs : \#bs + 1 \leq 3 : \#bs \rangle + 1$

Ejercicios

Ejercicio 14a:

$\langle \sum i : |i| < 5 : i \text{ div } 2 \rangle$ con $f.j = 2 * j$

¿Qué debe cumplir f ? debe ser biyectiva considerando a la imagen de la función

como el conjunto definido por el rango de la cuantificación.

En este caso el rango es: $\{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$

¿es f inyectiva acá? Si

¿es f sobreyectiva acá? No, porque no tengo forma de llegar al -3 , -1 ni a ningún impar.

No se puede aplicar cambio de variable.

¿qué valores puede tomar j ? estamos hablando del **dominio de f** . Este conjunto se determina tomando la imagen y volviendo hacia atrás con f . O sea, encontrar aquellos j tales que $f.j \in \{-4, -3, -2, -1, 0, 1, 2, 3, 4\}$. En este caso el conjunto es $\{-2, -1, 0, 1, 2\}$.

Ejercicio: Aplicarlo como si se pudiera y verificar que **NO** da lo mismo usando la lectura operacional de ambas versiones.

Ejercicio 14b:

$\langle \sum i : \text{par}.i \wedge |i| < 5 : i \text{ div } 2 \rangle$ con $f.j = 2 * j$

En este caso el rango es: $\{-4, -2, 0, 2, 4\}$ (o sea la imagen de f)

¿cuál es el dominio de f ? es el conjunto $\{-2, -1, 0, 1, 2\}$.

¿es f inyectiva acá? Si.

¿es f sobreyectiva acá? Sí, porque cubro todos los elementos de la imagen: $\{-4, -2, 0, 2, 4\}$

Luego, **sí se puede aplicar cambio de variable.**

Ejercicio: Aplicarlo y verificar que da lo mismo usando la lectura operacional de ambas versiones.

T2: Eliminación de variable

$\langle \oplus i, j : i = C \wedge R.i.j : T.i.j \rangle = \langle \oplus j : R.C.j : T.C.j \rangle$

Tenemos dos variables cuantificadas pero una de ellas tiene un valor fijo C . Luego esa variable no hace falta, podemos eliminarla y reemplazar toda ocurrencia (apariciones en R y T) de i por su valor C .

Se diferencia del rango unitario en que igual queda el cuantificador porque tengo más variables cuantificadas.

Demostración: Partimos del lado izquierdo y llegamos al lado derecho.

$\langle \oplus i, j : i = C \wedge R.i.j : T.i.j \rangle$

$= \{ \text{anidado} \}$

$\langle \oplus i : i = C : \langle \oplus j : R.i.j : T.i.j \rangle \rangle$

$= \{ \text{rango unitario} \}$

$\langle \oplus j : R.C.j : T.C.j \rangle$

T3: Rango unitario y condición

$\langle \oplus i : i = C \wedge P.i : T.i \rangle = (P.C \rightarrow T.C$
 $\quad \quad \quad [] \neg P.C \rightarrow e$
 $\quad \quad \quad)$

donde e es el elemento neutro de $\oplus$.

Tengo una variable cuantificada i, y un predicado que me dice que i tiene un valor fijo pero que además debe satisfacer una condición adicional. Si la condición no vale, tengo rango vacío, si vale, tengo rango unitario.

Demostración: Partimos del lado izquierdo, llegamos al derecho:

$\langle \oplus i : i = C \wedge P.i : T.i \rangle$
 $= \{ \text{Leibniz 2: } i = C \wedge P.i \text{ es lo mismo que } i = C \wedge P.C \}$
 $\langle \oplus i : i = C \wedge \underline{P.C} : T.i \rangle$

- (abrimos un análisis por casos)
- Primer caso: **$P.C$** (esto quiere decir “P.C es verdadero”)
 - $= \{ \text{sustitución} \}$
 - $\langle \oplus i : i = C \wedge \text{True} : T.i \rangle$
 - $= \{ \text{lógica} \}$
 - $\langle \oplus i : i = C : T.i \rangle$
 - $= \{ \text{rango unitario} \}$
 - $T.C$
- Segundo caso: **$\neg P.C$**
 - $= \{ \text{sust.} \}$
 - $\langle \oplus i : i = C \wedge \text{False} : T.i \rangle$
 - $= \{ \text{lógica} \}$
 - $\langle \oplus i : \text{False} : T.i \rangle$
 - $= \{ \text{rango vacío, siendo e el elemento neutro de } \oplus \}$
 - e

T11: Leibniz 2

Esto es una propiedad de la lógica:

$$e = f \wedge E(z := e) \equiv e = f \wedge E(z := f)$$

Primero recordemos: $X(a := b)$ que decir agarrar la expresión “X” y reemplazar toda ocurrencia de “a” por “b”.

Esto quiere decir que si tengo una conjunción entre una igualdad y una segunda expresión, en la segunda expresión puedo hacer reemplazo de iguales por iguales según indica la igualdad.

Ejemplo: $i = 37 \wedge x \bmod i = 2 \equiv i = 37 \wedge x \bmod 37 = 2$

- "e" es i
- "i" es 37
- "E(z := e)" es $x \bmod i = 2$
 - Para que esto suceda, E debe ser $(x \bmod z = 2)$.

Conteo

A8: Definición de conteo

Muchas veces vamos a querer escribir expresiones cuantificadas para contar cuántas veces sucede algo.

Ejemplo: ¿Cuántos números divisibles por 7 hay entre 0 y 100?

$\langle \sum i : 0 \leq i \leq 100 \wedge i \bmod 7 = 0 : 1 \rangle$

Lectura operacional: El rango es $i \in \{0, 7, 14, \dots, 70, 77, \dots\}$

(uno por cada divisible por 7 entre 0 y 100). El resultado es:

$1 + 1 + 1 + \dots + 1 + 1 + \dots$ (tantas veces como elementos tenga en el rango)

Luego, el resultado es el que yo esperaba.

Fin del ejemplo

Como vamos a hacer esto muy seguido, nos vamos a inventar una notación para escribir este tipo de cuantificaciones (sumas de 1's), y la vamos a llamar **conteo**:

$\langle N i : R.i : T.i \rangle = \langle \sum i : R.i \wedge T.i : 1 \rangle$

(N mayúsculas es la notación para el cuantificador)

Importante: El conteo no es una expresión cuantificada de primer nivel (como las que venimos viendo) sino sólo una notación para ahorrar espacio. Por lo tanto, no valen necesariamente las reglas y axiomas que venimos viendo.

Ejemplo usando conteo:

$\langle N i : 0 \leq i \leq 100 : i \bmod 7 = 0 \rangle$

Observación: A diferencia de los cuantificadores de primer nivel, en el conteo el término siempre es un booleano y el resultado siempre es un número (o sea el tipo del término y el tipo del resultado son distintos).

La **lectura operacional** vista para los cuantificadores comunes ya no vale para el cuantificador de conteo (estaríamos sumando booleanos). Lo que hay que hacer es primero pasar a sumatoria usando def. de conteo.

Digesto

El conteo tiene su propio digesto: [conteo.pdf](#)

Reglas adicionales para el conteo

Recordemos la definición de conteo:

$$\langle N i : R.i : T.i \rangle = \langle \sum i : R.i \wedge T.i : 1 \rangle$$

Para pasar cosas del rango al término:

$$\langle N i : R.i : T.i \rangle = \langle N i : R.i \wedge T.i : \text{True} \rangle$$

$$\langle N i : R.i : T.i \rangle = \langle N i : T.i : R.i \rangle$$

(ejercicio: demostrar)

(otro ejercicio: verificar con ejemplos)

Ejercicio 15b

$$\begin{aligned} & \langle N i : i - n = 1 : \text{par}.i \rangle \\ &= \{ \text{definición de conteo} \} \\ & \langle \sum i : \underline{i - n = 1} \wedge \text{par}.i : 1 \rangle \\ &= \{ \text{aritmética} \} \\ & \langle \sum i : i = 1 + n \wedge \text{par}.i : 1 \rangle \\ &= \{ \text{rango unitario y condición} \} \\ & \quad (\text{par}.(1 + n) \rightarrow 1 \\ & \quad \square \neg \text{par}.(1 + n) \rightarrow 0 \\ & \quad) \end{aligned}$$

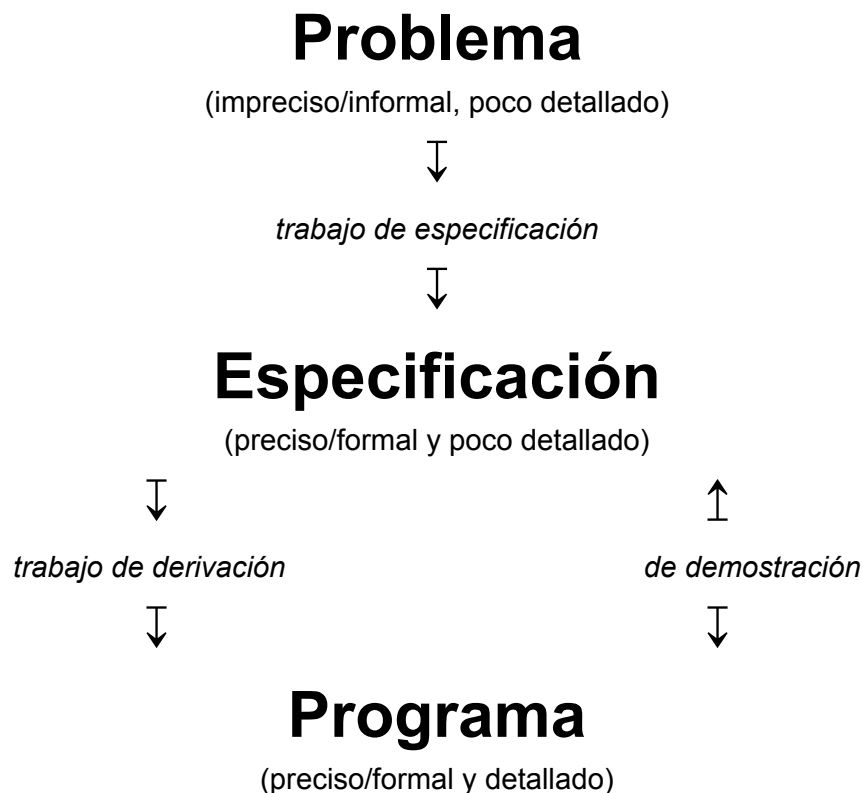
Ejemplo: si $n=4$, queda: 0 porque $(n + 1)$ es impar. si $n=37$, queda 1 porque $(n + 1)$ es par.

Ejercicios

[practico1.pdf](#)

Programación funcional

Introducción



Problemas

Lo que uno quiere solucionar, expresado de manera informal e imprecisa en lenguaje natural.

Ejemplo 1: “dado un número quiero saber si es impar”

Ejemplo 2: “dada una lista de números, quiero obtener su promedio”

Ejemplo 3: “dada una lista de números, quiero saber si todos sus elementos son iguales a un valor dado” (ejercicio 2b del práctico 2)

Especificaciones

Una especificación formaliza el problema a resolver de manera formal pero no detallada (o sea, expresa el qué pero no el cómo).

En el caso de programación funcional, decidimos resolver el problema a través de una función.

Luego, una especificación tendrá los siguientes componentes:

1. **Nombre para la función que resuelve el problema**
2. **Tipo de la función**
3. **Predicado que indica qué debe satisfacer la función**
(para que se pueda considerar que resuelve el problema)

Ejemplos

Ejemplo 1: “dado un número quiero saber si es impar”

- nombre de la función: esImpar (esto es camel case, snake case es “es_impar”)
- tipo de la función: $\text{Int} \rightarrow \text{Bool}$
- **predicado:**

$$\text{esImpar}.n = (n \bmod 2 = 1)$$

Ejemplo 2: “dada una lista de números, quiero obtener su promedio”

- nombre de la función: promedio
- tipo de la función: $[\text{Int}] \rightarrow \text{Float}$
- **predicado:**

$$\text{promedio}.xs = \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle / \#xs$$

Observaciones:

- promedio.[] no está definido.
- Especificación precisa: Me distingue claramente cuáles funciones son solución.
- Poco detallada: Hay que sumar los elementos de la lista, pero a la especificación no le importa cómo se hace esto en la compu.

Ejemplo 3: “dada una lista de números, quiero saber si todos son iguales a un valor dado”

- nombre de la función: usamos “iga”, como en el práctico
- tipo de la función: $\text{Int} \rightarrow [\text{Int}] \rightarrow \text{Bool}$
- predicado:

Recurso útil para especificar: hacer un ejemplo y usar “lectura operacional” pero hacia atrás. Supongamos que $xs = [2, 4, -7]$, $e = 4$, ¿qué predicado me interesa calcular?

$$(xs[0] = e) \wedge (xs[1] = e) \wedge (xs[2] = e)$$

Esto me da una idea de que el término de lo que quiero cuantificar tiene la forma:

$$(xs[i] = e)$$

con i tomando valores $\{0, 1, 2\}$.

$$\text{iga}.e.xs = \langle \forall i : 0 \leq i < \#xs : xs[i] = e \rangle$$

Observación: En clase se propuso usar $\in$, pero no existe esto para listas (ver digesto de listas: [PDF listas.pdf](#)). En introalg, se vio una $\in_L$ que sí funciona para listas, pero para poder usarlo tendríamos que especificarlo y derivarlo primero, así que no nos interesa complicarnos con eso. Recomendación para algoritmos 1: **Nunca usen $\in_L$.**

Desarrollo sobre $\in_L$: “sumar los elementos de una lista”

$$\langle \sum x : x \in_L xs : x \rangle$$

¿es correcto esto? ¿realmente me suma todos los elementos de la lista?

Queda como ejercicio. Piensenlo. Y comparar con:

$$\langle \sum i : 0 \leq i < \#xs : xs[i] \rangle$$

Observación: Todos los ejemplos vistos hasta ahora tienen la forma $f.x = E$ (la función aplicada a sus parámetros es igual a algo). **No siempre una especificación es de esta forma.**

Ejemplo 4: “dados dos números, quiero obtener un factor común de ellos (que no sea el 1)”.

Ejemplo: Si me dan el 44 y el 16, un factor común posible es 4, otro es 2.

Especifiquemos:

- nombre de la función: factorComun
- tipo: $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
- predicado de especificación:

$$n \bmod x = 0 \wedge m \bmod x = 0 \wedge x \neq 1$$

¿Está bien esta especificación? Está mal porque **no menciona a la función que está siendo especificada**. Una especificación siempre debe hablar de lo que tiene que satisfacer la función que va a resolver mi problema. Además acá se menciona una “x” que no se sabe qué es.

La arreglamos así, usando una **definición local**:

$$n \bmod x = 0 \wedge m \bmod x = 0 \wedge x \neq 1$$

donde $x = \text{factorComun}.n.m$

O también sin usar definiciones locales:

$$\begin{aligned} & n \bmod (\text{factorComun}.n.m) = 0 \wedge \\ & m \bmod (\text{factorComun}.n.m) = 0 \wedge \\ & (\text{factorComun}.n.m) \neq 1 \end{aligned}$$

Una especificación me indica todo lo que es **necesario y suficiente** para que la función devuelva una solución correcta a mi problema.

¿Porqué una especificación es precisa y al mismo tiempo poco detallada?

Precisa: de todo el mundo de soluciones posibles (o sea todas las funciones de tipo: $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$), la **especificación me distingue claramente cuáles resuelven el problema y cuales no**.

Poco detallada: Igual, puede haber más de una función que me resuelva el problema, y la especificación no tiene preferencia por ninguna de ellas (para el ejemplo que vimos con $n=44$ y $m=16$, podría dar 2 o 4).

Observación última: No necesariamente un problema tiene una única solución.

Observaciones

1. Usualmente llamamos especificación directamente al predicado de especificación (el nombre y el tipo de la función se deducen de ver el predicado).
2. El predicado de especificación **debe mencionar** a la función que está siendo especificada.
3. **No necesariamente debo usar expresiones cuantificadas.** Éstas son sólo una herramienta que puedo usar o no usar de acuerdo a mis necesidades.
4. Si uso expresiones cuantificadas, no necesariamente es lo que está afuera de todo en el predicado (en el ejemplo dividimos por #xs afuera). E incluso puedo tener varias expresiones cuantificadas operadas entre sí.
5. **No siempre** una especificación es de la forma: $f.x = E$ (la función aplicada a sus parámetros es igual a algo).

Predicados

¿Qué predicados son válidos en una especificación? Básicamente todas las cosas formales que podemos tomar de la matemática, la lógica de predicados, la lógica de primer orden, y la cuantificación general. También usamos todos los tipos conocidos y sus funciones asociadas (ver Introducción a los algoritmos):

- Booleanos ($\wedge$, $\vee$, $\neg$, etc.)
- Integers (+ , - , div , mod , max , min , etc.)
- Listas (! , # , ++ , etc)
- etc.

Programas

Los programas son **expresiones ejecutables** (yo suelo decir “**programables**”), es decir que tienen reglas asociadas que les permiten ser reducidas a otras expresiones (con suerte, que me permiten llegar a un valor final). En los programas, también podemos escribir definiciones. Las definiciones de las funciones crean nuevas reglas.

- Expresiones:
 - Constantes: 0, 42, -10, 3.14159, ∞ , $-\infty$, True, False, [], [9], (3, True), etc.
 - Variables: n, x, xs, xss, etc.
 - Operaciones básicas: +, *, div, mod, max, min, $\blacktriangleright$, $\wedge$, $\vee$, $\neg$, <, >, $\leq$, $\geq$, =, etc.
 - Llamadas a funciones
 - Análisis por casos
 - Definiciones locales
 - **No son expresiones válidas las expresiones cuantificadas.**
- Definiciones:

- Introducen nuevas funciones con sus respectivos nombres, tipos, y reglas de reducción.
- En las reglas de reducción se indica cómo se sustituye una aplicación de una función por una expresión válida en el lenguaje de programación.
- En las definiciones se puede usar pattern-matching (ver intro a los algoritmos).

En el contexto de la resolución de problemas, cada problema será resuelto con una **definición para la función** que fue especificada. La expresión que resuelve el problema es simplemente una llamada a la función con los parámetros apropiados.

Ejemplos

Ejemplo 1:

La especificación era:

$$\text{esImpar}.n = (n \bmod 2 = 1)$$

El programa se puede obtener directamente de la especificación ya que la especificación contiene todas operaciones permitidas en el lenguaje de programación (son ejecutables o “programables”).

La **definición** de la función es directamente:

$$\text{esImpar}.n \doteq (n \bmod 2 = 1)$$

Aclaración importante: El símbolo “ $\doteq$ ” quiere decir que estamos ante una definición. En una definición, **el lado izquierdo es algo nuevo que no existía antes** (que no estaba definido antes) y que está siendo definido ahora. No hace falta demostrar una definición, ya que la definición está estableciendo esa identidad. **Una definición no es un predicado.**

Se diferencia de la igualdad (“ $=$ ”) en que en la igualdad, ambos lados están previamente definidos, y lo que se está haciendo es **afirmar** que ambas cosas son iguales. Esta afirmación puede ser cierta o no (se puede demostrar o refutar). **Una igualdad es un predicado.**

En el ejemplo, la función “esImpar” no estaba definida previamente, pero sí estaba “especificada”, esto quiere decir, estaba enunciado el predicado que la función debería satisfacer una vez que fuera definida.

Ejemplo 2:

La especificación era:

$$\text{promedio}.xs = \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle / \#xs$$

En este caso, el programa no sale directamente de la especificación, ya que tenemos una expresión cuantificada (esto es algo **no programable**).

¿Cómo obtenemos el programa que me resuelve el problema?

1. **De la galera:** Como somos expertos programadores funcionales, sabemos escribir directamente la función que resuelve este problema.

2. **Derivando:** Como no somos expertos programadores funcionales, pero sí somos muy metódicos, vamos a aplicar estrategias conocidas que me permiten obtener el programa a partir de la especificación.

De la galera: (lo hago yo)

promedio :: [Int] → Float
promedio.xs ≡ sum.xs / #xs

sum.[] ≡ 0
sum.(x ► xs) ≡ x + sum.xs

¿Cómo sabemos que este programa satisface la especificación?

¿Con ejemplos? No, porque se nos puede escapar algún caso en el que no valga. Igual **es buena idea probar ejemplos**.

Debemos **demostrar** que la función satisface su especificación. Para eso usaremos todos los recursos matemáticos conocidos, incluyendo el principio de inducción de ser necesario.

Ejemplo 3:

La especificación era:

$$\text{iga.e.xs} = \langle \forall i : 0 \leq i < \#xs : xs[i] = e \rangle$$

Un programa posible es (sacado de la galera):

iga : Int -> [Int] -> Bool
iga.e.[] ≡ True
iga.e.(x ► xs) ≡ (x = e) ∧ iga.e.xs

Recordar: Siempre usar puntitos “.” al aplicar parámetros a funciones.

Demostración

Dados una especificación y un programa, podemos demostrar que el programa satisface la especificación.

Ejemplo 2: Queremos demostrar que la función definida como:

promedio :: [Int] → Float
promedio.xs ≡ sum.xs / #xs

sum.[] ≡ 0
sum.(x ► xs) ≡ x + sum.xs

Satisface el siguiente predicado: **Para toda** lista posible xs ≠ [] .

$$\text{promedio.xs} = \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle / \#xs$$

Como tengo que demostrar un “para todo” sobre listas, puedo usar inducción sobre listas. Pero en este caso el **caso base** es con listas de un elemento (xs es de la forma [x], lista de un solo elemento x).

Caso base: Supongamos que $xs = [x]$. Agarremos todo y tratemos de llegar a True (que es lo más seguro ya que podemos aplicar propiedades de ambos lados).

$$\text{promedio}.[x] = \langle \sum i : 0 \leq i < \# [x] : [x]!i \rangle / \# [x]$$

= { def. de # }

$$\text{promedio}.[x] = \langle \sum i : 0 \leq i < 1 : [x]!i \rangle / 1$$

= { lógica }

$$\text{promedio}.[x] = \langle \sum i : i = 0 : [x]!i \rangle / 1$$

= { rango unitario }

$$\text{promedio}.[x] = [x]!0 / 1$$

= { definición de ! }

$$\text{promedio}.[x] = x / 1$$

= { aritmética }

$$\text{promedio}.[x] = x$$

= { def. de promedio }

$$\text{sum}.[x] / \# [x] = x$$

= { def. # y aritmética }

$$\text{sum}.[x] = x$$

= { hago explícito el patrón }

$$\text{sum}(x \triangleright []) = x$$

= { def. de sum }

$$x + \text{sum}[] = x$$

= { def. de sum }

$$x + 0 = x$$

= { lógica }

True

Paso inductivo: Queda como ejercicio.

Derivación

Es el proceso **creativo** pero metódico que permite obtener un programa a partir de una especificación.

Como el lenguaje de los programas es un subconjunto del lenguaje de las especificaciones

Ejemplo 3 (práctico 2 ejercicio 2b): Recordemos la especificación:

$$\text{iga.e.xs} = \langle \forall i : 0 \leq i < \#xs : xs!i = e \rangle$$

¿Cómo obtenemos un programa a partir de esta especificación?

Observamos que tenemos una expresión cuantificada. Esto quiere decir que tenemos una cantidad indeterminada de términos operados entre sí (una conjunción de cosas, cuya cantidad de términos depende de los parámetros de entrada). ¿Cuántos términos tiene?

Tiene #xs términos. Para resolver eso con mi lenguaje de programación, el único recurso que tengo es el de definir una **función recursiva**. Luego, vamos a necesitar aplicar inducción para obtener esta definición recursiva. La función se va a definir de la siguiente forma:

Caso base:

$\text{iga.e.}[\] \doteq ???$

Caso recursivo/Paso inductivo:

$\text{iga.e.}(x \blacktriangleright xs) \doteq ????$

(seguramente acá va a aparecer la llamada recursiva "iga.e.xs")

Intentemos resolver todas las incógnitas que tenemos **usando siempre la especificación**.

Caso base:

$\text{iga.e.}[\]$
 $= \{ \text{especificación de iga} \}$
 $\langle \forall i : 0 \leq i < \#[\] : [\] ! i = e \rangle$
 $= \{ \text{def } \# \}$
 $\langle \forall i : 0 \leq i < 0 : [\] ! i = e \rangle$
 $= \{ \text{lógica} \}$
 $\langle \forall i : \text{False} : [\] ! i = e \rangle$
 $= \{ \text{rango vacío} \}$
 True

Paso inductivo:

En la derivación, voy a partir de " $\text{iga.e.}(x \blacktriangleright xs)$ " y quiero llegar a algo que sea ejecutable / programable. En el medio, seguramente voy a necesitar hacer que aparezca la llamada recursiva " iga.e.xs ". Para eso necesito plantear mi Hipótesis Inductiva:

H.I. : $\text{iga.e.xs} = \langle \forall i : 0 \leq i < \#xs : xs ! i = e \rangle$

Vamos a eso:

$\text{iga.e.}(x \blacktriangleright xs)$
 $= \{ \text{especificación} \}$
 $\langle \forall i : 0 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs) ! i = e \rangle$
 $= \{ \text{def } \# \}$
 $\langle \forall i : 0 \leq i < \#xs + 1 : (x \blacktriangleright xs) ! i = e \rangle$
 $= \{ 0 \leq i < \#xs + 1 \text{ es lo mismo que } (i = \#xs) \vee (0 \leq i < \#xs) \}$
 $0 \leq i < \#xs + 1 \text{ también es lo mismo que } (i = 0) \vee (1 \leq i < \#xs + 1)$
 Acá puedo aplicar cualquiera de las dos, pero sólo una me sirve para llegar a la H.I.
 Probemos la primera. (**vamos a ver que no anda**) }

$\langle \forall i : 0 \leq i < \#xs \vee i = \#xs : (x \blacktriangleright xs) ! i = e \rangle$

$= \{ \text{partición de rango} \}$

$\langle \forall i : 0 \leq i < \#xs : (x \blacktriangleright xs) ! i = e \rangle \wedge$

$\langle \forall i : i = \#xs : (x \blacktriangleright xs) ! i = e \rangle$

$= \{ \text{¿en qué terminó me conviene concentrarme? Siempre primero me conviene ver si puedo llegar a la H.I., porque si no se puede, todos estos pasos no sirven.} \}$

Me conviene concentrarme en el 1er término.

¿podemos aplicar alguna regla que me lleve a la H.I.? me falta que el término quede bien, podría intentar hacer cambio de variable $i \rightarrow i + 1$.

$\langle \forall i : 0 \leq i + 1 < \#xs : (x \blacktriangleright xs) ! (i + 1) = e \rangle \wedge$

$\langle \forall i : i = \#xs : (x \triangleright xs) ! i = e \rangle$
 $= \{ \text{def. !} \}$
 $\langle \forall i : 0 \leq i + 1 < \#xs : xs ! i = e \rangle \wedge$
 $\langle \forall i : i = \#xs : (x \triangleright xs) ! i = e \rangle$
 $= \{ \text{Ahora se me arregló el término para la H.I., pero se rompió el rango.}$
 $\text{NO HAY FORMA DE LLEGAR A LA H.I. NO ERA POR ACÁ.} \}$
 $= \{ \text{APLICAMOS LA OTRA ESTRATEGIA:}$
 $0 \leq i < \#xs + 1 \text{ es lo mismo que } (i = 0) \vee (1 \leq i < \#xs + 1) \}$
 $\langle \forall i : (i = 0) \vee (1 \leq i < \#xs + 1) : (x \triangleright xs) ! i = e \rangle$
 $= \{ \text{partición de rango} \}$
 $\langle \forall i : i = 0 : (x \triangleright xs) ! i = e \rangle \wedge$
 $\langle \forall i : 1 \leq i < \#xs + 1 : (x \triangleright xs) ! i = e \rangle$
 $= \{ \text{probemos de nuevo cambio de variable } i \rightarrow i + 1 \}$
 $\langle \forall i : i = 0 : (x \triangleright xs) ! i = e \rangle \wedge$
 $\langle \forall i : 1 \leq i + 1 < \#xs + 1 : (x \triangleright xs) ! (i + 1) = e \rangle$
 $= \{ \text{álgebra (resto 1 en los tres miembros de la desigualdad)} \}$
 $\langle \forall i : i = 0 : (x \triangleright xs) ! i = e \rangle \wedge$
 $\langle \forall i : 0 \leq i < \#xs : (x \triangleright xs) ! (i + 1) = e \rangle$
 $= \{ \text{def. !} \}$
 $\langle \forall i : i = 0 : (x \triangleright xs) ! i = e \rangle \wedge$
 $\langle \forall i : 0 \leq i < \#xs : xs ! i = e \rangle$
 $= \{ \text{llegamos a la Hipótesis Inductiva !!} \}$
 $\langle \forall i : i = 0 : (x \triangleright xs) ! i = e \rangle \wedge \text{iga.e.xs}$
 $= \{ \text{rango unitario} \}$
 $(x \triangleright xs) ! 0 = e \wedge \text{iga.e.xs}$
 $= \{ \text{acá ya es un programa correcto pero igual puedo simplificar aplicando def !} \}$
 $x = e \wedge \text{iga.e.xs}$

Listo. Resultado final:

$\text{iga} : \text{Int} \rightarrow [\text{Int}] \rightarrow \text{Bool}$
 $\text{iga.e.}[] \doteq \text{True}$
 $\text{iga.e.}(x \triangleright xs) \doteq (x = e) \wedge \text{iga.e.xs}$

Testing / Ejecución

(reducción, cálculo, etc.)

Ejemplo (ejercicio 3 del practico 2): Vamos a **testear** la función ahora a ver si funciona como esperamos.

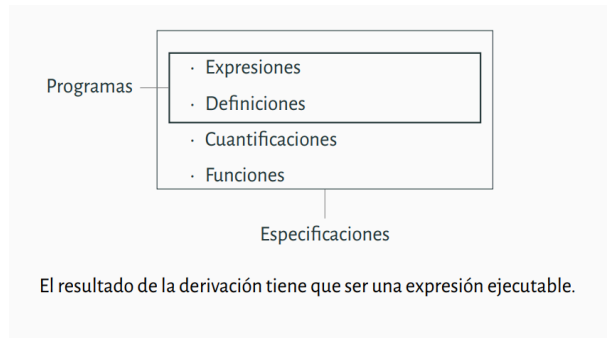
Probemos con $xs = [5, 5, -7]$, $e = 5$. Vamos a ejecutar el programa (o sea reducirlo a un valor canónico).

$\text{iga.5.}[5, 5, -7] \rightarrow (5 = 5) \wedge \text{iga.5.}[5, -7]$
 $\rightarrow (5 = 5) \wedge ((5 = 5) \wedge \text{iga.5.}[-7])$
 $\rightarrow (5 = 5) \wedge ((5 = 5) \wedge ((-7 = 5) \wedge \text{iga.5.}[]))$

→ $(5 = 5) \wedge ((5 = 5) \wedge ((-7 = 5) \wedge \text{True}))$
 → $(5 = 5) \wedge ((5 = 5) \wedge (\text{False} \wedge \text{True}))$

 → False

Resumen: El lenguaje de programación funcional



Expresiones: Es lo que efectivamente se ejecuta.

1. Constantes y variables.
2. Operadores
3. Llamadas a funciones
4. Análisis por casos
5. Definiciones locales

Definiciones: Para definir funciones que pueden ser usadas (y re-usadas) en expresiones (llamadas).

- Reglas de reducción (de la forma $f.x \doteq E$)
- Pattern matching

Tipos:

1. Tipos básicos: Bool, Nat, Int, Char, Num o Float.
2. Listas
3. Tuplas
4. Funciones

Modularización

Ejemplo: Promedio de una lista

Ejemplo: “Dada una lista de números, quiero obtener su promedio.”

La especificación era:

$$\begin{aligned}\text{promedio} &: [\text{Num}] \rightarrow \text{Num} \\ \text{promedio.xs} &= \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle / \#xs\end{aligned}$$

Derivación: Queremos obtener un programa a partir de la especificación.

¿Es mi especificación ya directamente ejecutable (programable)?

No porque contiene una expresión cuantificada que no es ejecutable/programable.

Sin embargo, todo el resto (la división y el denominador) sí es programable. Luego podemos inventar una función nueva para la parte que no es programable y derivarla aparte.

Esta función me suma los elementos de la lista así que la llamaremos sum.

Especificamos la nueva función:

$$\begin{aligned}\text{sum} &: [\text{Num}] \rightarrow \text{Num} \\ \text{sum.xs} &= \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle\end{aligned}$$

Separamos la derivación en dos partes. Primero la derivación de promedio, después la de sum.

Derivación de promedio: Sale derecho así:

$$\begin{aligned}\text{promedio.xs} &= \{ \text{especificación} \} \\ &= \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle / \#xs \\ &= \{ \text{introducimos modularización con función sum.xs} = \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle \} \\ &\quad \text{sum.xs} / \#xs\end{aligned}$$

Listo la derivación de promedio. Resultado parcial:

$$\text{promedio.xs} \doteq \text{sum.xs} / \#xs$$

Derivación de sum:

$$\text{Especificación: } \text{sum.xs} = \langle \sum i : 0 \leq i < \#xs : xs[i] \rangle$$

Acá tenemos que hacer inducción en xs. Tendremos dos casos:

- Caso base: $\text{sum}[] \doteq ???$
- Paso inductivo: $\text{sum}(x \blacktriangleright xs) \doteq ???? \text{ (mencionando a sum.xs)}$

Ejercicio: Derivar!

Resultado final:

$$\text{promedio.xs} \doteq \text{sum.xs} / \#xs$$

$$\text{sum}[] \doteq 0$$

$$\text{sum}(x \blacktriangleright xs) \doteq x + \text{sum.xs}$$

Testing / verificación: Probar la función promedio con una lista de ejemplo: $xs = [7, 4, 6, 2]$. **Ejercicio!!**

Ejemplo: Suma de potencias

Otro ejemplo (ejercicio 4a del práctico 2):

Especificación:

$\text{sum_pot} : \text{Num} \rightarrow \text{Nat} \rightarrow \text{Num}$

$\text{sum_pot.x.n} = \langle \sum i : 0 \leq i < n : x^i \rangle$

¿qué hace sum_pot? Computa la suma de potencias de un número.

Ejemplo: $x = 3.33$, $n = 4$.

$\text{sum_pot.x.n} = 3.33^0 + 3.33^1 + 3.33^2 + 3.33^3$

Vamos a suponer que $x \neq 0$ ya que si no, mi función no va a estar definida.

Derivación: Necesitamos hacer inducción en n . (en x no tiene sentido ya que no existe la inducción sobre los reales). Mi programa resultado va a tener la forma:

Caso base: $\text{sum_pot.x.0} \doteq ???$

Caso recursivo/paso inductivo: $\text{sum_pot.x.(n+1)} \doteq ???$ (en términos de sum_pot.x.n)

Vamos a eso.

Caso base:

sum_pot.x.0

$= \{ \text{especificación} \}$

$\langle \sum i : 0 \leq i < 0 : x^i \rangle$

$= \{ \text{lógica y rango vacío} \}$

0

Paso inductivo: Hipótesis Inductiva: $\text{sum_pot.x.n} = \langle \sum i : 0 \leq i < n : x^i \rangle$

sum_pot.x.(n + 1)

$= \{ \text{especificación} \}$

$\langle \sum i : \underline{0 \leq i < n + 1} : x^i \rangle$

$= \{ \text{tenemos dos opciones de lógica, probemos primero: } 0 \leq i < n \vee i = n \}$

$\langle \sum i : 0 \leq i < n \vee i = n : x^i \rangle$

$= \{ \text{partición de rango ya q son disjuntas las dos partes} \}$

$\langle \sum i : 0 \leq i < n : x^i \rangle + \langle \sum i : i = n : x^i \rangle$

$= \{ \text{Hipótesis Inductiva} \}$

$\text{sum_pot.x.n} + \langle \sum i : i = n : x^i \rangle$

$= \{ \text{Rango unitario} \}$

$\text{sum_pot.x.n} + x^n$

$= \{ x^n \text{ no es programable (así definimos el lenguaje), así que } \underline{\text{introducimos una modularización "exp" especificada por } \text{exp.x.n} = x^n} \}$

$\text{sum_pot.x.n} + \text{exp.x.n}$

Ya llegamos a algo programable. Terminamos de derivar sum_pot (falta exp).

Resultado parcial:

$$\text{sum_pot.x.0} \doteq 0$$

$$\text{sum_pot.x.(n + 1)} \doteq \text{sum_pot.x.n} + \text{exp.x.n}$$

Derivación de exp: Especificación: $\text{exp.x.n} = x^n$

¿Cómo derivamos? Esto es como un cuantificador, sale por inducción en n.

Caso base:

$$\begin{aligned} &\text{exp.x.0} \\ &= \{ \text{especificación} \} \\ &\quad x^0 \\ &= \{ \text{arit} \} \\ &\quad 1 \end{aligned}$$

Paso inductivo: Hipótesis Inductiva: $\text{exp.x.n} = x^n$

$$\begin{aligned} &\text{exp.x.(n + 1)} \\ &= \{ \text{especificación} \} \\ &\quad x^{n+1} \\ &= \{ \text{prop. de la exponenciación} \} \\ &\quad x^n * x \\ &= \{ \text{H.I.} \} \\ &\quad \text{exp.x.n} * x \end{aligned}$$

¡¡Listo!! Resultado final:

$$\text{sum_pot.x.0} \doteq 0$$

$$\text{sum_pot.x.(n + 1)} \doteq \text{sum_pot.x.n} + \text{exp.x.n}$$

$$\text{exp.x.0} \doteq 1$$

$$\text{exp.x.(n + 1)} \doteq \text{exp.x.n} * x$$

Resumen: ¿Qué hicimos?

En el medio de una derivación, apareció una expresión no programable (por ejemplo una expresión cuantificada) (y que no es la H.I. si es que estoy en el medio de una inducción).

Lo que hacemos al modularizar es inventar una nueva función cuya especificación me indica que calcula esa expresión. Luego, puedo usar esa función en mi derivación para reemplazar la expresión no programable por una llamada a la función.

Para terminar el programa, debemos obtener aparte un programa para la función que modularicé (haciendo otra derivación).

Usualmente, la función modularizada lo que hace es resolver un problema accesorio del problema original que estoy resolviendo.

(por ejemplo, para el promedio de una lista, la suma de la lista,
otro ejemplo, para la suma de potencias, las potencias mismas)

NUNCA VAMOS a aplicar por anticipado la estrategia de modularización. Solamente vamos a aplicar modularización en el momento en el que surja la necesidad en el medio de una derivación.

APARTADO: Derivación de sum_pot usando una estrategia diferente (sale sin modularización):

Caso base:

sum_pot.x.0
 = { ... }
 0

Paso inductivo: Hipótesis Inductiva: $\text{sum_pot.x.n} = \langle \sum i : 0 \leq i < n : x^i \rangle$

sum_pot.x.(n + 1)
 = { especificación }
 $\langle \sum i : 0 \leq i < n + 1 : x^i \rangle$
 = { tenemos dos opciones de lógica, probemos ahora **la otra opción**: $i = 0 \vee 1 \leq i < n + 1$ }
 $\langle \sum i : i = 0 \vee 1 \leq i < n + 1 : x^i \rangle$
 = { partición de rango }
 $\langle \sum i : i = 0 : x^i \rangle + \langle \sum i : 1 \leq i < n + 1 : x^i \rangle$
 = { en la 2da, cambio de variable $i \rightarrow i + 1$ }
 $\langle \sum i : i = 0 : x^i \rangle + \langle \sum i : 1 \leq i + 1 < n + 1 : x^{i+1} \rangle$
 = { aritmética }
 $\langle \sum i : i = 0 : x^i \rangle + \langle \sum i : 0 \leq i < n : x^{i+1} \rangle$
 = { prop. de exponenciación }
 $\langle \sum i : i = 0 : x^i \rangle + \langle \sum i : 0 \leq i < n : x^i * x \rangle$
 = { distributividad }
 $\langle \sum i : i = 0 : x^i \rangle + \langle \sum i : 0 \leq i < n : x^i \rangle * x$
 = { H.I. }
 $\langle \sum i : i = 0 : x^i \rangle + \text{sum_pot.x.n} * x$
 = { rango unitario }
 $x^0 + \text{sum_pot.x.n} * x$
 = { aritmética }
 $1 + \text{sum_pot.x.n} * x$

¡¡Listo!! Resultado final:

$\text{sum_pot.x.0} \doteq 0$
 $\text{sum_pot.x.(n + 1)} \doteq 1 + \text{sum_pot.x.n} * x$

No hizo falta hacer modularización!!

Acabo de obtener dos programas distintos que calculan exactamente lo mismo (o sea resuelven el mismo problema) de dos maneras distintas.

Testing / verificación:

Ejemplo: $x = 3.33$, $n = 4$.

Me debería dar: $\text{sum_pot}.x.n = 3.33^0 + 3.33^1 + 3.33^2 + 3.33^3$

```
sum_pot.(3.33).4 = sum_pot.(3.33).(3+1)    // luego aplica el patrón n+1 con n=3.
    → 1 + sum_pot.(3.33).3 * (3.33)
    → 1 + (1 + sum_pot.(3.33).2 * (3.33)) * (3.33)
    → 1 + (1 + (1 + sum_pot.(3.33).1 * (3.33)) * (3.33)) * (3.33)
    → 1 + (1 + (1 + (1 + sum_pot.(3.33).0 * (3.33)) * (3.33)) * (3.33)) *
(3.33)    // acá apliqué el caso base
    → 1 + (1 + (1 + (1 + 0 * (3.33)) * (3.33)) * (3.33)) * (3.33)
    → 1 + (1 + (1 + (1 + 0) * (3.33)) * (3.33)) * (3.33)
    → 1 + (1 + (1 + 1 * (3.33)) * (3.33)) * (3.33)
    → 1 + (1 + (1 + 3.33) * (3.33)) * (3.33)
    = 1 + (1 + 3.33 + 3.332) * (3.33) // por distributividad
    = 1 + 3.33 + 3.332 + 3.333 // por distributividad
```

(el resultado es 52.344937)

Inducción

Usamos la inducción a la hora de derivar programas que calculan expresiones cuantificadas (o “esconden” una cuantificación, como la exponenciación o el factorial). La especificación de la función debe ser exactamente una expresión cuantificada, sin ninguna operación extra por fuera de la cuantificación. El resultado será una función recursiva. En una ejecución, la recursión se encargará de recorrer todos los términos representados por la expresión cuantificada.

Esquemas de inducción

Hay muchas formas de derivar usando inducción. Pero siempre cada forma está asociada a un teorema que me dice que efectivamente la inducción aplicada vale.

Supongamos que tenemos una función f sobre los números naturales y su predicado de especificación P , un predicado sobre la función y los naturales.

Ejemplo: El factorial de un número: La función es fac , y el predicado de especificación es:

$$P.n \equiv \text{fac}.n = n!$$

Lo que queremos obtener es una definición para la función “ fac ” tal que valga P para todo n :

$\forall P.n$ (o sea: $P.0 \wedge P.1 \wedge P.2 \wedge P.3 \wedge \dots$)

¿Qué nos dice el teorema de inducción? Lo siguiente:

$\forall n : P.n$ (o sea: $P.0 \wedge P.1 \wedge P.2 \wedge P.3 \wedge \dots$)
 $\equiv$
 $P.0 \wedge (\forall n : P.n \Rightarrow P.(n+1))$ (o sea: $P.0 \wedge (P.0 \Rightarrow P.1) \wedge (P.1 \Rightarrow P.2) \wedge (P.2 \Rightarrow P.3) \wedge \dots$)

En esta equivalencia, veamos:

¿Vale la ida $\Rightarrow$? Sí, muy fácilmente porque todo es True, $P.0$, $P.n$ y $P.(n+1)$.

¿Vale la vuelta $\Leftarrow$? Es un poquito más complicado.

La hipótesis es:

$P.0 \wedge (\forall n : P.n \Rightarrow P.(n+1))$

Queremos demostrar:

$\forall n : P.n$ (o sea: $P.0 \wedge P.1 \wedge P.2 \wedge P.3 \wedge \dots$)

¿Vale $P.0$? Sí, muy fácilmente porque es una de mis hipótesis.

¿Vale $P.1$? Sí, porque tengo hipótesis $P.0 \Rightarrow P.1$, y además sé que vale $P.0$.

¿Vale $P.2$? Sí, porque tengo hipótesis $P.1 \Rightarrow P.2$, y además acabo de ver que vale $P.1$

Y así vale para siempre: $P.3$, $P.4$,

La idea de la inducción es que hay uno o más casos base (cosas que valen de entrada) y otros casos en los que cuando una cosa vale, hacen valer una cosa nueva:

~~$P.0 \wedge P.1 \wedge P.2 \wedge P.3 \wedge \dots$~~

Ahora, esquemas de este tipo hay muchos más, siempre y cuando garanticen que el predicado vale para todo n .

Otro esquema:

$P.0 \wedge P.1 \wedge (\forall n : P.n \wedge P.(n+1) \Rightarrow P.(n+2))$

Este es el que se usa para fibonacci:

$\text{fib}.0 \doteq 0$

$\text{fib}.1 \doteq 1$

$\text{fib}.(n+2) \doteq \text{fib}.(n+1) + \text{fib}.n$

¿Vale este esquema? Verifiquemos:

~~$P.0 \wedge P.1 \wedge P.2 \wedge P.3 \wedge P.4 \wedge \dots$~~

Otro esquema:

$P.0 \wedge P.1 \wedge (\forall n : P.n \Rightarrow P.(n+2))$

¿Vale este esquema? Verifiquemos:

$$\neg P.0 \wedge \neg P.1 \wedge \neg P.2 \wedge \neg P.3 \wedge \neg P.4 \wedge \dots$$

Otro esquema:

$$P.0 \wedge (\forall n : P.n \Rightarrow P.(n+2))$$

¿Vale este esquema? No!! Verifiquemos:

$$\neg P.0 \wedge P.1 \wedge \neg P.2 \wedge P.3 \wedge \neg P.4 \wedge \dots$$

A la hora de hacer derivaciones, debemos usar un esquema inductivo que nos garantice que la especificación vale para todos los elementos del dominio de mi función.

La definición de la función va a tener un **pattern matching** que es acorde al esquema inductivo usado.

Inducciones con naturales

Inducción básica:

$$f.0 \doteq ???$$

$$f.(n+1) \doteq ??? \text{ (en términos de } f)$$

$$fac.0 \doteq 1$$

$$fac.(n+1) \doteq (n+1) * fac.n$$

Inducción a lo fibonacci:

$$fib.0 \doteq 0$$

$$fib.1 \doteq 1$$

$$fib.(n+2) \doteq fib.(n+1) + fib.n$$

Otras inducciones:

$$f.1 \doteq 1$$

$$f.(n+2) \doteq (n+2) * f.n$$

(función sólo definida para los números impares)

(es una función que me acabo de inventar y no tengo idea qué calcula)

(ah, es como el factorial pero sólo de impares: $f.9 = 9 * 7 * 5 * 3 * 1$).

Inducciones con listas

Inducción básica:

$$f.[] \doteq ???$$

$$f.(x \blacktriangleright xs) \doteq ??? \text{ (en función de } f.xs)$$

(ejemplos: iga, sum)

Inducción con dos o más elementos:

$f.[] \doteq ???$

$f.[x] \doteq ???$

$f.(x \blacktriangleright y \blacktriangleright xs) \doteq ???$ (en función de $f.(y \blacktriangleright ys)$, o también $f.ys$)

(sirve para las funciones iguales, mínimo y creciente)

Inducciones combinadas: Sub-inducción

A veces vamos a tener más de un parámetro sobre el cual se puede aplicar inducción.

La estrategia general que vamos a tomar es hacer inducción en uno solo de ellos.

Posiblemente, en la derivación del caso base o del paso inductivo, surja la necesidad de hacer inducción también en el otro

Dos naturales:

Tenemos una función $f : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$.

Hacemos inducción en el primer parámetro:

- Caso base: $f.0.m \doteq ????$ (salió bien la derivación)
- Paso inductivo: $f.(n+1).m \doteq ???$ (acá nos trabamos al derivar y vemos que necesitamos hacer inducción también en m). Lo llamamos sub-inducción.
 - Caso base para m : $f.(n+1).0 \doteq ????$
 - Paso inductivo para m : $f.(n+1).(m+1) \doteq ???$ (en términos de $f.n.m$).

La función queda definida así:

$f.0.m \doteq ????$

$f.(n+1).0 \doteq ???$

$f.(n+1).(m+1) \doteq ???$ (en términos de $f.n.m$)

Dos listas: También puede haber necesidad de hacer subinducción.

Ejemplo: (práctico 2 ejercicio 5d) $\text{prod} : [\text{Num}] \rightarrow [\text{Num}] \rightarrow \text{Num}$, que calcula el producto entre pares de elementos en iguales posiciones de las listas y suma estos resultados (producto punto). Si las listas tienen distinto tamaño se opera hasta la última posición de la más chica.

$\text{prod}.xs.ys = \langle \sum i : 0 \leq i < \#xs \min \#ys : (xs ! i) * (ys ! i) \rangle$

Un posible resultado de derivar sería:

$\text{prod}.[].ys \doteq 0$

$\text{prod}.(x \blacktriangleright xs).[] \doteq 0$

$\text{prod}.(x \blacktriangleright xs).(y \blacktriangleright ys) \doteq x * y + \text{prod}.xs.ys$

Un natural y una lista: Se ve mucho en las funciones de listas estándar.

Ejemplos: Indexación:

$$(x \blacktriangleright xs) ! 0 \doteq x$$

$$(x \blacktriangleright xs) ! (n+1) \doteq xs ! n$$

(acá hay inducción en n)

Tirar: "xs ↓ n = tirar los primeros n elementos de xs"

$$[] \downarrow n \doteq []$$

$$(x \blacktriangleright xs) \downarrow 0 \doteq (x \blacktriangleright xs)$$

$$(x \blacktriangleright xs) \downarrow (n+1) \doteq xs \downarrow n$$

(acá hay inducción en los dos parámetros, primero en la lista, y luego **solo en el caso inductivo** hacemos también subinducción en el número.)

¿Cómo sería si hacemos primero inducción en el número?

$$xs \downarrow 0 \doteq xs$$

$$[] \downarrow (n+1) \doteq []$$

$$(x \blacktriangleright xs) \downarrow (n+1) \doteq xs \downarrow n$$

Esta definición es equivalente a la anterior pero usa un esquema inductivo distinto. Primero inducción en el número y después, dentro del caso inductivo, sub-inducción en la lista.

APARTADO: FUNCIONES CON NOTACION INFIJA, PREFIJA Y SUFIJA

Notación infija: Una función cuya llamada se pone al medio de los dos parámetros (como un operador): $x * y$, $xs ! 6$, etc.

Notación prefija: Una función cuya llamada se pone antes de los parámetros: $tirar.6.xs$, $tail.xs$, $iga.7.xs$, $\#xs$, etc.

Notación sufija: la función al final. Ejemplo: $n!$

Generalización

Ejemplo: Función psum

Práctico 3 - Ejercicio 1a:

Tenemos esta especificación:

$$\text{psum} : [\text{Num}] \rightarrow \text{Bool}$$

$$\text{psum}.xs = \langle \forall i : 0 \leq i \leq \#xs : \text{sum}.(xs \uparrow i) \geq 0 \rangle$$

¿Qué calcula esta función? En palabras:

Algo que no es: “Me dice si la suma de toda la lista es ≥ 0 ”.

Si fuera esto, sería $\text{psum.xs} = \text{sum.xs} \geq 0$

Una posible: “La suma de los n primeros elementos siempre es ≥ 0 para todo n posible.”

Otra: “Todos los segmentos iniciales de la lista suman ≥ 0 .”

Definición: Un segmento inicial (o prefijo) de una lista xs es una lista que está “al principio de xs ”. Formalmente, es una lista ys tal que: $xs = ys ++ as$ (para alguna otra lista as).

Ejemplo: Los segmentos iniciales de la lista $xs = [4, -3, 5, -7, 10]$ son:

$[], [4], [4, -3], [4, -3, 5], [4, -3, 5, -7], [4, -3, 5, 7, 10]$.

Observaciones:

- ¿Cuántos segmentos iniciales tiene una lista xs ? $\#xs + 1$
- Tanto la lista vacía $[]$ como la lista entera xs son siempre considerados segmentos iniciales.

Ejemplo: $xs = [4, -3, 5, -7, 10]$.

Lectura operacional para psum :

1. El rango es: $i \in \{0, 1, 2, 3, 4, 5\}$ (porque es con $\leq$)
2. Los términos son:

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 0}) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 1}) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 2}) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 3}) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 4}) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]^{\uparrow 5}) \geq 0 \quad \wedge$$

O sea, resolviendo $\uparrow$:

$$\text{sum}([]) \geq 0 \quad \wedge$$

$$\text{sum}([4]) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3]) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5]) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7]) \geq 0 \quad \wedge$$

$$\text{sum}([4, -3, 5, -7, 10]) \geq 0 \quad \wedge$$

Resolviendo las sumas:

$$0 \geq 0 \quad \wedge \quad 4 \geq 0 \quad \wedge \quad 1 \geq 0 \quad \wedge \quad 6 \geq 0 \quad \wedge \quad \underline{-1 \geq 0} \quad \wedge \quad 9 \geq 0$$

O sea:

False

Observación: En la especificación de psum , la variable cuantificada “ i ” indica cuántos elementos tiene el segmento inicial.

Derivación: Intentaremos hacerlo por inducción en xs .

Si resulta, el programa tendrá la forma:

$\text{psum}[] \doteq ???$

$\text{psum}(x \blacktriangleright xs) \doteq ???$

Caso base:

$\text{psum.}[\]$
 $= \{ \text{especificación} \}$
 $\langle \forall i : 0 \leq i \leq \#[\] : \text{sum.}([\] \uparrow i) \geq 0 \rangle$
 $= \{ \text{def. } \# \}$
 $\langle \forall i : 0 \leq i \leq 0 : \text{sum.}([\] \uparrow i) \geq 0 \rangle$
 $= \{ \text{lógica} \}$
 $\langle \forall i : i = 0 : \text{sum.}([\] \uparrow i) \geq 0 \rangle$
 $= \{ \text{rango unitario} \}$
 $\text{sum.}([\] \uparrow 0) \geq 0$
 $= \{ \text{def } \uparrow \text{ y de sum} \}$
 $0 \geq 0$
 $= \{ \text{lógica} \}$
 True

Paso inductivo: Hipótesis Inductiva: $\text{psum.xs} = \langle \forall i : 0 \leq i \leq \#xs : \text{sum.}(xs \uparrow i) \geq 0 \rangle$

$\text{psum.}(x \blacktriangleright xs)$
 $= \{ \text{especificación} \}$
 $\langle \forall i : 0 \leq i \leq \#(x \blacktriangleright xs) : \text{sum.}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
 $= \{ \text{def de } \# \}$
 $\langle \forall i : \underline{0 \leq i \leq \#xs + 1} : \text{sum.}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
 $= \{ \text{lógica} \}$
 $\langle \forall i : i = 0 \vee 1 \leq i \leq \#xs + 1 : \text{sum.}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
 $= \{ \text{partición de rango} \}$
 (observación: estamos separando el caso del segmento inicial vacío ($i = 0$) de los segmentos iniciales no vacíos ($1 \leq i \leq \#xs + 1$))
 $\}$
 $\langle \forall i : i = 0 : \text{sum.}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle \wedge$
 $\underline{\langle \forall i : 1 \leq i \leq \#xs + 1 : \text{sum.}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle}$
 $= \{ \text{Acá la prioridad es llegar a la H.I. así que nos concentramos en la segunda parte.} \}$
 Cambio de variable $i \rightarrow i + 1.$
 $\langle \forall \dots \rangle \wedge \langle \forall i : \underline{1 \leq i + 1 \leq \#xs + 1} : \text{sum.}((x \blacktriangleright xs) \uparrow (i + 1)) \geq 0 \rangle$
 $= \{ \text{arit: resto 1} \}$
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : \text{sum.}((x \blacktriangleright xs) \uparrow (i + 1)) \geq 0 \rangle$
 $= \{ \text{def de } \uparrow \}$
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : \text{sum.}(x \blacktriangleright (xs \uparrow i)) \geq 0 \rangle$
 $= \{ \text{def de sum} \}$
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : \underline{x +} \text{sum.}((xs \uparrow i)) \geq 0 \rangle$

Recordemos la H.I.: $\text{psum.xs} = \langle \forall i : 0 \leq i \leq \#xs : \text{sum.}(xs \uparrow i) \geq 0 \rangle$.

No hay forma de llegar a la H.I., no hay nada que podamos hacer para deshacernos de ese cacho "x + " que molesta.

No podemos seguir. Peeeeeeeeeeeeero: Si mi especificación original hubiera sido ligeramente diferente, acá capaz sí podría funcionar la H.I.

Podría **especificar** por ejemplo así:

$\text{gpsum.n.xs} = \langle \forall i : 0 \leq i \leq \#xs : n + \text{sum.}(xs \uparrow i) \geq 0 \rangle$

¿Qué acabo de hacer acá? Acabo de especificar una función nueva, parecida a la psum pero que tiene un parámetro adicional “n” (que además aparece sumado en un lugar que va a ser conveniente para la derivación).

¿Cómo se relaciona gpsum con psum?

- psum.xs **es caso particular** de gpsum.n.xs cuando $n = 0$. Más formalmente:
 $\text{psum.xs} = \text{gpsum.0.xs}$.
- gpsum **generaliza** a psum

Si yo tuviera un programa para gpsum ya resuelto, podría derivar psum y obtener un programa de la siguiente manera:

Nueva derivación de psum:

psum.xs
= { especificación psum }
 $\langle \forall i : 0 \leq i \leq \#xs : \text{sum.}(xs \uparrow i) \geq 0 \rangle$
= { aritmética }
 $\langle \forall i : 0 \leq i \leq \#xs : 0 + \text{sum.}(xs \uparrow i) \geq 0 \rangle$
= { especificación gpsum }
gpsum.0.xs

Resultado: $\text{psum.xs} \doteq \text{gpsum.0.xs}$

Esto quiere decir que la derivación por inducción que habíamos intentado antes **se descarta**:

~~psum.[] $\doteq$???~~
~~psum.(x $\blacktriangleright$ xs) $\doteq$???~~

Sólo me sirvió para darme cuenta de que debo generalizar.
(por eso conviene empezar por el paso inductivo, para ahorrar tiempo)

¿Podemos derivar un programa para gpsum? Intentemos.

Derivación de gpsum: Especificación:

$\text{gpsum.n.xs} = \langle \forall i : 0 \leq i \leq \#xs : n + \text{sum.}(xs \uparrow i) \geq 0 \rangle$

Parecido a lo que hicimos con psum. Intentamos con inducción en xs.

El programa tendrá la forma:

gpsum.n.[] $\doteq$???
gpsum.n.(x $\blacktriangleright$ xs) $\doteq$???

Caso base:

gpsum.n.[]
= { especificación }
 $\langle \forall i : 0 \leq i \leq \#[] : n + \text{sum.}([] \uparrow i) \geq 0 \rangle$
= { def. # }
 $\langle \forall i : 0 \leq i \leq 0 : n + \text{sum.}([] \uparrow i) \geq 0 \rangle$
= { lógica }
 $\langle \forall i : i = 0 : n + \text{sum.}([] \uparrow i) \geq 0 \rangle$

= { rango unitario }
 $n + \text{sum}.\langle \uparrow 0 \rangle \geq 0$
= { def $\uparrow$ y de sum }
 $n + 0 \geq 0$
= { arit }
 $n \geq 0$

¡Listo este caso!

Paso inductivo: H.I.: $\forall E : \text{gpsum}.E.xs = \langle \forall i : 0 \leq i \leq \#xs : E + \text{sum}.(xs \uparrow i) \geq 0 \rangle$

$\text{gpsum}.n.(x \blacktriangleright xs)$
= { especificación }
 $\langle \forall i : 0 \leq i \leq \#(x \blacktriangleright xs) : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
= { def de # }
 $\langle \forall i : 0 \leq i \leq \#xs + 1 : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
= { lógica }
 $\langle \forall i : i = 0 \vee 1 \leq i \leq \#xs + 1 : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
= { partición de rango
(observación: de nuevo estamos separando el caso del segmento inicial vacío ($i = 0$) de los segmentos iniciales no vacíos ($1 \leq i \leq \#xs + 1$))
}
 $\langle \forall i : i = 0 : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle \wedge$
 $\langle \forall i : 1 \leq i \leq \#xs + 1 : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle$
= { **Acá la prioridad es llegar a la H.I. así que nos concentramos en la segunda parte.**
Cambio de variable $i \rightarrow i + 1$ }
 $\langle \forall \dots \rangle \wedge \langle \forall i : 1 \leq i + 1 \leq \#xs + 1 : n + \text{sum}((x \blacktriangleright xs) \uparrow (i + 1)) \geq 0 \rangle$
= { arit: resto 1 }
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : n + \text{sum}((x \blacktriangleright xs) \uparrow (i + 1)) \geq 0 \rangle$
= { def de $\uparrow$ }
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : n + \text{sum}.(x \blacktriangleright (xs \uparrow i)) \geq 0 \rangle$
= { def de sum }
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : n + (x + \text{sum}((xs \uparrow i))) \geq 0 \rangle$
= { **¿Podemos llegar a la H.I.? Aplicamos asociatividad.** }
 $\langle \forall \dots \rangle \wedge \langle \forall i : 0 \leq i \leq \#xs : (n + x) + \text{sum}((xs \uparrow i)) \geq 0 \rangle$
= { Ahora sí H.I. y volvemos a mirar la primera parte. }
 $\langle \forall i : i = 0 : n + \text{sum}((x \blacktriangleright xs) \uparrow i) \geq 0 \rangle \wedge \text{gpsum}.(n + x).xs$
= { Rango unitario y más pasos como el caso base. }
 $n \geq 0 \wedge \text{gpsum}.(n + x).xs$

¡¡Listo!! Ya tenemos un programa para gpsum.

Resultado final de **todo el ejercicio**:

$\text{psum}.xs \doteq \text{gpsum}.0.xs$

$\text{gpsum}.n.[] \doteq n \geq 0$

$\text{gpsum}.n.(x \blacktriangleright xs) \doteq n \geq 0 \wedge \text{gpsum}.(n + x).xs$

¿Cómo funciona este programa? Es raro porque si bien la especificación usa sum y ↑ (tomar), en la definición no se usan para nada.

Verificación / testing: $xs = [4, -3, 5, -7, 10]$.

Recordemos el programa:

$psum.xs \doteq gsum.0.xs$

$gsum.n.[] \doteq n \geq 0$

$gsum.n.(x \blacktriangleright xs) \doteq n \geq 0 \wedge gsum.(n+x).xs$

$psum.[4, -3, 5, -7, 10] \rightarrow gsum.0.[4, -3, 5, -7, 10]$
 $\rightarrow 0 \geq 0 \wedge \underline{gsum.(0+4).[-3, 5, -7, 10]}$
 $\rightarrow 0 \geq 0 \wedge (4 \geq 0 \wedge \underline{gsum.(4+(-3)).[5, -7, 10]})$
 $\rightarrow 0 \geq 0 \wedge (4 \geq 0 \wedge (4+(-3) \geq 0 \wedge \underline{gsum.(4+(-3)+5).[-7, 10]}))$
 $\rightarrow \dots$ (ejercicio: completar!)
 $\rightarrow 0 \geq 0 \wedge 4 \geq 0 \wedge 4+(-3) \geq 0 \wedge 4+(-3)+5 \geq 0 \wedge$
 $4+(-3)+5+(-7) \geq 0 \wedge 4+(-3)+5+(-7)+10 \geq 0$
 $\rightarrow \dots$
 $\rightarrow \text{False}$

Vemos que en el primer parámetro de gsum se va “guardando” la suma de los elementos que ya pasaron (o mejor dicho de los segmentos iniciales). Este parámetro se suele llamar **acumulador**.

Resumen: ¿Qué hicimos?

En una derivación por inducción, en el paso inductivo nos trabamos y no hubo forma de llegar a la H.I. Pero notamos que si definimos una función más general con un parámetro adicional, la H.I. sí se podría aplicar y la derivación saldría bien. Entonces, **especificamos y derivamos (por inducción) esta nueva función, y a la función original la definimos directamente en una línea** como el caso particular de la función más general.

Observación: El intento de definición por inducción que hicimos para la función original se descarta. Sólo sirvió para darnos cuenta de que debo generalizar.

En el ejercicio, hicimos este intento de definición pero fracasó:

$psum.[] \doteq ???$

$psum.(x \blacktriangleright xs) \doteq ???$

Como al final el problema se resolvió con generalización, la definición sale en una línea (*one-liner*):

$psum.xs \doteq gsum.0.xs$

APARTADO: OTRA SOLUCIÓN PARA PSUM

Una solución distinta sale de generalizar:

$$\text{gpsum2.n.xs} \doteq \langle \forall i : 0 \leq i \leq n : \text{sum}.\text{xs} \uparrow i \geq 0 \rangle$$

Se debe derivar gpsum2 por inducción en n (**no en xs**).

El resultado de la derivación es:

$$\text{psum.xs} \doteq \text{gpsum2}.\text{(#xs).xs}$$

$$\text{gpsum2.0.xs} \doteq \text{True}$$

$$\text{gpsum2}.\text{(n+1).xs} \doteq \text{sum}.\text{(xs} \uparrow \text{(n+1))} \geq 0 \wedge \text{gpsum2.n.xs}$$

Esta definición es más parecida a la especificación, pero es más ineficiente que la solución con gpsum, ya que debe calcular explícitamente sum y $\uparrow$ muchas veces.

Más ejemplos

Generalización con dos parámetros nuevos

<https://docs.google.com/document/d/1cOvl0hd8SyacGPHtvQxON4FpRo3kQW6z1vq2aQb7Q14/edit>

$$\text{h.xs} = \langle \exists \text{as, bs} : \text{xs} = \text{as} ++ \text{bs} : \text{sum.as} = \text{\#as} + 1 \rangle$$

Salteamos cosas y vamos derecho al paso inductivo:

$$\begin{aligned} & \text{h.(x} \blacktriangleright \text{xs)} \\ &= \{ \text{especificación} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} : \text{sum.as} = \text{\#as} + 1 \rangle \\ &= \{ \text{3ro excluído} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge (\text{as} = [] \vee \text{as} \neq []) : \text{sum.as} = \text{\#as} + 1 \rangle \\ &= \{ \text{distrib. y part. rango} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge \text{as} = [] : \text{sum.as} = \text{\#as} + 1 \rangle \vee \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge \text{as} \neq [] : \text{sum.as} = \text{\#as} + 1 \rangle \\ &= \{ \text{cambio var. as} \rightarrow \text{a} \blacktriangleright \text{as} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge \text{as} = [] : \text{sum.as} = \text{\#as} + 1 \rangle \vee \\ & \langle \exists \text{a, as, bs} : \text{x} \blacktriangleright \text{xs} = (\text{a} \blacktriangleright \text{as}) ++ \text{bs} \wedge \text{a} \blacktriangleright \text{as} \neq [] : \text{sum}.\text{(a} \blacktriangleright \text{as)} = \text{\#(a} \blacktriangleright \text{as)} + 1 \rangle \\ &= \{ \text{varias propiedades de listas} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge \text{as} = [] : \text{sum.as} = \text{\#as} + 1 \rangle \vee \\ & \langle \exists \text{a, as, bs} : \text{x} = \text{a} \wedge \text{xs} = \text{as} ++ \text{bs} : \text{sum}.\text{(a} \blacktriangleright \text{as)} = \text{\#(a} \blacktriangleright \text{as)} + 1 \rangle \\ &= \{ \text{eliminación de variable a} \} \\ & \langle \exists \text{as, bs} : \text{x} \blacktriangleright \text{xs} = \text{as} ++ \text{bs} \wedge \text{as} = [] : \text{sum.as} = \text{\#as} + 1 \rangle \vee \\ & \langle \exists \text{as, bs} : \text{xs} = \text{as} ++ \text{bs} : \text{sum}.\text{(x} \blacktriangleright \text{as)} = \text{\#(x} \blacktriangleright \text{as)} + 1 \rangle \end{aligned}$$

$= \{ \text{def sum y def \#} \}$
 $\langle \exists \text{ as, bs : } x \blacktriangleright xs = \text{as ++ bs} \wedge \text{as} = [] : \text{sum.as} = \#as + 1 \rangle \vee$
 $\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : x + \text{sum.as} = (\#as + 1) + 1 \rangle$
 $= \{ \text{arit.} \}$
 $\langle \exists \text{ as, bs : } x \blacktriangleright xs = \text{as ++ bs} \wedge \text{as} = [] : \text{sum.as} = \#as + 1 \rangle \vee$
 $\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : x + \text{sum.as} = \#as + 1 + 1 \rangle$

Acá me trabo. No hay forma de llegar a la H.I. Debo generalizar.

Opción 1: Metemos dos variables

$\text{genh.xs.n.m} = \langle \exists \text{ as, bs : } xs = \text{as ++ bs} : n + \text{sum.as} = \#as + 1 + m \rangle$

genh generaliza a h ya que: $\text{genh.xs.0.0} = h.xs$

Opción 2: Si paso restando el 1 para la izquierda y lo junto con el x me queda:

$\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : (\underline{x - 1}) + \text{sum.as} = \#as + 1 \rangle$

Así que puedo generalizar así:

$\text{genh.xs.n} = \langle \exists \text{ as, bs : } xs = \text{as ++ bs} : n + \text{sum.as} = \#as + 1 \rangle$

genh generaliza a h ya que: $\text{genh.xs.0} = h.xs$

Segmentos de lista

Definiciones

Segmento / sublista: Un segmento de una lista es otra lista que está “contenida” en la lista original. Formalmente, una lista ys es *segmento* (también *sublista*) de una lista xs si y sólo si:

existen as, bs tales que $xs = \text{as ++ ys ++ bs}$.

(o sea, puedo obtener xs agarrando ys y concatenandole una lista a la izquierda y otra a la derecha)

¿Es xs segmento de xs? Sí. En este caso $\text{as} = []$, $\text{bs} = []$.

¿Es [] segmento de xs? Sí. En este caso $\text{as} = xs$, $\text{bs} = []$ o también $\text{as} = []$, $\text{bs} = xs$, o también muchas otras posibilidades...

¿Cuántos segmentos tiene una lista xs?

Propuesta: $\#xs + 1$?? No.

Ejemplo: $xs = [-3, 2, -7]$.

Segmentos: $[], [-3], [2], [-7], [-3, 2], [2, -7], [-3, 2, -7]$.

En este caso son **7 segmentos únicos**.

Ojo porque hay dos formas de contar: Si $xs = [-3, 2, -3]$, tengo el segmento $[-3]$ dos veces. Contar segmentos únicos no se puede en general porque puede haber segmentos repetidos. Lo que vamos a contar es todas formas posibles de dividir la lista xs en una concatenación $as ++ ys ++ bs$:

$xs = [-3, 2, -7] = as ++ ys ++ bs$

as	ys	bs
$[]$	$[]$	$[-3, 2, -7]$
$[]$	$[-3]$	$[2, -7]$
$[]$	$[-3, 2]$	$[-7]$
$[]$	$[-3, 2, -7]$	$[]$
$[-3]$	$[]$	$[2, -7]$
$[-3]$	$[2]$	$[-7]$
$[-3]$	$[2, -7]$	$[]$
$[-3, 2]$	$[]$	$[-7]$
$[-3, 2]$	$[-7]$	$[]$
$[-3, 2, 7]$	$[]$	$[]$

En total obtuvimos 10 formas distintas de dividir en 3 partes una lista de 3 elementos.

Observación: Si nos fijamos en la columna para ys obtenemos:

$[], [-3], [-3, 2], [-3, 2, -7], [], [2], [2, -7], [], [-7], []$.

¿Cuántas veces aparece la lista $[]$ como segmento de xs ? Aparece $\#xs + 1$ veces.

Ejercicio: Resolver usando conteo una fórmula general para la cantidad de combinaciones posibles (respuesta: $(\#xs + 1) * (\#xs + 2) / 2$).

Segmento inicial / prefijo: Un segmento inicial de una lista es un segmento que además está al principio de la lista. Formalmente, una lista ys es *segmento inicial* (también *prefijo*) de una lista xs si y sólo si:

existe as tal que $xs = ys ++ as$.

¿Es xs segmento inicial de xs ? Sí, con $as = []$

¿Es $[]$ segmento inicial de xs ? Sí, con $as = xs$

¿Cuántos segmentos iniciales tiene una lista xs ? $\#xs + 1$.

Segmento final / sufijo: Un segmento final de una lista es un segmento que además está al final de la lista. Formalmente, una lista ys es *segmento final* (también *sufijo*) de una lista xs si y sólo si:

existe as tal que $xs = as ++ ys$.

¿Es xs segmento final de xs ? Sí, con $as = []$

¿Es $[]$ segmento final de xs ? Sí, con $as = xs$

¿Cuántos segmentos finales tiene una lista xs ? $\#xs + 1$.

Derivaciones con segmentos iniciales

Habiendo visto estas definiciones ya estamos listos para usar estos conceptos para especificar y derivar problemas con segmentos de lista.

Práctico 3 - Ejercicio 1a:

$psum : [Num] \rightarrow Bool$

$psum.xs = \langle \forall i : 0 \leq i \leq \#xs : sum.(xs \upharpoonright i) \geq 0 \rangle$

¿Qué calcula esta función? En palabras:

“Todos los **segmentos iniciales** de la lista suman ≥ 0 .”

Esta especificación usa la función `tomar`, pero podemos reescribirla usando la definición que conocemos para segmentos iniciales:

$psum : [Num] \rightarrow Bool$

$psum.xs = \langle \forall as : \text{“}as \text{ es segmento inicial de } xs\text{”} : sum.as \geq 0 \rangle$

Refinando:

$psum : [Num] \rightarrow Bool$

$psum.xs = \langle \forall as : \langle \exists bs : xs = as ++ bs \rangle : sum.as \geq 0 \rangle$

O equivalentemente, y mucho más simple:

$psum : [Num] \rightarrow Bool$

$psum.xs = \langle \forall as, bs : xs = as ++ bs : sum.as \geq 0 \rangle$

¿Estamos seguros de que esta especificación es equivalente a la original?

Verifiquemos con un ejemplo: $xs = [4, -3, 5, -7, 10]$.

Lectura operacional para $psum$:

1. El rango es: $(as, bs) \in \{ ([], [4, -3, 5, -7, 10]), ([4] , [-3, 5, -7, 10]), ([4, -3] , [5, -7, 10]), ([4, -3, 5] , [-7, 10]), ([4, -3, 5, -7] , [10]), ([4, -3, 5, -7, 10] , []) , \}$
2. Aplicamos el término estas 6 veces:
 $sum.[] \geq 0 \wedge$
 $sum.[4] \geq 0 \wedge$
 $sum.[4, -3] \geq 0 \wedge$
 $sum.[4, -3, 5] \geq 0 \wedge$
 $sum.[4, -3, 5, -7] \geq 0 \wedge$
 $sum.[4, -3, 5, -7, -10] \geq 0$

El resto queda como ejercicio pero ya se ve que es igual a la especificación original.

Recordemos la especificación:

$$psum.xs = \langle \forall as, bs : xs = as ++ bs : sum.as \geq 0 \rangle$$

Vamos a derivar por inducción en xs. Hacemos primero el paso inductivo para ver si podemos llegar a la H.I.

Paso inductivo: La H.I. es $psum.xs = \langle \forall as, bs : xs = as ++ bs : sum.as \geq 0 \rangle$

$psum.(x \blacktriangleright xs)$
 $= \{ \text{especificación} \}$
 $\langle \forall as, bs : x \blacktriangleright xs = as ++ bs : sum.as \geq 0 \rangle$
 $= \{ \text{NUEVA ESTRATEGIA: Acá vamos a preguntar por "as". ¿Es vacía o no?}$
 $\text{Sale en varios pasos: 1er paso: lógica} \}$
 $\langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge \text{True} : sum.as \geq 0 \rangle$
 $= \{ \text{lógica (3ro excluido)} \}$
 $\langle \forall as, bs : \underline{x \blacktriangleright xs = as ++ bs \wedge (as = [] \vee as \neq [])} : sum.as \geq 0 \rangle$
 $= \{ \text{distributividad} \}$
 $\langle \forall as, bs : (x \blacktriangleright xs = as ++ bs \wedge as = []) \vee (x \blacktriangleright xs = as ++ bs \wedge as \neq []) : sum.as \geq 0 \rangle$
 $= \{ \text{partición de rango}$
 $\text{(observación: de nuevo estamos separando el caso del segmento inicial vacío (as = [])$
 $\text{de los segmentos iniciales no vacíos (as \neq []))}$
 $\}$
 $\langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : sum.as \geq 0 \rangle$
 $\wedge \langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge as \neq [] : sum.as \geq 0 \rangle$
 $= \{ \text{Acá la prioridad es llegar a la H.I. así que nos concentramos en la segunda parte.}$
 $\text{Como as es no vacía (as \neq []) tiene la forma c \blacktriangleright cs.}$
 $\text{Podemos hacer el siguiente cambio de variable: as} \leftarrow c \blacktriangleright cs.$
 $\text{Mejor reusamos el nombre "as": cambio de variable as} \leftarrow a \blacktriangleright as. \}$
 $\langle \forall \dots \rangle \wedge \langle \forall a, as, bs : x \blacktriangleright xs = (a \blacktriangleright as) ++ bs \wedge \underline{a \blacktriangleright as \neq []} : sum.(a \blacktriangleright as) \geq 0 \rangle$
 $= \{ \text{prop. listas} \}$

$$\begin{aligned}
& \langle \forall \dots \rangle \wedge \langle \forall a, as, bs : x \blacktriangleright xs = (a \blacktriangleright as) ++ bs \wedge \text{True} : \text{sum}.(a \blacktriangleright as) \geq 0 \rangle \\
&= \{ \text{lógica} \} \\
& \langle \forall \dots \rangle \wedge \langle \forall a, as, bs : x \blacktriangleright xs = \underline{(a \blacktriangleright as) ++ bs} : \text{sum}.(a \blacktriangleright as) \geq 0 \rangle \\
&= \{ \text{prop. listas (def. ++)} \} \\
& \langle \forall \dots \rangle \wedge \langle \forall a, as, bs : x \blacktriangleright xs = a \blacktriangleright (as ++ bs) : \text{sum}.(a \blacktriangleright as) \geq 0 \rangle \\
&= \{ \text{prop. listas} \} \\
& \langle \dots \rangle \wedge \langle \forall a, as, bs : x = a \wedge xs = as ++ bs : \text{sum}.(a \blacktriangleright as) \geq 0 \rangle \\
&= \{ \text{eliminación de variable} \} \\
& \langle \forall \dots \rangle \wedge \langle \forall as, bs : xs = as ++ bs : \text{sum}.(x \blacktriangleright as) \geq 0 \rangle \\
&= \{ \text{def. de sum} \} \\
& \langle \forall \dots \rangle \wedge \langle \forall as, bs : xs = as ++ bs : x + \text{sum}.as \geq 0 \rangle
\end{aligned}$$

Acá de nuevo nos pasa que no hay nada que podamos hacer para llegar a la Hipótesis Inductiva. **Debemos generalizar:**

$$\text{gpsum}.n.xs = \langle \forall as, bs : xs = as ++ bs : n + \text{sum}.as \geq 0 \rangle$$

gpsum generaliza a psum ya que $\text{gpsum}.0.xs = \text{psum}.xs$ (ejercicio: demostrar! igual sale en un paso). Luego, podemos definir (o sea, este es el programa para psum):

psum.xs $\doteq$ gpsum.0.xs

(y descartamos el intento de derivar por inducción psum)

Falta derivar gpsum. Lo haremos por inducción en xs.

Paso inductivo: H.I.: para todo E : $\text{gpsum}.E.xs = \langle \forall as, bs : xs = as ++ bs : E + \text{sum}.as \geq 0 \rangle$

$$\begin{aligned}
& \text{gpsum}.n.(x \blacktriangleright xs) \\
&= \{ \text{especificación} \} \\
& \langle \forall as, bs : x \blacktriangleright xs = as ++ bs : n + \text{sum}.as \geq 0 \rangle \\
&= \{ \text{mismos pasos que hicimos antes solo que con el "n +"} \text{ agregado} \} \\
& \langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : n + \text{sum}.as \geq 0 \rangle \\
&\wedge \langle \forall as, bs : xs = as ++ bs : n + (x + \text{sum}.as) \geq 0 \rangle \\
&= \{ \text{asociatividad} \} \\
& \langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : n + \text{sum}.as \geq 0 \rangle \\
&\wedge \underline{\langle \forall as, bs : xs = as ++ bs : (n + x) + \text{sum}.as \geq 0 \rangle} \\
&= \{ \text{H.I. con } E = n+x \} \\
& \langle \forall as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : n + \text{sum}.as \geq 0 \rangle \\
&\wedge \text{gpsum}.(n + x).xs \\
&= \{ \text{eliminación de variable as} \} \\
& \langle \forall bs : x \blacktriangleright xs = [] ++ bs : n + \text{sum}.[] \geq 0 \rangle \\
&\wedge \text{gpsum}.(n + x).xs \\
&= \{ \text{def. ++} \} \\
& \langle \forall bs : x \blacktriangleright xs = bs : n + \text{sum}.[] \geq 0 \rangle \\
&\wedge \text{gpsum}.(n + x).xs \\
&= \{ \text{rango unitario!! (siempre es más seguro aplicar rango unitario que término constante)} \}
\end{aligned}$$

$$\begin{aligned}
& n + \text{sum}[_] \geq 0 \quad \wedge \quad \text{gpsum}.(n + x). \text{xs} \\
= & \{ \text{def. sum} \} \\
& n + 0 \geq 0 \quad \wedge \quad \text{gpsum}.(n + x). \text{xs} \\
= & \{ \text{arit.} \} \\
& n \geq 0 \quad \wedge \quad \text{gpsum}.(n + x). \text{xs}
\end{aligned}$$

Ahora sí terminamos el paso inductivo.

Caso base:

$$\begin{aligned}
& \text{gpsum}.n.[_] \\
= & \{ \text{especificación} \} \\
& \langle \forall \text{ as, bs} : [_] = \text{as} ++ \text{bs} : n + \text{sum.as} \geq 0 \rangle \\
= & \{ \text{prop. listas} \} \\
& \langle \forall \text{ as, bs} : [_] = \text{as} \wedge [_] = \text{bs} : n + \text{sum.as} \geq 0 \rangle \\
= & \{ \text{eliminación de variable as} \} \\
& \langle \forall \text{ bs} : [_] = \text{bs} : n + \text{sum}[_] \geq 0 \rangle \\
= & \{ \text{rango unitario} \} \\
& n + \text{sum}[_] \geq 0 \\
= & \{ \dots \} \\
& n \geq 0
\end{aligned}$$

Listo!! Resultado final:

$$\begin{aligned}
\text{psum.xs} & \doteq \text{gpsum}.0.\text{xs} \\
\text{gpsum}.n.[_] & \doteq n \geq 0 \\
\text{gpsum}.n.(x \blacktriangleright \text{xs}) & \doteq n \geq 0 \quad \wedge \quad \text{gpsum}.(n + x). \text{xs}
\end{aligned}$$

Obtuvimos exactamente el mismo programa que habíamos obtenido para la otra especificación.

Ejercicio: Repasar la verificación (el testing).

Derivaciones con segmentos finales

La saltamos pero la estrategia es igual a la de segmentos iniciales.

Derivaciones con segmentos arbitrarios

Veremos que salen siempre con una modularización, donde la función modularizada resuelve el mismo problema pero para segmentos iniciales.

Ejemplo: Segmento de suma máxima: “Dada una lista xs, considerar todos los segmentos posibles y obtener la suma de aquel que tiene suma máxima”.

Especificación:

$\text{sumax.xs} = \langle \text{Max bs} : \text{“bs es segmento de xs”} : \text{sum.bs} \rangle$

Refinando:

$\text{sumax.xs} = \langle \text{Max bs} : \langle \exists \text{ as, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} \rangle : \text{sum.bs} \rangle$

O más simple:

$\text{sumax.xs} = \langle \text{Max as, bs, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$

Me quedo con esta especificación.

Ejemplo: $\text{xs} = [-3, 2, -7]$.

1. Lectura operacional: $(\text{as}, \text{bs}, \text{cs}) \in \{ ([], [], [-3, 2, 7]) \dots \}$ (y así como en la tabla que hicimos antes, son 10 tuplas).
2. Aplicamos el término a los bs: $[], [-3], [-3, 2], [-3, 2, -7], [], [2], [2, -7], [-7], []$.
 $\text{sum.[]} \text{ max}$
 $\text{sum.}[-3] \text{ max}$
 $\dots \text{ max}$
 $\text{sum.}[]$
3. Al final me queda: 2. (ejercicio: verificar)

Derivación de sumax:

Recordemos la especificación:

$\text{sumax.xs} = \langle \text{Max as, bs, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$

Hacemos inducción en xs.

Paso inductivo: H.I.: $\text{sumax.xs} = \langle \text{Max as, bs, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$

$\text{sumax.}(x \blacktriangleright \text{xs})$
 $= \{ \text{especificación} \}$
 $\langle \text{Max as, bs, cs} : x \blacktriangleright \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$
 $= \{ \text{aplicamos la estrategia: preguntamos por as (3ro excluido)} \}$
 $\langle \text{Max as, bs, cs} : x \blacktriangleright \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} \wedge (\text{as} = [] \vee \text{as} \neq []) : \text{sum.bs} \rangle$
 $= \{ \text{distributividad y partición de rango} \}$
 $\langle \text{Max as, bs, cs} : x \blacktriangleright \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} \wedge \text{as} = [] : \text{sum.bs} \rangle \text{max}$
 $\langle \text{Max as, bs, cs} : x \blacktriangleright \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} \wedge \text{as} \neq [] : \text{sum.bs} \rangle$
 $= \{ \text{seguimos con la estrategia: cambio de variable: as} \leftarrow a \blacktriangleright \text{as} \}$
 $\langle \text{Max as, bs, cs} : x \blacktriangleright \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} \wedge \text{as} = [] : \text{sum.bs} \rangle \text{max}$
 $\langle \text{Max a, as, bs, cs} : x \blacktriangleright \text{xs} = (a \blacktriangleright \text{as}) ++ \text{bs} ++ \text{cs} \wedge (a \blacktriangleright \text{as}) \neq [] : \text{sum.bs} \rangle$


```

= { prop listas }
  < Max as, bs, cs : x ▶ xs = as ++ bs ++ cs  ∧  as = [] : sum.bs  >max
  < Max a, as, bs, cs : x ▶ xs = (a ▶ as) ++ bs ++ cs : sum.bs  >
= { más prop listas }
  < Max as, bs, cs : x ▶ xs = as ++ bs ++ cs  ∧  as = [] : sum.bs  >max
  < Max a, as, bs, cs : x ▶ xs = a ▶ (as ++ bs ++ cs) : sum.bs  >
= { más prop listas }
  < Max as, bs, cs : x ▶ xs = as ++ bs ++ cs  ∧  as = [] : sum.bs  >max
  < Max a, as, bs, cs : x = a  ∧  xs = as ++ bs ++ cs : sum.bs  >
= { eliminación de variable a }
  < Max as, bs, cs : x ▶ xs = as ++ bs ++ cs  ∧  as = [] : sum.bs  >max
  < Max as, bs, cs : xs = as ++ bs ++ cs : sum.bs  >
= { Hipótesis Inductiva! no hace falta generalizar }
  < Max as, bs, cs : x ▶ xs = as ++ bs ++ cs  ∧  as = [] : sum.bs  >max
    sumax.xs
= { eliminación de variable as }
  < Max bs, cs : x ▶ xs = [] ++ bs ++ cs : sum.bs  >max  sumax.xs
= { prop. listas }
  < Max bs, cs : x ▶ xs = bs ++ cs : sum.bs  >max  sumax.xs
= { no es rango unitario ¿qué es?
    posible estrategia: continuar con bs = [] ó bs ≠ [] (ejercicio: probar!).
    igual no tiene mucho sentido porque no tengo H.I. a la que quiero llegar.
    La expresión cuantificada que queda molestando es exactamente el mismo problema
    original pero esta vez sólo sobre segmentos iniciales.
    Es un problema accesorio al problema original (o un subproblema).
    ¡¡Debemos modularizar!! Primero especificamos una nueva función sumaxp:
      sumaxp.xs = < Max bs, cs : xs = bs ++ cs : sum.bs  >
    Ahora llamemos a esta función en lo que estamos derivando.
  }
    sumaxp.(x ▶ xs)  max  sumax.xs

```

Listo el paso inductivo de sumax!! **Falta el caso base (ejercicio).** El resultado parcial es:

```

sumax.[ ]      ≡ ???
sumax.(x ▶ xs) ≡ sumaxp.(x ▶ xs) max  sumax.xs

```

Falta derivar sumaxp. Ejercicio.

(sale parecido a psum pero en este caso no hace falta generalizar gracias a que las suma distribuye con Max).

Resultado final:

```

sumax.[ ]      ≡ 0
sumax.(x ▶ xs) ≡ sumaxp.(x ▶ xs) max  sumax.xs

sumaxp.[ ]      ≡ 0
sumaxp.(x ▶ xs) ≡ 0 max (x + sumaxp.xs)

```

Testing: TODO

Otro ejemplo: TODO: ssum (como psum pero para segmentos en general)

Rangos con pares de elementos de lista

Ver también:

https://wiki.cs.famaf.unc.edu.ar/lib/exe/fetch.php?media=algo1:2017-2:consejos_funcional.pdf

Problema: Dada una lista, considerar todas las formas posibles de tomar dos elementos, y contar cuántas veces éstos son iguales:

$f : [\text{Num}] \rightarrow \text{Nat}$

$f.xs = \langle N \ i, j: 0 \leq i < j < \#xs: xs[i] = xs[j] \rangle$

Ejemplo: $xs = [2, 1, 3, 2, 3, 2]$.

$(i, j) \in \{ (0, 1), (0, 2), \underline{(0, 3)}, (0, 4), \underline{(0, 5)},$
 $(1, 2), (1, 3), (1, 4), (1, 5),$
 $(2, 3), \underline{(2, 4)}, (2, 5),$
 $(3, 4), \underline{(3, 5)},$
 $(4, 5) \}$

Resultado: 4 (coinciden en las posiciones: (0, 3), (0, 5), (2, 4), (3, 5)).

Más gráficamente:

$i \setminus j$	0	1	2	3	4	5
0		x	x	x	x	x
1			x	x	x	x
2				x	x	x
3					x	x
4						x

Derivación: Por inducción en xs .

$f.[] \doteq ???$

$f.(x \blacktriangleright xs) \doteq ???$

Paso inductivo:

$f.(x \blacktriangleright xs)$
 $= \{ \text{esp. } f \}$

$\langle N i, j : 0 \leq i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { def # }
 $\langle N i, j : 0 \leq i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { lógica } (MAL)
 $\langle N i, j : i = 0 \vee 1 \leq i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$ (MAL)
 (este paso está mal, la parte “i = 0” no dice nada sobre j, debemos proceder con mayor cuidado)
 = { reescribo desigualdad para separar $0 \leq i$ }
 $\langle N i, j : 0 \leq i \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { ahora sí lógica sobre $0 \leq i$ }
 $\langle N i, j : (i = 0 \vee 1 \leq i) \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { distributiva }
 $\langle N i, j : (i = 0 \wedge i < j < \#xs + 1) \vee (1 \leq i \wedge i < j < \#xs + 1) : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { partición de rango }
 $\langle N i, j : i = 0 \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle +$
 $\langle N i, j : 1 \leq i \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { lógica: vuelvo a juntar desigualdades }
 $\langle N i, j : i = 0 \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle +$
 $\langle N i, j : 1 \leq i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle$
 = { dos cambios de variable $i \leftarrow i + 1, j \leftarrow j + 1$ }
 $\langle N i, j : i = 0 \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle +$
 $\langle N i, j : 1 \leq i + 1 < j + 1 < \#xs + 1 : (x \blacktriangleright xs)!(i+1) = (x \blacktriangleright xs)!(j+1) \rangle$
 = { aritmética en el rango, def. ! en el término }
 $\langle N i, j : i = 0 \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle +$
 $\langle N i, j : 0 \leq i < j < \#xs : xs!i = xs!j \rangle$
 = { H.I. }
 $\langle N i, j : i = 0 \wedge i < j < \#xs + 1 : (x \blacktriangleright xs)!i = (x \blacktriangleright xs)!j \rangle +$
 f.xs
 = { eliminación de variable i }
 $\langle N j : 0 < j < \#xs + 1 : (x \blacktriangleright xs)!0 = (x \blacktriangleright xs)!j \rangle + f.xs$
 = { (acá ya podríamos modularizar, pero mejor hacer unos pasos para simplificar primero) }
 lógica: $0 < j$ es lo mismo que $1 \leq j$
 $\langle N j : 1 \leq j < \#xs + 1 : (x \blacktriangleright xs)!0 = (x \blacktriangleright xs)!j \rangle + f.xs$
 = { cambio de variable $j \leftarrow j + 1$ }
 $\langle N j : 1 \leq j + 1 < \#xs + 1 : (x \blacktriangleright xs)!0 = (x \blacktriangleright xs)!(j+1) \rangle + f.xs$
 = { aritmética en el rango, def. ! en el término }
 $\langle N j : 0 \leq j < \#xs : x = xs!j \rangle + f.xs$
 = { modularización con nueva función:
 m : Num -> [Num] -> Nat
 m.e.xs = $\langle N j : 0 \leq j < \#xs : e = xs!j \rangle$
 ¿Qué hace esta función? Dice cuántas veces aparece “e” en la lista “xs”.
 }
 m.x.xs + f.xs

Resultado final:

$f.[] \doteq 0$
 $f.(x \blacktriangleright xs) \doteq m.x.xs + f.xs$

$$\begin{aligned}
m.e.[] &\doteq 0 \\
m.e.(x \blacktriangleright xs) &\doteq (x = e \rightarrow 1 + m.e.xs \\
&\quad [] x \neq e \rightarrow m.e.xs \\
&\quad)
\end{aligned}$$

Testing: TODO

Rangos con análisis por casos

TODO

Resumen de técnicas de derivación

Supongamos que tenemos una especificación :

$$f.x = E$$

Queremos obtener un programa, esto es, una definición para la función f.

1. Si toda la expresión E es programable, la definición es directamente E:

$$f.x \doteq E$$

2. Si E tiene algunas partes programables y otras no, las partes no programables se **modularizan** con funciones nuevas. En E se reemplazan todas las partes no programables por llamadas a las funciones, para obtener E' que es completamente programable:

$$f.x \doteq E'$$

Después hay que obtener definiciones para todas las funciones modularizadas.

3. Si E es una expresión cuantificada, hará falta una recursión para calcularla, y por lo tanto hay que hacer **inducción** sobre alguno de los parámetros.

3.1. Elegir parámetro

3.2. Elegir esquema inductivo

3.3. Derivar: priorizar el caso inductivo, y dentro de éste, lograr aplicar la hipótesis inductiva. Si no se puede, es posible que haya que **generalizar** (ver punto siguiente). Si se puede, es posible que hayan otras partes extra que sean no programables, en cuyo caso hay que **modularizar**.

4. Si se intentó derivar por inducción y no se pudo, hay que **generalizar**. Se especifica una nueva función gf de manera parecida a f pero con un parámetro adicional "y":

$$gf.x.y = E'$$

donde E' es parecida a E pero aparece el nuevo parámetro "y". Además, gf generaliza a f o, lo que es lo mismo decir, f es caso particular de gf . El caso particular se da cuando "y" tiene un valor específico constante "e". Luego, se puede definir f directamente en una sola línea:

$$f.x \doteq gf.x.e$$

Después hay que obtener una definición recursiva para gf , cosa que debería ser posible si se hizo bien la generalización.

Temas complementarios

Técnicas de autocorrección

Señales de que hay algo mal en una especificación:

- Hay errores de tipo:
 - En una expresión cuantificada, el tipo del término no se corresponde con el tipo del operador.
Ejemplo: $\langle \sum i : 0 \leq i < \#xs : xs!i \bmod 2 = 0 \rangle$
(MAL: el término debe ser un número)

Señales de que hay algo mal en un programa:

- Variables cuantificadas que aparecen en el programa resultado.
Ejemplo:
 $gpsum.n.(x \blacktriangleright xs) \doteq n \geq 0 \wedge gpsum.(n * a).xs$ (MAL: "a" no existe!)
- Parámetros que faltan: variables que aparecen en el lado derecho de una definición pero que no están entre los parámetros.
Ejemplo:
 $m.(x \blacktriangleright xs) \doteq e = x \wedge m.xs$ (MAL: falta parámetro e)
- Parámetros que colisionan: dos parámetros o más comparten el mismo nombre.
Ejemplo:
 $m.x.(x \blacktriangleright xs) \doteq \dots$ (MAL: está dos veces x como parámetro)

Testing!!

Traducción a Haskell

Recursos:

- <https://wiki.cs.famaf.unc.edu.ar/lib/exe/fetch.php?media=introalg:guia-ghc.pdf>
- [Introduction to GHCi](#)

A partir de Haskell 2010, Haskell no soporta más *pattern matchings* de la forma “n+k” (“n+1”, “n+2”, etc.).²

```
fac.0 ÷ 1
fac.(n+1) ÷ (n+1) * fac.n
```

se debe traducir a:

```
fac 0 = 1
fac n = n * fac (n-1)
```

—

```
fib.0 ÷ 0
fib.1 ÷ 1
fib.(n+2) ÷ fib.n + fib.(n+1)
```

se traduce a:

```
fib 0 = 0
fib 1 = 1
fib n = fib (n-2) + fib (n-1)
```

Tabla de correspondencia de funciones y operadores:

En el teórico/práctico	En Haskell	¿Qué es?
÷	=	definición de función
=	==	comparación de igualdad
►	:	constructor de listas
!	!!	indexación
xs ↑ n	take n xs	tomar
xs ↓ n	drop n xs	tirar

² Se puede activar este pattern matching usando el pragma `NPlusKPatterns`. Ver:

https://en.wikipedia.org/wiki/Haskell#Haskell_2010

<https://stackoverflow.com/questions/3748592/what-are-nk-patterns-and-why-are-they-banned-from-haskell-2010>

Problemas Exponenciales

Ejemplo: Problemas con sublistas de listas (salteando elementos).

Dada una lista xs y un número n: ¿Existe una sublista cuya suma da n ?

Especificación: Usaremos una lista de ceros y unos para especificar todas las formas posibles de seleccionar elementos de la lista:

$$\text{sublistaN}.xs.n = \langle \exists \text{ as} : \text{binaria.as} \wedge \#as = \#xs : \langle \sum i : 0 \leq i < \#xs : xs!i * as!i \rangle = n \rangle$$

Donde: $\text{binaria}.xs = \langle \forall i : 0 \leq i < \#xs : xs!i = 0 \vee xs!i = 1 \rangle$

O mejor:

$\text{binaria}.[] = \text{True}$

$\text{binaria}.(x \blacktriangleright xs) = (x = 0 \vee x = 1) \wedge \text{binaria}.xs$

Ejemplo: $xs = [-2, 1, 10]$, $n = 8$

```
as ∈ { [0, 0, 0] , // suma 0
        [0, 0, 1] , // suma 10
        [0, 1, 0] , // suma 1
        [0, 1, 1] , // suma 11
        [1, 0, 0] , // suma -2
        [1, 0, 1] , // ESTE DA SUMA 8!
        [1, 1, 0] , // suma -1
        [1, 1, 1] } // suma 9
```

Resultado: True

Derivación: Por inducción en xs.

Caso base: $\text{sublistaN}.[].n = (n = 0)$

Paso inductivo:

```
sublistaN.(x ▶ xs).n
= { esp. }
⟨ ∃ as : binaria.as ∧ #as = #(x ▶ xs) : ⟨ ∑ i : 0 ≤ i < #(x ▶ xs) : (x ▶ xs)!i * as!i ⟩ = n ⟩
= { cambio var as → a ▶ as }
⟨ ∃ a, as : binaria.(a ▶ as) ∧ #(a ▶ as) = #(x ▶ xs) :
    ⟨ ∑ i : 0 ≤ i < #(x ▶ xs) : (x ▶ xs)!i * (a ▶ as)!i ⟩ = n ⟩
= { def. # y aritmética }
⟨ ∃ a, as : binaria.(a ▶ as) ∧ #as = #xs :
    ⟨ ∑ i : 0 ≤ i < #(x ▶ xs) : (x ▶ xs)!i * (a ▶ as)!i ⟩ = n ⟩
= { def. binaria }
⟨ ∃ a, as : (a = 0 ∨ a = 1) ∧ binaria.as ∧ #as = #xs : ... ⟩
= { distrib. y partición de rango }
```

$\langle \exists a, as : a = 0 \wedge \dots : \dots \rangle$
 $\vee \langle \exists a, as : a = 1 \wedge \dots : \dots \rangle$
 = { eliminación de a en ambos cuantificadores }
 $\langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{\langle \sum i : 0 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (0 \blacktriangleright as)!i \rangle} = n \rangle$
 $\vee \langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \langle \sum i : 0 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (1 \blacktriangleright as)!i \rangle = n \rangle$
 = { def #, partición de rango y rango unitario }
 $\langle \exists as : \text{binaria.as} \wedge \#as = \#xs :$
 $\quad \underline{(x \blacktriangleright xs)!0 * (0 \blacktriangleright as)!0} + \langle \sum i : 1 \leq i < \#xs+1 : (x \blacktriangleright xs)!i * (0 \blacktriangleright as)!i \rangle = n \rangle$
 $\vee \dots$
 = { def ! y neutro * }
 $\langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{\langle \sum i : 1 \leq i < \#xs+1 : (x \blacktriangleright xs)!i * (0 \blacktriangleright as)!i \rangle} = n \rangle$
 $\vee \dots$
 = { cambio var. $i \rightarrow i + 1$ }
 $\langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{\langle \sum i : 1 \leq i+1 < \#xs+1 : (x \blacktriangleright xs)!(i+1) * (0 \blacktriangleright as)!(i+1) \rangle} = n \rangle$
 $\vee \dots$
 = { aritmética y def. ! }
 $\langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \langle \sum i : 0 \leq i < \#xs : xs!i * as!i \rangle = n \rangle$
 $\vee \dots$
 = { H.I. y retomamos la otra parte }
 sublistaN.xs.n
 $\vee \langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{\langle \sum i : 0 \leq i < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (1 \blacktriangleright as)!i \rangle} = n \rangle$
 = { mismos pasos: partición, rango unitario, cambio de variable, etc. }
 sublistaN.xs.n
 $\vee \langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{x * 1 + \langle \sum i : 0 \leq i < \#xs : xs!i * as!i \rangle} = n \rangle$
 = { aritmética: paso restando x }
 sublistaN.xs.n $\vee \langle \exists as : \text{binaria.as} \wedge \#as = \#xs : \underline{\langle \sum i : 0 \leq i < \#xs : xs!i * as!i \rangle} = n - x \rangle$
 = { H.I. de nuevo! }
 sublistaN.xs.n $\vee$ sublistaN.xs.(n - x)

Resultado final:

sublistaN.[].n $\doteq$ n = 0
 sublistaN.(x $\blacktriangleright$ xs).n $\doteq$ sublistaN.xs.n $\vee$ sublistaN.xs.(n - x)

Testing: xs = [-2, 1, 10], n = 8

sublistaN.[-2, 1, 10].8
 $\rightarrow$ sublistaN.[1, 10].8 $\vee$ sublistaN.[1, 10].10
 $\rightarrow$ (sublistaN.[10].8 $\vee$ sublistaN.[10].7) $\vee$ sublistaN.[1, 10].10
 $\rightarrow$ (sublistaN.[10].8 $\vee$ sublistaN.[10].7) $\vee$ (sublistaN.[10].10 $\vee$ sublistaN.[10].9)
 $\rightarrow$ (sublistaN.[].8 $\vee$ sublistaN.[].(-2)) $\vee$ sublistaN.[10].7
 $\quad \vee$ sublistaN.[10].10 $\vee$ sublistaN.[10].9
 $\rightarrow \dots$
 $\rightarrow$ sublistaN.[].8 $\vee$
 sublistaN.[].(-2) $\vee$
 sublistaN.[].7 $\vee$
 sublistaN.[].(-3) $\vee$
 sublistaN.[].10 $\vee$
 sublistaN.[].0 $\vee$


```

sublistaN.[ ].9    V
sublistaN.[ ].(-1) V
→ 8 = 0    V    // [0, 0, 0] <-- EN "n = 0" , EL n significa "lo que le falta a la suma para dar
8"
-2 = 0    V    // [0, 0, 1] <-- acá es -2 porque sumó 10, se pasó en dos
7 = 0     V    // [0, 1, 0] <-- acá es 7 porque sumó 1, le faltan 7
-3 = 0    V    // [0, 1, 1] <-- y así...
10 = 0    V    // [1, 0, 0]
0 = 0     V    // [1, 0, 1] <-- acá sumó 8, le faltan 0, da True!!
9 = 0     V    // [1, 1, 0]
-1 = 0    V    // [1, 1, 1]
→ True

```

Técnica de Tuplas

Cómo pasar Fibonacci de exponencial (lenta) a lineal (rápida):

```

fib.0 ÷ 0
fib.1 ÷ 1
fib.(n+2) ÷ fib.n + fib.(n+1)

```

Problema: este programa es lentísimo, para calcular fib.n se requiere aprox. 2^n pasos de reducción (exponencial).

Ejemplo: fib.6 → fib.4 + fib.5
→ fib.4 + (fib.3 + fib.4)

(TODO: hacer dibujo del árbol de llamadas)

Observación: Hay muchas llamadas redundantes.

Especificación:

```

tfib : Nat -> (Nat, Nat)
tfib.n = (fib.n, fib.(n+1))

```

Fibonacci se puede definir en términos de tfib así:
fib.n ÷ fst.(tfib.n)

Derivación: Por inducción en n.

Caso base: tfib.0 ÷ (0, 1) (ejercicio)

Paso inductivo: La H.I. es "tfib.n = (fib.n, fib.(n+1))".

```

tfib.(n+1)
= { esp. tfib }

```

$$\begin{aligned}
& (\text{fib}.(n+1), \text{fib}.(n+2)) \\
= & \{ \text{def. fib} \} \\
& (\text{fib}.(n+1), \text{fib}.n + \text{fib}.(n+1)) \\
= & \{ \text{definición local} \} \\
& (b, a + b) \\
& \quad [[a = \text{fib}.n \\
& \quad \quad b = \text{fib}.(n+1)]] \\
= & \{ \text{armo tupla} \} \\
& (b, a + b) \\
& \quad [[(a, b) = (\text{fib}.n, \text{fib}.(n+1))]] \\
= & \{ \text{H.I.} \} \\
& (b, a + b) \\
& \quad [[(a, b) = \text{tfib}.n]]
\end{aligned}$$

Resultado final:

$$\text{fib}.n \doteq \text{fst}.(\text{tfib}.n)$$

$$\text{tfib}.0 \doteq (0, 1)$$

$$\begin{aligned}
\text{tfib}.(n+1) & \doteq (b, a + b) \\
& \quad [[(a, b) = \text{tfib}.n]]
\end{aligned}$$

Recursión final

Ver capítulos 15 y 20 del libro [Cálculo de Programas](#).

Ejercicios

[practico2.pdf](#)

Programación Imperativa

Introducción

Antes: Modelo computacional de la programación funcional

En programación funcional un programa es una expresión, y la ejecución de un programa es la reducción de esa expresión a una forma normal o canónica (a un valor).

Ejemplo: Definimos la función sum:

$$\text{sum}.[] \doteq 0$$

$\text{sum.}(x \blacktriangleright xs) \doteq x + \text{sum}.xs$

Un programa posible es la expresión:

$\text{sum.}([1,2] ++ [3,4])$

Una ejecución de este programa:

$\begin{aligned} \text{sum.}([1,2] ++ [3,4]) &\rightarrow \text{sum.}(1 \blacktriangleright [2] ++ [3,4]) \\ &\rightarrow \text{sum.}(1 \blacktriangleright [2] ++ [3,4]) \\ &\rightarrow 1 + \text{sum.}([2] ++ [3,4]) \\ &\dots \\ &\rightarrow 10 \end{aligned}$

Este modelo computacional es una abstracción que dista mucho del verdadero funcionamiento de una computadora (y su modelo computacional subyacente).

Ahora: Modelo computacional de la programación imperativa

En programación imperativa un programa es una “receta” o una “serie de pasos” y la ejecución de un programa parte de un **estado** inicial y lo va transformando hasta llegar a un **estado** final.

DEFINICIÓN: Estado: es una asignación de valores a un conjunto fijo y predeterminado de variables. En un programa, las variables que componen el estado (con su nombre y tipo) se definen al principio en lo que se llama la “declaración de constantes y variables”.

Programa de ejemplo:

$$\begin{array}{ll} \text{Var } x, y : \text{Int} & \leftarrow \text{declaración de variables} \\ \\ \ell_1 \quad x := 3 ; & \leftarrow \text{sencencias del programa (la “receta”)} \\ \ell_2 \quad y := x + 3 & \end{array}$$

Secuenciación: Usamos “;” para secuenciar una sentencia atrás de otra.

Asignación: Un tipo de sentencia que modifica determinados valores del estado.

Variables que componen el estado: “x” e “y” de tipo Int.

Ejemplo de estado inicial: $\sigma_0: x \mapsto 4, y \mapsto 8$

(observación: **siempre** ponemos valores en los estados, nunca expresiones)
(el estado me indica qué contiene la memoria de la computadora en un momento determinado)

Ejemplo de ejecución del programa a partir de este estado inicial:

línea	estado	observaciones
	$\sigma_0: x \mapsto 4, y \mapsto 8$	estado inicial
$\ell_1 \quad x := 3$	$\sigma_1: x \mapsto 3, y \mapsto 8$	
$\ell_2 \quad y := x + 3$	$\sigma_2: x \mapsto 3, y \mapsto 6$	estado final

Siempre el estado final de la ejecución de un programa depende tanto del programa como del estado inicial.

Un programa imperativo es un “**transformador de estados**”.

Sintaxis vs. Semántica

Siempre que hablemos de lenguajes de programación tendremos estos dos aspectos:

Sintaxis: ¿Cómo se escriben los programas? Un programa es un texto (una secuencia de letras). La sintaxis de un lenguaje me dice qué textos son programas válidos.

Semántica: ¿Qué significan los programas? ¿Qué hacen? Un programa se puede ejecutar, y esa ejecución tiene un efecto sobre un “mundo semántico”. En el caso de la programación imperativa, el mundo semántico es el estado. En el caso de la programación funcional, el mundo semántico es el de las expresiones.

El lenguaje de programación Imperativa

Asignación

Otro ejemplo:

```
y := (x + 3) * y // en una asignación puedo usar expresiones
```

Otro ejemplo:

```
n := n + 1 // en una asignación puedo usar
           // la misma variable que estoy asignando
```

Ejemplo: con estado inicial $n \rightarrow 7$, el estado final es $n \rightarrow 8$.

Otro ejemplo (asignación “múltiple”):

```
x , y := 3 , x + 3 // asigno dos variables al mismo tiempo
                // (x a “3”, y a “x + 3”)
```

Ejemplo: con estado inicial $x \mapsto 4$, $y \mapsto 8$, el estado final es $x \mapsto 3$, $y \mapsto 7$. Es 7 y no 6 ya que en la semántica de la asignación siempre usamos para todas las expresiones los valores en el estado anterior. La asignación es atómica e indivisible (no hay “estados intermedios”).

Observación: C no tiene asignación múltiple (hay otros lenguajes que sí, como Python).

Verificación en la consola de Python:

```
>>> x = 4          ← acá definimos el estado inicial
>>> y = 8          ← acá definimos el estado inicial
>>>
>>> x, y = 3, x + 3 ← este es el programa (asignación múltiple)
>>>
>>> x              ← muestro el resultado
3
>>> y
7
```

Condicional (if) y skip

Un if es una sentencia que me permite ejecutar distintas sub-sentencias dependiendo del cumplimiento de una o más condiciones.

Ejemplo:

Var $x : \text{Int}$

```
if    $x \geq 0 \rightarrow$           ← las condiciones se llaman “guardas”
    skip                ← skip es una sentencia que no hace nada (no modifica el estado)
 $\square$    $x < 0 \rightarrow$ 
     $x := -x$ 
fi
```

¿Qué hace este programa? Este programa calcula el módulo de x un estado final en el x vale el “valor absoluto” o “módulo” del valor de x en el estado inicial.

Observación importante: Nunca el resultado de un programa es un valor (como un número), si no un estado final.

Ejemplo: con estado inicial $x \rightarrow 33$, el estado final es $x \rightarrow 33$.

Ejemplo: con estado inicial $x \rightarrow -11$, el estado final es $x \rightarrow 11$.

Observación: El “if” es una sentencia compuesta ya que tiene sentencias adentro.

Repetición / ciclo (do)

```
 $\ell_1$  do  $x \neq 0 \rightarrow$           // el “do” tiene una guarda
 $\ell_2$     $x := x - 1$           // y tiene una sentencia adentro llamada “cuerpo”
```

ℓ_3 **od**

Semántica: “Mientras valga que $x \neq 0$, ejecutar $x := x - 1$.”

Ejemplo: con estado inicial $x \mapsto 2$

línea	estado/guardas	observaciones
	$\sigma_0: x \mapsto 2$	estado inicial
ℓ_1	$\sigma_0: x \mapsto 2, x \neq 0 \equiv \text{True}$	evalúo guarda
$\ell_2 \quad x := x - 1$	$\sigma_1: x \mapsto 1$	cuerpo del ciclo
ℓ_1	$\sigma_1: x \mapsto 1, x \neq 0 \equiv \text{True}$	evalúo guarda
$\ell_2 \quad x := x - 1$	$\sigma_2: x \mapsto 0$	cuerpo del ciclo
ℓ_1	$\sigma_2: x \mapsto 0, x \neq 0 \equiv \text{False}$	evalúo guarda
ℓ_3	$\sigma_2: x \mapsto 0$	estado final

Semántica más detallada:

1. a partir del estado inicial, chequeo si vale o no vale la guarda.
2. si la guarda vale,
 - a. ejecuto el cuerpo del ciclo.
 - b. a partir del estado que resulta de ejecutar el cuerpo del ciclo, chequeo nuevamente si vale o no la guarda, y **vuelvo al paso 2**.
3. si la guarda no vale, el ciclo termina y el estado queda como estaba.

Observaciones:

- puede pasar que la guarda no valga al principio de todo y el cuerpo del ciclo no se ejecute nunca (el estado no cambia).
- puede pasar que la guarda valga para siempre y entre en un ciclo infinito. un programa que hace esto es un programa que **“no termina”**.

Cosas que NO tiene el lenguaje de programación

Los lenguajes imperativos reales tienen muchos otros tipos de sentencias. Uno de los ejemplos más antiguos es la **sentencia “GO TO”** (o “GOTO”). Otros ejemplos más vigentes son las **sentencias break y continue** presentes en muchos lenguajes incluyendo C y Python.

Este tipo de sentencias no nos interesan y no serán parte de nuestro lenguaje de programación teórico por dos grandes razones:

1. No son necesarias, ya que es bien sabido que:

Todo problema que puede ser resuelto por programación, puede ser resuelto usando solamente asignaciones ($:=$), condicionales (if) y ciclos (do)

(Teorema de Böhm–Jacopini)

2. Agregan complejidad lógica y dificultad para entender y razonar sobre los programas. Sobran publicaciones respaldando esto, siendo la siguiente carta de Dijkstra una de las más famosas:

Dijkstra, Edsger W. (March 1968). *"Letters to the editor: Go to statement considered harmful"* (PDF). Communications of the ACM. 11 (3): 147–148.

doi:10.1145/362929.362947. S2CID 17469809.

<https://www.cs.utexas.edu/~EWD/ewd02xx/EWD215.PDF>

Ejemplo: contar múltiplos de 6

Vamos a hacer un programa que usa todo lo que acabamos de ver.

Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.

Idea del algoritmo: Con un ciclo, recorrer los números desde 0 hasta N (o al revés, desde N hasta 0). Para cada valor, fijarme si es múltiplo de 6 o no. Si sí lo es, sumamos 1 al resultado.

Programa final:

```
Const N : Int ;
Var numero_actual, resultado : Int ;

numero_actual , resultado := 0 , 0 ; // esta asignación se llama "inicialización"
do numero_actual ≤ N →
    if numero_actual mod 6 = 0 →
        resultado := resultado + 1 ;           // FUNDAMENTAL: ASIGNACIÓN ES :=
                                                // USAR ";" PARA SECUENCIAR
        numero_actual := numero_actual + 1
    □ numero_actual mod 6 ≠ 0 →
        numero_actual := numero_actual + 1
    fi
od
```

Importante: Indentación!! Organizar y tabular el código para que pueda leerse correctamente.

¿Es correcto este programa? ¿Hace lo que esperamos?

Si $N = 12$, **debería dar** un estado final con resultado=3 (el 0, el 6 y el 12).

Sí es correcto.

Ejercicio: verificar haciendo la ejecución con un ejemplo concreto de estado inicial (valores para **N**, **numero_actual** y **resultado**).

Otro programa posible:

```
Const N : Int ;
Var numero_actual, resultado : Int ;

numero_actual , resultado := 0 , 0 ; // esta asignación se llama "inicialización"
do numero_actual ≤ N →
    if numero_actual mod 6 = 0 →
        resultado := resultado + 1
    □ numero_actual mod 6 ≠ 0 →
        skip
    fi ; // <- secuenciación entre el if y la asignación que sigue
    numero_actual := numero_actual + 1
od
```

Otro programa posible (recorriendo hacia atrás):

```
Const N : Int ;
Var numero_actual, resultado : Int ;

numero_actual , resultado := N , 0 ;
do numero_actual ≥ 0 →
    if numero_actual mod 6 = 0 →
        resultado := resultado + 1
    □ numero_actual mod 6 ≠ 0 →
        skip
    fi ; // <- secuenciación entre el if y la asignación que sigue
    numero_actual := numero_actual - 1
od
```

Otro programa posible (usando conocimientos matemáticos básicos):

```
Const N : Int ;
Var numero_actual, resultado : Int ;

numero_actual , resultado := 0 , 0 ;
do numero_actual ≤ N →
    resultado := resultado + 1 ;
    numero_actual := numero_actual + 6
od
```

Otro programa (usando conocimientos matemáticos básicos):

Sale con una sola asignación:

```
Const N : Int ;
```



```

Var resultado : Int ;

resultado := N div 6 + 1

```

Otro ejemplo: factorial

Dado un número $N > 0$ queremos calcular su factorial:

$$\text{resultado} = 1 * 2 * 3 * \dots * (N-1) * N$$

Es evidente que no es posible hacer todas las multiplicaciones en una sola asignación ya que no existe una expresión válida en el lenguaje para hacer eso.

Por lo tanto es necesario usar un ciclo **do para ir incorporando de a uno cada número que se desea multiplicar**. El resultado comenzará valiendo 1 y luego en cada iteración del ciclo será modificado con una nueva multiplicación:

- 1er paso: resultado = 1
- 2do paso: resultado = 1 * 2
- 3er paso: resultado = 1 * 2 * 3
- ...
- N-1-ésimo paso: resultado = 1 * 2 * 3 * ... * (N-1)
- N-ésimo paso: resultado = 1 * 2 * 3 * ... * (N-1) * N

Para ello necesitaremos una nueva variable que recorra todos los números que hay que multiplicar. La llamaremos `numero_actual`, y tomará todos los valores a multiplicar desde 2 hasta N.

El programa va a comenzar con una asignación múltiple que da los valores iniciales para las variables `resultado` y `numero_actual`:

```

resultado, numero_actual := 1, 2

```

Luego sigue el ciclo. En cada paso del ciclo se debe multiplicar un nuevo término al `resultado`, mientras que el número actual debe avanzar al siguiente:

```

do numero_actual ≤ N →
    resultado, numero_actual := resultado * numero_actual, numero_actual + 1
od

```

Resultado final:

```

Const N : Int ;
Var numero_actual, resultado : Int ;

resultado, numero_actual := 1, 1 ;    // inicialización

```

```

do numero_actual ≤ N →
    resultado, numero_actual := resultado * numero_actual, numero_actual + 1
od

```

Arreglos

Es el único tipo de dato que me permite tener una colección de valores de algún tipo (en el lenguaje imperativo del teórico/práctico). Como de costumbre, la semántica de la programación imperativa se acerca más a la verdadera arquitectura de las computadoras, y los arreglos son el tipo de colecciones más cercano al verdadero funcionamiento de la memoria de una computadora.

El **arreglo** define una colección de valores **ordenada** y de **tamaño fijo**.

Se puede ver como una lista de programación funcional pero de **largo fijo**. No existe el concepto de agregar o quitar elementos en un arreglo (tampoco de concatenar arreglos). También se puede ver como una tupla pero con todas las componentes son del mismo tipo.

En **Algoritmos I** usaremos los arreglos sólo para leer su información y no para modificarlos, escribirlos o calcular resultados contenidos en ellos.

Declaración (tipo)

Sintaxis: Array[N, M) of <tipoX>

Semántica: Tengo un arreglo de (M - N) elementos de tipo <tipoX>, cuyos índices son N, N+1, N+2, ..., M-1.

Observaciones:

- <tipoX> puede ser cualquier tipo válido, incluso otro arreglo (e.g. para definir matrices o arreglos multidimensionales).
- los arreglos son siempre “cerrado/abierto” (o sea “[/ “ ”)).
- por lo general usaremos letras mayúsculas para variables que contienen arreglos.
- por lo general usaremos N = 0.

Ejemplos:

```

Const A : Array[0 , 4) of Int ;
    // este arreglo tiene elementos A.0, A.1, A.2 y A.3

Var MiArreglo : Array[11, 15) of Bool;

Const N, M : Nat ;
Var Matriz : Array[0, N) of Array[0, M) of Int;
    // tenemos una matriz de N x M números enteros.

```

Consulta / acceso (expresión)

Si yo tengo un arreglo, me interesa acceder a los valores para usarlos en una expresión (por ejemplo para usar la expresión en una asignación o en una guarda de un if o un do).

Sintaxis: Si mi arreglo se llama A , y tengo una expresión E que es tipo entero, la sintaxis es:

A.E

Semántica:

1. Calcular el valor entero asociado a la expresión E. Sea n este valor.
2. "A.E" me devuelve el valor alojado en la n-ésima posición del arreglo A.
(este valor devuelto es del tipo <tipoX>)

Observaciones:

- si E resulta en un valor n que no es una posición válida del arreglo, el acceso da error (el programa se rompe).

Ejemplo: Si tengo:

```
Const A : Array[0 , 4) of Int ;  
Var x : Int ;
```

Puedo hacer:

```
x := x + A.3 * 2
```

Otro ejemplo: Si tengo una matriz de 3x3:

```
Var Matriz : Array[0, 3) of Array[0, 3) of Int;  
Var x : Int ;
```

Puedo hacer la suma de la diagonal así:

```
x := (A.0).0 + (A.1).1 + (A.2).2
```

Asignación para arreglos (sentencia)

Con la sentencia de asignación para arreglos puedo modificar un valor en una posición.

Sintaxis:

A.E := F

donde E es una expresión de tipo Nat, y F es una expresión de tipo <tipoX> a donde <tipoX> es el tipo de los elementos del arreglo.

Semántica:

1. Calcular los valores asociados a E y a F.

2. Modificar el estado, cambiando el valor asociado a la posición E por el resultado de F.

Observaciones:

- esta asignación es una asignación distinta a la normal y no se pueden mezclar.
- **NO LO VAMOS A USAR EN ALGORITMOS I.**

Ejemplo: Sumar elementos de un arreglo

Supongamos que tengo $N > 0$ y un arreglo de N elementos de tipo entero. Quiero calcular la suma de todos los elementos.

Idea del programa: Con un ciclo, usamos una variable para recorrer las posiciones del arreglo, desde la primera hasta la última, y en otra variable vamos calculando la suma.

```
Const N : Int, A : Array[0, N) of Int;  
Var pos, res : Int;
```

```
res, pos := 0, 0 ;  
do pos < N →  
    res := res + A.pos ;  
    pos := pos + 1  
od
```

Equivalentemente:

```
Const N : Int, A : Array[0, N) of Int; <- declaración del arreglo  
Var pos, res : Int;
```

```
res, pos := 0, 0 ;  
do pos < N →  
    res, pos := res + A.pos, pos + 1  
    <- consulto el valor en una posición del  
    arreglo  
od
```

Uno que no anda:

```
Const N : Int, A : Array[0, N) of Int;  
Var pos, res : Int;  
res, pos := 0, 0 ;  
do pos < N →  
    pos := pos + 1  
    res := res + A.pos ;  
od
```

Este no anda porque se saltea el primer elemento y además da error porque intenta acceder a la posición "N" que no es una posición válida.

Ejercicio: Verificar los tres programas con un ejemplo de un arreglo en concreto.

Testing / Ejecución

Lenguaje imperativo completo

Sintaxis de un programa. Dos secciones principales:

1. Declaración de constantes y variables
2. Sentencia del programa

Semántica de un programa:

- Estado: variables y constantes definidas por las declaraciones.
- Ejecución: Dado un estado inicial, ejecutar la sentencia del programa, modificando el estado hasta llegar a un estado final.

Diferencias con funcional: En imperativo (del teórico) no hay funciones y **no hay un valor resultado**. El **resultado es el estado final entero**.

Diferencias con C: En imperativo (del teórico) no hay funciones ni procedimientos. Sólo estamos tomando de C lo que se llama programación “in the small” (en pequeña escala): Los algoritmos pequeños, sin la superestructura de los programas que podemos definir en C. Tampoco vamos a hacer input/output.

Declaración de constantes y variables

Especificador: Var o Const. (para avisar si declaro variable o constante)

Nombres (identificadores): Palabras con letras y números (pero empezando con una letra).

Tipos: Nat, Int, Real, Bool, Char, arreglos. **No hay listas**.

Sintaxis:

```
Var/Const  nombre1, nombre2, ... , nombren  : Tipo;
...
Var/Const  nombre1, nombre2, ... , nombren  : Tipo;
```

Ejemplo:

```
Var x, y : Int;
Var resultado : Bool;
```

Otro ejemplo:

```
Const divisor, dividendo : Int;
Var cociente, resto : Int;
```

Observación: El valor de las constantes no estará determinado en los programas (no lo escribiremos como parte de la sintaxis), si no que podrá ser cualquier valor (en principio). Luego veremos cómo se podrá restringir los valores posibles. (por ejemplo: me interesa que el divisor no sea cero).

Otro ejemplo (con variables de varios tipos en la misma línea):

```
Var x : Int, b : Bool;
```

Sentencias

Dos tipos:

- Elementales: sentencias básicas que no necesitan de otras sentencias.
- Compuestas: sentencias que se arman a partir de otras sentencias (o sea, sentencias que tienen adentro otras sentencias).

Skip

Es una sentencia elemental que no hace nada.

Sintaxis:

skip

Semántica: No modifica el estado.

Asignación (:=)

Sentencia elemental que se usa para asignar valores a variables.

Sintaxis:

v1, v2, ..., vn := E1, E2, ..., En

donde $v_1, v_2, \dots, v_n$ son variables declaradas del programa, y $E_1, E_2, \dots, E_n$ son expresiones válidas en lenguaje y del tipo que se corresponde con las variables.

Semántica:

1. Se calculan valores-resultado para todas las expresiones $E_1, E_2, \dots, E_n$. (O sea, se hace usando el estado actual, no el nuevo estado.)
2. Se modifica el estado, asignando a cada variable v_i el resultado de calcular la expresión E_i .

Expresiones

Las expresiones válidas en programación imperativa son parecidas a las expresiones en funcional, pero con algunas diferencias:

- valores constantes
- variables y constantes declaradas
- operaciones básicas (+, -, *, /, div, mod, max, min, $\wedge$, $\vee$, $\neg$, =, $\neq$, $\leq$, $<$, $>$, etc.)
- accesos a elementos de arreglos
- **NO: llamadas a funciones, cuantificadores ni cosas que esconden cuantificadores (factorial, exponenciación, etc.)**
- **NO: análisis por casos, definiciones locales**
- **NO: expresiones de tipo lista**

Secuenciación (;)

Primera sentencia compuesta que vemos. La secuenciación me permite ejecutar una sentencia y después otra.

Sintaxis:

S1 ; S2

donde S1 y S2 son dos sentencias.

Semántica:

1. Ejecutar S1 a partir del estado inicial.
2. Ejecutar S2 a partir del estado que resulta del paso 1. (el estado queda como lo deja la ejecución de S2).

Observaciones:

- La secuenciación es **asociativa**:

Es lo mismo (semánticamente)

(S1 ; S2) ; S3

que

S1 ; (S2 ; S3)

Luego, podremos escribir directamente:

S1 ; S2 ; S3

- Un programa es en realidad siempre **una sola sentencia**. Cuando hablamos de varias sentencias en realidad éstas forman una sola sentencia a través de la secuenciación.

Condicional (if)

Sentencia compuesta. El condicional me permite ejecutar diferentes sentencias dependiendo de una condición booleana.

Sintaxis:

```
if B1 → S1  
□ B2 → S2  
...  
□ Bn → Sn  
fi
```

donde B1, B2, ..., Bn (llamadas **guardas**) son expresiones de tipo booleano, y S1, S2, ..., Sn son sentencias.

Semántica:

1. Se evalúan todas las guardas B1, ..., Bn usando el estado actual. (se obtienen valores booleanos).
2. Se elige alguna de las guardas que da True. Si ninguna da True, el programa **termina con error** ("se rompe"). Supongamos que elegimos la k-ésima guarda (Bk = True).
3. Ejecutar esa rama del if: Sk. (el estado final es el que resulta de esta ejecución).

Observación: **No determinismo:** Puede haber varias guardas que den True. En ese caso, la semántica dicta que se puede ejecutar cualquiera de esas (pero siempre una y sólo una). La semántica no me dice cuál va a ser (no necesariamente es la primera). El programador no elige.

Repetición, ciclo o bucle (do)

Sentencia compuesta. Me permite repetir la ejecución de una sentencia mientras se cumpla una condición.

Sintaxis:

```
do B → S od
```

donde B es una expresión booleana (**guarda**), y S es una sentencia (**cuerpo del ciclo**).

Semántica:

1. Se evalúa la guarda B en el estado actual.
2. Si da False, termina la ejecución.
Si da True, modifica el estado ejecutando S, y luego vuelve al punto 1.

Anotaciones de programa

Recordemos este programa: "Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6."

Solución:

```
Const N : Int ;
Var numero_actual, resultado : Int ;

l1 numero_actual, resultado := 0, 0 ;
l2 do numero_actual ≤ N →
    < ~ prestamos atención a los estados posibles en este punto
l3   if numero_actual mod 6 = 0 →
l4       resultado := resultado + 1 ;
l5   □ numero_actual mod 6 ≠ 0 →
l6       skip
l7   fi ;
l8   numero_actual := numero_actual + 1
l9 od
```

Supongamos el siguiente estado inicial:

$\sigma_0: N \mapsto 7, \text{numero_actual} \mapsto -11, \text{resultado} \mapsto 77$

En la línea 2, voy a tener una secuencia de estados:

$\sigma_1: N \mapsto 7, \text{numero_actual} \mapsto 0, \text{resultado} \mapsto 0$
 $\sigma_2: N \mapsto 7, \text{numero_actual} \mapsto 1, \text{resultado} \mapsto 1$
 $\sigma_3: N \mapsto 7, \text{numero_actual} \mapsto 2, \text{resultado} \mapsto 1$
 $\sigma_4: N \mapsto 7, \text{numero_actual} \mapsto 3, \text{resultado} \mapsto 1$
 $\sigma_5: N \mapsto 7, \text{numero_actual} \mapsto 4, \text{resultado} \mapsto 1$
 $\sigma_6: N \mapsto 7, \text{numero_actual} \mapsto 5, \text{resultado} \mapsto 1$
 $\sigma_7: N \mapsto 7, \text{numero_actual} \mapsto 6, \text{resultado} \mapsto 1$
// todavia no se hizo el chequeo para el 6
 $\sigma_8: N \mapsto 7, \text{numero_actual} \mapsto 7, \text{resultado} \mapsto 2$
 $\sigma_9: N \mapsto 7, \text{numero_actual} \mapsto 8, \text{resultado} \mapsto 2$
// aca la guarda se hace False, el programa termina
...

Para otros estados iniciales posibles, voy a tener otras secuencias posibles de estados en ese punto (la línea 2).

En general, en toda línea de mi programa voy a tener un conjunto de estados posibles en ese punto. Para describir propiedades que cumplen los estados posibles en un punto de mi programa puedo usar **predicados**.

Ejemplo: En la línea 2 puedo escribir los siguientes predicados:

- True
- $\text{numero_actual} \geq 0$
- $\text{numero_actual} \leq N+1$
- $\text{numero_actual} \leq N+10000$
- $\text{resultado} \geq 0$

Incluso podemos poner conjunciones de los anteriores:

- $0 \leq \text{numero_actual} \leq N+1$
- $0 \leq \text{numero_actual} \leq N+1 \wedge 0 \leq \text{resultado} \leq N+1$

Incluso podemos usar expresiones cuantificadas para decir cosas más interesantes:

- $\text{resultado} = \langle N \ i : 0 \leq i \leq N : i \bmod 6 = 0 \rangle$

¿es cierto esto en la línea 2? ¿vale en todo estado posible en la línea 2?

(según este predicado, resultado debe valer 5 que sería en realidad el resultado final)

NO VALE SIEMPRE en la línea 2.

Lo corregimos un poco:

- $\text{resultado} = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle$

(estoy diciendo que en la línea 2 el estado es tal que en resultado he calculado un “**resultado parcial**”, que en este caso sería contar los múltiplos de 6 pero sólo hasta $\text{numero_actual}-1$).

Reescribamos el programa pero con algunas anotaciones:

```
Const N : Int ;
Var numero_actual, resultado : Int ;
    { N > 0 }
// PRECONDICIÓN: esta anotación de programa me habla del estado inicial
l1 numero_actual, resultado := 0, 0 ;
    { numero_actual = 0 ∧ resultado = 0 }
l2 do numero_actual ≤ N →
    { resultado = ⟨ N i : 0 ≤ i < numero_actual : i mod 6 = 0 ⟩ }
l3   if numero_actual mod 6 = 0 →
l4     resultado := resultado + 1 ;
l5   [] numero_actual mod 6 ≠ 0 →
l6     skip
l7   fi ;
l8   numero_actual := numero_actual + 1
l9 od
    { numero_actual = N + 1 ∧ resultado = ⟨ N i : 0 ≤ i ≤ N : i mod 6 = 0 ⟩ }
// POSTCONDICIÓN: esta anotación de programa me habla del estado final
```

DEFINICIÓN: Una **anotación de programa** es un predicado que se puede introducir en un punto de un programa. Este predicado afirma algo que es satisfecho por todo estado posible en ese punto del programa.

Observaciones:

- Las anotaciones de programa **no son** parte de los programas.
- Las vamos a insertar en medio de los programas usando llaves “{”, “}”.
- Todo estado posible en ese punto del programa debe satisfacer el predicado, pero también el predicado puede admitir más estados que nunca suceden en ese punto.
- **Corolario:** El predicado “True” es siempre una anotación de programa correcta. (aunque es poco informativa)

Precondición y postcondición

DEFINICIÓN: Llamamos **precondición** a la anotación de programa que usamos para describir el estado inicial, o sea la que ubicamos en el punto inicial del programa.

DEFINICIÓN: Llamamos **postcondición** a la anotación de programa que usamos para describir el estado final, o sea la que ubicamos en el punto final del programa.

Observaciones:

- La precondición me dice cosas que puedo asumir acerca de la información de entrada de mi problema.
- La postcondición me sirve para expresar qué debe calcular el programa, o sea, qué problema está siendo resuelto.
- La precondición y las postcondición me sirven para **especificar**: Puedo traducir un enunciado informal en una especificación.

Especificaciones

DEFINICIÓN: En programación imperativa, una **especificación** es una descripción formal del problema que se quiere resolver a través de una precondición y una postcondición. Debe incluirse también la declaración de las constantes y variables que definen el problema.

Notación para especificar:

Const ...

Var ...

{ P } ← precondición (predicado explícito)

S ← programa incógnita (letra S)

{ Q } ← postcondición (predicado explícito)

Ejemplos

Ejemplo 1: Para el problema ya visto: “Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.”

Una especificación posible es:

```
Const N : Int
Var resultado : Int

{ N > 0 }
S
{ resultado = < N i : 0 ≤ i ≤ N : i mod 6 = 0 > }
```

(observación: no se declara numero_actual, porque no es parte de la especificación si no de la solución.)

Ejemplo 2: Sumar los elementos de un arreglo: “Dados $N > 0$ y un arreglo de N elementos de tipo entero, calcular la suma de los elementos.”

Especificación:

```
Const N : Int, A : array [0, N) of Int
Var resultado : Int
{ P:  $N > 0$  }
S
{ Q: resultado =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
```

Ejemplo 3 (práctico 5, ejercicio 1): Especificar! Ejercicio...

Ejemplo 4: Dado $N \geq 0$, calcular el factorial de N .

Especificiquemos:

```
Const N : Int
Var factorial : Int

{ P:  $N \geq 0$  }
S
{ Q: factorial =  $\langle \prod i : 1 \leq i \leq N : i \rangle$  }
```

Equivalentemente, ya que podemos usar cualquier cosa de la matemática:

```
Const N : Int
Var factorial : Int

{ P:  $N \geq 0$  }
S
{ Q: factorial =  $N!$  }
```

(no hace falta usar expresiones cuantificadas)

Ejemplo 5: “Escriba un programa que sume, por un lado, los valores pares y, por otro, los valores múltiplos de 3 de los números entre N y M (inclusives).”

Especificamos:

```
Const N, M : Int
Var suma_mult2, suma_mult3 : Int

{ P:  $N \leq M$  }
S
{ Q: suma_mult2 =  $\langle \sum i : N \leq i \leq M \wedge i \bmod 2 = 0 : i \rangle \wedge$   
      suma_mult3 =  $\langle \sum i : N \leq i \leq M \wedge i \bmod 3 = 0 : i \rangle$  }
```

Ejemplo 6: Algoritmo de la división: “Dados dos números x e y , con $x \geq 0$ e $y > 0$, calcular cociente y resto de la división entera de x por y .”

Especificamos:

```
Const x, y : Int
Var q, r : Int

{ P:  $x \geq 0 \wedge y > 0$  }
S
{ Q:  $q = x \text{ div } y \wedge r = x \text{ mod } y$  }
```

Equivalentemente, usando el teorema de la división entera:

```
Const x, y : Int
Var q, r : Int

{ P:  $x \geq 0 \wedge y > 0$  }
S
{ Q:  $x = q * y + r \wedge 0 \leq r < y$  }
```

Ternas de Hoare

Introducción

¿Para qué sirve una especificación? Sirve para describir lo que el programa **tiene** que hacer si quiere resolver el problema especificado.

```
Const ...
Var ...
{ P }    ← precondition (predicado explícito)
S        ← programa incógnita (letra S)
{ Q }    ← postcondition (predicado explícito)
```

Para que el programa S resuelva el problema especificado, debe suceder lo siguiente: Que **cada vez que se ejecute el programa comenzando de un estado inicial que satisface la precondition P , la ejecución debe terminar en un estado final que satisface la postcondition Q .**

Ejemplo: Supongamos que tenemos dos variables dadas x e y de tipo entero, quiero intercambiar sus valores.

Especificación:

```
Var x, y : Int
```

```

{ P: x = A  ∧  y = B }  // fijo los valores de x e y en el estado inicial.
                        // A y B son variables de especificación

S

                        // en el programa, A y B no existen

{ Q: x = B  ∧  y = A }

```

Variables de especificación

DEFINICIÓN: son variables que usamos sólo en las anotaciones de los programas para fijar valores en determinados puntos. No son parte del programa, no se declaran (ni como Const ni como Var) y no se usan en el programa.

¿Qué sentencia **S** soluciona el problema especificado?

```

x, y := A, B // NO: justamente dijimos no se pueden usar A y B.

x, y := y, x   // ANDA!

```

También se soluciona introduciendo una tercer variable aux : Int:

```

aux := x ;
x := y ;
y := aux   // ANDA!

```

Ambas sentencias funcionan, esto quiere decir que valen las especificaciones si yo reemplazo S por las sentencias correctas. Haciendo el reemplazo me quedan:

```

Var x, y : Int ;

{ P: x = A  ∧  y = B }
  x, y := y , x
{ Q: x = B  ∧  y = A }

Var x, y, aux : Int ;

{ P: x = A  ∧  y = B }
  aux := x ;
  x := y ;
  y := aux
{ Q: x = B  ∧  y = A }

```

En general, si tenemos dos predicados P y Q **dados**, y tenemos una sentencia S **dada**, podemos preguntarnos si vale o no vale lo siguiente:

***Cada vez que empiezo de un estado inicial que cumple P,
la ejecución de S termina en un estado final que satisface Q.***

Esta pregunta tiene respuesta SI o NO (True o False), o sea es un **predicado** que puede valer o no.

A este predicado lo llamaremos **Terna de Hoare** y lo escribiremos de la siguiente manera:

$$\begin{array}{c} \{ P \} \\ S \\ \{ Q \} \end{array}$$

o todo junto:

$$\{ P \} S \{ Q \}$$

(se llama “Terna” porque tiene tres componentes: la pre, la sentencia y la post)

Definición

DEFINICIÓN: Dados dos predicados P y Q y una sentencia S, decimos

$$\{ P \} S \{ Q \}$$

cuando sucede que

*“para todo estado inicial que satisface P,
la ejecución de S termina en un estado final que satisface Q”*

Llamamos **Terna de Hoare** a este predicado “{P} S {Q}”.

(se llama “Terna” porque tiene tres componentes: la pre, la sentencia y la post)

Observaciones

Observación 1: Siempre vale la terna

$$\{ P \} S \{ \text{True} \}$$

, siempre y cuando S termine (no dé error ni pueda entrar en ciclo infinito).

Observación 2:

$$\{ \text{False} \} S \{ Q \}$$

¿Vale? **SI.** Afirma que siempre que empiece en un estado que satisface **False** termine en un estado que satisface Q. Si no valiera, debería ser capaz de dar un contraejemplo: un **estado inicial que satisface False** tal que el estado final no satisface Q. No existe tal cosa, así que la terna vale. (es una especie de rango vacío)

Observación 3:

$$\{ P \} S \{ False \}$$

¿Vale? Depende... si P es False, **vale**. Si P no es False, debe haber algún estado inicial posible y por lo tanto la terna **no vale**.

Observación 4:

$$\{ True \} S \{ Q \}$$

¿Vale? Depende, debe suceder que para cualquier estado inicial, ejecutar S me deja en un estado final que satisface Q.

Ejemplos

Ejemplo 1: “Swap” (intercambio de valores):

$$\begin{array}{l} \{ x = A \wedge y = B \} \\ x, y := y, x \\ \{ x = B \wedge y = A \} \end{array}$$

(A y B son variables de especificación)

¿Vale esta terna?

Si empezamos en un estado que satisface P, ¿al ejecutar S terminamos en un estado que satisface Q? ¡Sí! Siempre.

Ejemplo 2: ¿Vale esta terna?

$$\begin{array}{l} \{ P: x = A \wedge y = B \} \\ \text{aux} := x ; \\ x := y ; \\ y := \text{aux} \\ \{ Q: x = B \wedge y = A \} \end{array}$$

También.

Ejemplo 3: ¿Vale esta terna?

$$\begin{array}{l} \{ P: x = A \wedge y = B \} \\ x := y ; \\ y := x \\ \{ Q: x = B \wedge y = A \} \end{array}$$

No vale. Luego, debe haber al menos un contraejemplo: Algún estado inicial que satisface P, tal que luego de ejecutar S el estado final no satisface Q.

Contraejemplo: $\sigma_0: x \rightarrow 17, y \rightarrow 20$ (vale P con A = 17, B = 20)

$\sigma_1: x \rightarrow 20, y \rightarrow 20$ (no vale Q con A = 17, B = 20)

Usaremos las Ternas de Hoare para afirmar cosas que satisfacen los programas / sentencias.

Ejemplo 4 (práctico 5, ejercicio 4a):

$$\{ x > 0 \} \quad x := x * x \quad \{ \text{True} \}$$

¿Vale? Sí vale, ya que siempre terminamos en un estado final que satisface **True**.

Ejemplo 5 (práctico 5, ejercicio 4b):

$$\{ x \neq 100 \} \quad x := x * x \quad \{ x \geq 0 \}$$

¿Vale? Sí, ya que x es el cuadrado de un número, que siempre es ≥ 0 .

$$\{ x = 100 \} \quad x := x * x \quad \{ x \geq 0 \}$$

¿Vale? También, en realidad no importa la precondition, la terna siempre vale.

Ejemplo 6: ¿Vale esta terna?

```
Var  r, x, y : Int

{ True }
if x ≥ y →
    r := x
[] x ≤ y →
    r := y
fi
{ r = x max y }
```

Observación: No determinismo: puede suceder que ambas guardas sean verdaderas pero esto no representa ningún problema (reparar la semántica del if). Si ambas son verdaderas, se ejecuta sólo una de las dos (no sabemos cual).

Ejemplo 7: Cálculo del máximo mal especificado:

```
Var r, x, y : Int

{ P: True }
S
{ Q: r = x max y }
```

¿Cuál es el programa más simple que podemos dar que satisface esta especificación?

¿Vale la siguiente terna?:

```
Var r, x, y : Int

{ True }
r, x, y := 0, 0, 0
```

$\{ r = x \max y \}$

¡Vale!

¿Es correcta esta especificación para el cálculo del máximo entre x e y? No, porque admite programas que no reflejan el problema que yo quiero solucionar.

La especificación funciona como **contrato** entre la persona que quiere resolver el problema y la persona que lo va a solucionar.

Especificación correcta para max:

Var $r, x, y : \text{Int}$

$\{ x = A \wedge y = B \}$
S

$\{ r = A \max B \}$

(usando variables de especificación A y B)

Derivación vs demostración



Derivación: Dada una especificación (con su correspondiente precondition P y postcondition Q), derivar es obtener un programa S (o sea, una sentencia S) tal que vale la Terna de Hoare:

$\{ P \} S \{ Q \}$

Demostración: Dada una especificación (con pre P y post Q), y un programa S, demostrar que el programa satisface la especificación, o equivalentemente, demostrar que vale la Terna de Hoare:

$\{ P \} S \{ Q \}$

En lo que resta de la materia veremos cómo hacer esto.

Precondición más débil

¿Para cuáles precondiciones vale la siguiente terna?

```
{ P }  
  x := x * x  
{ x ≥ 9 }
```

<Propongamos predicados P para los que vale esta terna:

- **True?** No vale: Si el estado inicial es $x \rightarrow 1$, se rompe la terna.
- $x = 1000$? Sí. Es un caso muy particular.
- $x \geq 20$? Sí, y abarca más casos que el predicado anterior.
- $x \geq 3$? Sí, y abarca más casos todavía.
- $x = -10$? Sí, otro caso **particular**
- **False?** Sí vale (no hay contraejemplo para refutarlo). Es el caso más particular posible (conjunto vacío de estados iniciales posibles).
- $x \leq -3$? Sí vale, y abarca todos los x negativos para los que vale.
- **El más general de todos (juntando dos anteriores):**
 - $x \leq -3 \vee x \geq 3$
 - equivalentemente, $|x| \geq 3$
 - equivalentemente, $x^2 \geq 9$

Siempre que tenemos un programa S y una postcondición Q, nos interesa preguntarnos cuál es la precondición más general (o abarcativa) P tal que vale la Terna de Hoare:

$$\{ P \} S \{ Q \}$$

Llamamos a P **precondición más débil (weakest precondition)** de S y Q, y la denotamos:

$$wp.S.Q \quad (\text{esto es } P)$$

Fortaleza y debilidad de predicados

Dados dos predicados P y Q, decimos que P es **más débil** que Q, o que Q es **más fuerte** que P, si y sólo si:

$$Q \Rightarrow P$$

(ejemplo: $P \equiv x \geq 3$,

$Q \equiv x = 10$.

Q es mas fuerte que P, P es más débil que Q, ya que claramente
 $x = 10 \Rightarrow x \geq 3$)

Intuiciones: Un predicado es más débil si exige menos cosas (o sea, admite más posibilidades, o un conjunto más grande de estados posibles), y es más fuerte y exige más cosas (un conjunto más chico de estados posibles).

Dibujo de el conjunto de estados que satisfacen un predicado. Si Q es mas fuerte que P, o sea $Q \Rightarrow P$, tengo lo siguiente:



Observaciones:

- El predicado más débil de todos es True (y define el conjunto universal de estados posibles).
- El predicado más fuerte de todos es False (y define el conjunto vacío de estados posibles).
- No siempre dos predicados están relacionados por debilidad/fortaleza.
 - Ejemplo: $P \equiv x = 10$, $Q \equiv x = -3$ (son disjuntos)
 - Ejemplo: $P \equiv x \geq 3$, $Q \equiv x \leq -3$ (son disjuntos)
 - Ejemplo de dos que no son disjuntos: $P \equiv x > 3$, $Q \equiv x < 5$, ya que el estado $x \rightarrow 1$ satisface el 2do pero no el 1ro, y el estado $x \rightarrow 7$ satisface el 1ro pero no el 2do. (igual no son disjuntos ya que ambos contienen al " $x \rightarrow 4$ ")
 - Otro: $P \equiv x \leq 3$, $Q \equiv x \geq -3$

Weakest Precondition (WP)

DEFINICIÓN: Dado un programa S (o sea, una sentencia) y un predicado Q (la postcondición), la **weakest precondition** (precondición más débil) es el predicado más débil P tal que vale

$$\{ P \} S \{ Q \}$$

Denotamos a P como "wp.S.Q".

Relación entre la WP y la Terna de Hoare

Observacion 1: ¿Vale la siguiente terna?

$$\{ wp.S.Q \} S \{ Q \}$$

Sí vale, ya que por definición la wp es tal que cumple la terna.

Observacion 2: Si buscamos otra precondition P' tal que vale la terna:

$$\{ P' \} S \{ Q \}$$

¿Qué podemos decir de la relación entre la wp y P'?

Podemos decir que **P' es más fuerte que la wp**:

$$P' \Rightarrow wp.S.Q$$

ya que por definición la wp es la más débil de las que cumplen la terna.

Resumiendo:

$$\{ P' \} S \{ Q \} \text{ implica } P' \Rightarrow wp.S.Q$$

Observación 3: Si tenemos un predicado **P' más fuerte que la wp**:

$$P' \Rightarrow wp.S.Q (*)$$

¿Qué podemos decir de la siguiente terna?

$$\{ P' \} S \{ Q \}$$

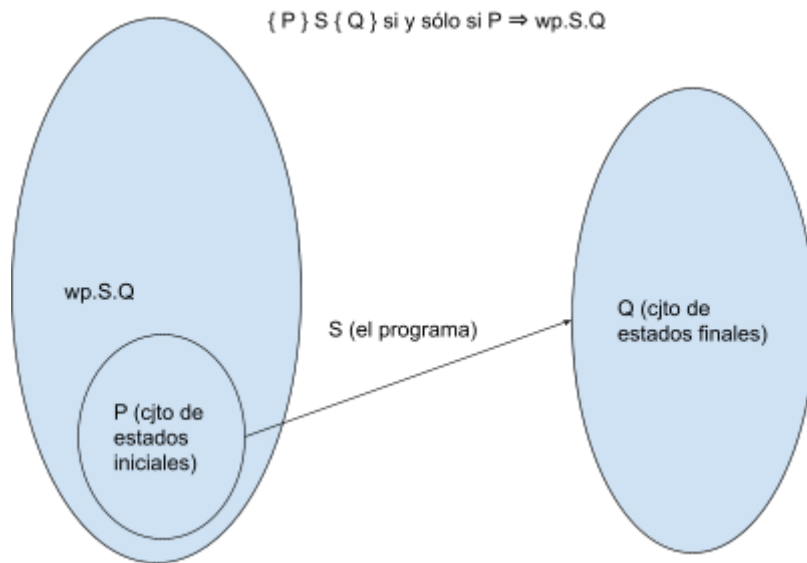
La terna vale, ya que si un estado inicial satisface P' entonces satisface la wp (por (*)). Por otro lado, la wp satisface la terna (observación 1), luego ejecutar S nos deja en un estado final que satisface Q.

Conclusión: A partir de las observaciones 2 y 3 podemos concluir que:

$$\{ P' \} S \{ Q \} \text{ sí y sólo si } P' \Rightarrow wp.S.Q$$

Reescribiendo usando P en lugar de P':

$\{ P \} S \{ Q \} \text{ sí y sólo si } P \Rightarrow wp.S.Q$
--



Intuición: La $wp.S.Q$ es la precondition más débil, luego incluye a cualquier estado inicial válido (o sea, estado inicial tal que al ejecutar S termino en un estado final que satisface Q). Luego cualquier otra precondition válida posible P debe ser más fuerte que la wp (o sea, implicarla: $P \Rightarrow wp.S.Q$)

Ejemplo anterior:

```
{ P }
x := x * x  ← S
{ x ≥ 9 }  ← Q
```

Acá ya vimos que la wp es $|x| \geq 3$. O sea:

$$|x| \geq 3 \equiv wp.S.Q \equiv wp.(x := x * x).(x \geq 9)$$

Otro ejemplo:

```
{ P }
x := x * x
{ x ≥ 9 ∧ x mod 2 = 0 }
```

¿Cuál es la $wp.S.Q$?

- Es $|x| \geq 3$ pero además x debe ser par: $|x| \geq 3 \wedge x \bmod 2 = 0$

Otro ejemplo:

```
{ P }
y, x := y + y, x + y
{ x = 7 ∧ y = 10 }
```

¿Cuál es la $wp.S.Q$? Es $x = 2 \wedge y = 5$. Podemos chequearlo haciendo la ejecución.

Otro ejemplo:

```

{ P }
  x := x + 2
{ x ≥ 7 }

```

La wp.S.Q es $x \geq 5$, ya que para que x sea ≥ 7 después de haberle asignado $x + 2$, originalmente x tendría que haber sido ≥ 5 .

¿Hay alguna forma sistemática de calcular la precondition más débil? Sí para la mayoría de los tipos de sentencias que tenemos:

- skip
- asignación ($:=$)
- secuenciación ($;$)
- condicional (if)

Para el ciclo (do) no vamos a tener una forma de calcular la wp (usaremos otras técnicas).

WP de la asignación

Supongamos que tenemos la siguiente asignación S:

$$v_1, v_2, \dots, v_n := E_1, E_2, \dots, E_n$$

y una postcondición Q.

Luego, la weakest precondition para S y Q es el siguiente predicado:

$$\text{wp.S.Q} \equiv Q(v_1 \leftarrow E_1, v_2 \leftarrow E_2, \dots, v_n \leftarrow E_n)$$

Esto es, tomar la postcondición, y reemplazar cada variable asignada por la expresión que lleva asignada.

Probemos con un ejemplo:

```

{ P }
  x := x + 2 ← S
{ x ≥ 7 } ← Q

```

Luego,

```

wp.(x := x + 2).(x ≥ 7)
≡ { definición de wp para la asignación }
  (x ≥ 7)(x ← x + 2)
≡ { hago la sustitución }
  x + 2 ≥ 7
≡ { arit. }
  x ≥ 5

```

o sea que parece que anda.

Otro:

{ P }

y, x := y + y, x + y

{ x = 7 ∧ y = 10 }

wp.(y, x := y + y, x + y).(x = 7 ∧ y = 10)

≡ { def. wp para := }

(x = 7 ∧ y = 10)(y ← y + y, x ← x + y)

≡ { aplico sust. }

x + y = 7 ∧ y + y = 10

(hasta acá alcanza pero podemos simplificar un poco:)

≡ { despejamos la y }

x + y = 7 ∧ y = 5

≡ { leibniz }

x + 5 = 7 ∧ y = 5

≡ { despejamos la x }

x = 2 ∧ y = 5

¡Anda!

Ejercicio: Calcular la wp analíticamente (o sea, usando la definición) para todos los ejemplos que vimos a ojo.

WP del skip

Supongamos que tengo el programa skip y una poscondición Q:

{ ??? }

skip

{ Q }

¿Cuál es la wp.skip.Q?

Veamos predicados que como pre, hacen valer la terna:

- True? Seguro que no porque al ejecutar skip (o sea, no hacer nada), no puedo garantizar que vale Q.
- False? Si, lo hemos visto varias veces
- Q? Sí, porque al ejecutar skip (o sea, no hacer nada), llego a un estado que satisface Q.
- Algo más fuerte que Q? O sea, un Q' tal que $Q' \Rightarrow Q$? Vale, ya que Q' sería un subconjunto de Q.
- Algo más débil que Q? No necesariamente, me estaría acercando a True.,

De todas estas reflexiones podemos ver que: con Q vale la terna, y no podemos encontrar nada más débil que Q, así que Q es la wp.S.Q:

$$wp.skip.Q \equiv Q$$

Derivación de programas imperativos

Tengo la terna:

$\{ P \}$ $\leftarrow$ **precondición dada**
 $S \quad ???$ $\leftarrow$ **el programa que tengo que encontrar**
 $\{ Q \}$ $\leftarrow$ **postcondición dada**

Proponer y demostrar. Refinar y encontrar incógnitas.

Dos grandes formas de derivar/demostrar con Ternas:

- directamente con un predicado asociado a la Terna
- usando la wp y demostrando el predicado: " $P \Rightarrow wp.S.Q$ ".

Digesto

[imperativo.pdf](#)

Skip

Verificación con Terna de Hoare:

$\{ P \} skip \{ Q \}$
 $\equiv$ ("vale sí y sólo si ...")

$P \equiv Q$ (podría ser, pero es demasiado fuerte)

$Q \Rightarrow P$ (Q es mas fuerte que $P = Q$ incluido en $P =$ hay elementos de P que no estan en

Q

empiezo de un estado que satisface P, no hago nada,

el estado satisface Q? NO!)

Ejemplo: $Q \equiv x = 100, P \equiv x \geq 3$.

¿Vale esta terna? $\{ P: x \geq 3 \} skip \{ Q: x = 100 \}$ NO!

$P \Rightarrow Q$ (empezamos de un estado que satisface P, como P está incluido en Q (o $P \Rightarrow Q$) ese estado necesariamente satisface Q sin necesidad de hacer nada).

Ejemplo: $P \equiv x = 100, Q \equiv x \geq 3$.

Precondición más débil:

$$wp.skip.Q \equiv Q$$

(la wp es el conjunto que está incluido en Q y que es lo más grande posible, por lo tanto es el mismo Q.)

Observación: Acá da lo mismo usar la verificación con la terna que con la wp.

El skip en derivaciones

Tengo la terna:

{ P } ← **precondición dada**
S ← **el programa que tengo que encontrar**
{ Q } ← **postcondición dada**

¿Cuál es el programa más simple que podría existir? Es skip.

¿Puede ser que S sea skip?

¿Qué predicado debe valer para que valga la terna?

Debe valer: $P \Rightarrow Q$. **Es lo primero que uno debe pensar a la hora de derivar.**

Ejemplo: Supongamos que tenemos las siguientes constantes/variables:

Const N : Int, A : Array[0, N) of Int;

Var pos, res : Int;

{ P: res = 0 $\wedge$ pos = 0 }
S
{ Q: res = $\langle \sum i : 0 \leq i < \text{pos} : A.i \rangle$ }

¿Puede S ser skip? Sí, ya que $P \Rightarrow Q$.

Hagamos la demostración. Supongamos P (hipótesis) y partiendo de Q lleguemos a True.

Q
 $\equiv \{ \text{def. Q} \}$
res = $\langle \sum i : 0 \leq i < \text{pos} : A.i \rangle$
 $\equiv \{ \text{hip. P} \}$
0 = $\langle \sum i : 0 \leq i < 0 : A.i \rangle$
 $\equiv \{ \text{rango vacío} \}$
0 = 0
 $\equiv \{ \text{lógica} \}$
True

Otro ejemplo:

{ P: res = $\langle \sum i : 0 \leq i < \text{pos} : A.i \rangle \wedge \text{pos} = N$ }
S
{ Q: res = $\langle \sum i : 0 \leq i < N : A.i \rangle$ } // res tiene la suma de todos los elementos del arreglo

¿Puede S ser skip? ¿Qué debe valer para que valga la terna? $P \Rightarrow Q$. ¿Vale? Sí vale!

Supongamos P y veamos Q:

Q
 $\equiv \{ \text{def. Q} \}$
res = $\langle \sum i : 0 \leq i < N : A.i \rangle$
 $\equiv \{ \text{hip. pos} = N \}$
res = $\langle \sum i : 0 \leq i < \text{pos} : A.i \rangle$

$\equiv \{ \text{esta es la otra parte de la hip} \}$
True

¿Qué pasa con $Q \Rightarrow P$? ¿Vale? Sí así fuera, tendría $P \equiv Q$. No vale!! Q no dice nada de pos, podría valer cualquier cosa. **Ejemplo:** $N = 3$, $A = [43, 65, -5]$, $\text{res} = 103$, $\text{pos} = 5$. (vale Q pero no vale P).

Asignación

Sea S la sentencia:

$$v1, v2, \dots, vn := E1, E2, \dots, En$$

Verificación con Terna de Hoare:

$$\begin{aligned} &\{ P \} \\ &v1, v2, \dots, vn := E1, E2, \dots, En \\ &\{ Q \} \\ &\equiv \\ &P \Rightarrow Q(v1 \leftarrow E1, v2 \leftarrow E2, \dots, vn \leftarrow En) \end{aligned}$$

Precondición más débil:

$$wp.(v1, v2, \dots, vn := E1, E2, \dots, En).Q \equiv Q(v1 \leftarrow E1, v2 \leftarrow E2, \dots, vn \leftarrow En)$$

Esto es, tomar la postcondición, y reemplazar cada variable asignada por la expresión que lleva asignada.

Observación: Acá da lo mismo usar la verificación con la terna que con la wp.

Ejemplo:

$$\begin{aligned} &\{ P \} \\ &y, x := y + y, x + y \\ &\{ x = 7 \wedge y = 10 \} \end{aligned}$$

Calculemos la wp:

$$\begin{aligned} &wp.(y, x := y + y, x + y).(x = 7 \wedge y = 10) \\ &\equiv \{ \text{def. wp para } := \} \\ &(x = 7 \wedge y = 10)(y \leftarrow y + y, x \leftarrow x + y) \\ &\equiv \{ \text{aplicamos el reemplazo} \} // \text{ a estos dos pasos muchas veces vamos a hacerlos juntos} \\ &(x + y) = 7 \wedge (y + y) = 10 \\ &\equiv \{ \text{despejo y de la 2da ecuación} \} \\ &(x + y) = 7 \wedge y = 5 \\ &\equiv \{ \text{leibniz (reemplazo iguales por iguales)} \} \\ &(x + 5) = 7 \wedge y = 5 \\ &\equiv \{ \text{despejo x} \} \\ &x = 2 \wedge y = 5 \end{aligned}$$

Otro ejemplo: Supongamos que tenemos las siguientes constantes/variables:

Const N : Int, A : Array[0, N) of Int;
Var pos, res : Int;

Veamos esta terna:

{ ??? }
res, pos := 0, 0
{ res = $\sum i : 0 \leq i < \text{pos} : A.i$ }

¿Cual es la wp? ¿Sale a ojo? Sí, no importa el estado inicial, la terna vale siempre, por lo tanto la wp es True. Cálculo:

wp.(res, pos := 0, 0).(res = $\sum i : 0 \leq i < \text{pos} : A.i$)
≡ { def wp para :=, y aplico la sustitución }
0 = $\sum i : 0 \leq i < 0 : A.i$
≡ { rango vacío y lógica }
True

La asignación en derivaciones

Tengo la terna:

{ P } ← precondition dada
S ← el programa que tengo que encontrar
{ Q } ← postcondition dada

Ya sabemos que S no es skip (o sea, P no implica Q).

¿Puede ser que S sea una asignación? Podemos probar tomando todas las variables del programa y buscando asignarlas a "incógnitas":

v1, v2, ..., vn := I1, I2, ..., In

Luego, intentar demostrar la terna (o sea, demostrar **P ⇒ wp.S.Q**), y en el medio de la demostración, elegir expresiones posibles para las incógnitas de manera estratégica (o sea, que me sirvan esas decisiones para llegar a **True**).

Ejemplo:

{ P: True }
S
{ Q: res = $\sum i : 0 \leq i < \text{pos} : A.i$ }

¿Puede S ser skip? No, P no implica Q.

¿Puede S ser una asignación? Probemos con esta:

res, pos := E, F (incógnitas E y F)

Tratemos de demostrar la terna, eligiendo convenientemente valores para E y F que me permitan llegar a True. O sea, demostrar **P ⇒ wp.(res, pos := E, F).Q**. Supongamos P (realmente no tengo hipótesis que me sirvan) y veamos la wp:

Libertades: puedo elegir que E y F sean lo que se me cante (siempre que sean expresiones programables).

Objetivo: Llegar a True

$$\begin{aligned} & wp.(res, pos := E, F).Q \\ \equiv & \{ \text{def. wp} \} \\ & E = \langle \sum i : 0 \leq i < F : A.i \rangle \\ \equiv & \{ \text{me conviene forzar un rango vacío, o sea } \underline{\text{elegir } F=0} \} \\ & E = \langle \sum i : 0 \leq i < 0 : A.i \rangle \\ \equiv & \{ \text{ahora ya tengo rango vacío} \} \\ & E = 0 \\ \equiv & \{ \text{está claro que } \underline{\text{debo elegir } E = 0} \} \\ & 0 = 0 \\ \equiv & \{ \text{lógica} \} \\ & \text{True} \end{aligned}$$

Luego, acabamos de derivar/demostrar el siguiente programa:

$$\begin{aligned} & \{ P: \text{True} \} \\ & res, pos := 0, 0 \\ & \{ Q: res = \langle \sum i : 0 \leq i < pos : A.i \rangle \} \end{aligned}$$

Otro ejemplo: A veces en las asignaciones vamos a saber algunas de las expresiones a priori (ya sea porque nos las dicen o porque lo sabemos por intuición/creatividad).

En este ejemplo ya sabemos la asignación para pos: Quiero sí o sí incrementar pos en 1.

$$\begin{aligned} & \{ P: res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos < N \} \\ & res, pos := E, pos + 1 \\ & \{ Q: res = \langle \sum i : 0 \leq i < pos : A.i \rangle \} \end{aligned}$$

¿Qué pasaría si no estuviera obligado a incrementar pos en 1? Podría directamente usar skip ya que la pre $\Rightarrow$ la post:

$$\begin{aligned} & \{ res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos < N \} \\ & \text{skip} \\ & \{ res = \langle \sum i : 0 \leq i < pos : A.i \rangle \} \end{aligned}$$

Obviamente si esto fuera el cuerpo de un ciclo, no serviría ya que no terminaría nunca.

Sólo nos falta encontrar la incógnita E en el programa. Hagamos la demostración y elijamos E convenientemente.

Supongamos la hipótesis P: $res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge \underline{0 \leq pos < N}$

Y veamos la wp:

$$\begin{aligned} & wp.(res, pos := E, pos + 1).(res = \langle \sum i : 0 \leq i < pos : A.i \rangle) \\ \equiv & \{ \text{def. wp para } := \} \\ & E = \langle \sum i : \underline{0 \leq i < pos + 1} : A.i \rangle \quad \begin{aligned} & \text{(esto suma: } A.0 + A.1 + \dots + A.(pos-1) + A.pos) \\ & \text{(de esta suma, lo subrayado es "res")} \\ & \text{(hagamos aparecer res:)} \end{aligned} \\ \equiv & \{ \text{lógica} \} \end{aligned}$$

$$\begin{aligned}
& E = \langle \sum i : 0 \leq i < \text{pos} \vee i = \text{pos} : A.i \rangle \\
& \equiv \{ \text{part. rango} \} \\
& E = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle + \langle \sum i : i = \text{pos} : A.i \rangle \\
& \equiv \{ \text{hipótesis} \} \\
& E = \text{res} + \langle \sum i : i = \text{pos} : A.i \rangle \\
& \equiv \{ \text{rango unitario} \} \\
& E = \text{res} + A.\text{pos} \\
& \equiv \{ \text{elijo } \mathbf{E = res + A.pos} \} \\
& \text{res} + A.\text{pos} = \text{res} + A.\text{pos} \\
& \equiv \{ \text{lógica} \} \\
& \text{True}
\end{aligned}$$

Lo logramos! Acabamos de derivar/demostrar el siguiente programa:

$$\begin{aligned}
& \{ P: \text{res} = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle \wedge 0 \leq \text{pos} < N \} \\
& \text{res, pos} := \text{res} + A.\text{pos}, \text{pos} + 1 \\
& \{ Q: \text{res} = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle \}
\end{aligned}$$

(que si se fijan, es el cuerpo del ciclo del programa que suma un arreglo)

Observación: Fijarse que la pre y la post tiene una parte en común muy importante:

$$\text{res} = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle$$

Que “no cambia” en el sentido de que vale **antes y después** de la asignación. Esta asignación, recordemos, es el cuerpo del ciclo de un programa más grande, y este predicado se denomina **invariante** del ciclo.

Pequeño repaso: Quiero encontrar E tal que vale la siguiente Terna:

{ P }

res, pos := E, pos + 1

{ Q }

O sea, equivalentemente, que vale:

$P \Rightarrow \text{wp.}(\text{res, pos} := E, \text{pos} + 1).Q$

O sea, equivalentemente, que si yo tomo P como hipótesis, vale:

$\text{wp.}(\text{res, pos} := E, \text{pos} + 1).Q$

Luego, si intento hacer la demostración, y en algún momento encuentro alguna expresión que vaya ser E que me sirva para llegar a True, **ya estaría.**

Y pude hacerlo. ya que si elijo que E sea “res + A.pos”, puedo llegar a True.

Condicional (if)

Verificación con Terna de Hoare:

{ P }

if $B_1 \rightarrow S_1$

[] $B_2 \rightarrow S_2$

...

[] $B_n \rightarrow S_n$

fi
{ Q }
 $\equiv$

i) Sí o sí debe valer al menos una de las guardas:

$$P \Rightarrow B_1 \vee B_2 \vee \dots \vee B_n$$

ii) Todas las ramas son válidas: Para todo i,

para la rama i, **si ejecuto S_i** , quedo en un estado final que satisface Q.

¿Qué puedo asumir que vale en el estado inicial? Vale P, pero además vale B_i , ya que si no, no ejecutaría esta rama.

Escrito con Ternas, sería:

$$\{ P \wedge B_i \} S_i \{ Q \}$$

Dijimos que era para todos los i, escribimos:

$$\begin{aligned} & \{ P \wedge B_1 \} S_1 \{ Q \} \\ & \wedge \{ P \wedge B_2 \} S_2 \{ Q \} \\ & \wedge \dots \\ & \wedge \{ P \wedge B_n \} S_n \{ Q \} \end{aligned}$$

Resumiendo:

$$\begin{aligned} & \{ P \} \\ & \textbf{if } B_1 \rightarrow S_1 \\ & \textbf{[] } B_2 \rightarrow S_2 \\ & \dots \\ & \textbf{[] } B_n \rightarrow S_n \\ & \textbf{fi} \\ & \{ Q \} \\ & \equiv \\ & (P \Rightarrow B_1 \vee B_2 \vee \dots \vee B_n) \\ & \wedge \{ P \wedge B_1 \} S_1 \{ Q \} \\ & \wedge \{ P \wedge B_2 \} S_2 \{ Q \} \\ & \wedge \dots \\ & \wedge \{ P \wedge B_n \} S_n \{ Q \} \end{aligned}$$

Ejemplo:

$\{ P \}$
if $x > 0 \rightarrow$ "hago algo"
[] $x < 0 \rightarrow$ "hago otra cosa"
fi
{ Q }

Acá, si P es True esta terna no vale ya que puede suceder que todas las guardas sean falsas (si en el estado $x \rightarrow 0$).

Si P es $x \neq 0$ tengo garantizado que sí o sí una de las guardas es verdadera.

—

Precondición más débil: Ejercicio verla uds en el digesto (no preocuparse por esto igual).

El condicional en derivaciones

Tengo la terna:

{ P } ← **precondición dada**
S ← **el programa que tengo que encontrar**
{ Q } ← **postcondición dada**

¿Puede ser que S sea un condicional?

Supongamos que ya sabemos que S no es skip, y vamos a probar con una asignación, planteando las incógnitas.

Si al derivar la asignación, surge la necesidad de un análisis por casos, entonces lo que estoy derivando en realidad es un **if** cuyas ramas son asignaciones distintas.

Ejemplo: Máximo entre dos números, suponiendo que no puedo usar el operador max en mi programa.

Var x, y, res : Int;
{ P: $x = X \wedge y = Y$ } (X e Y variables de especificación)
S
{ Q: $res = X \max Y$ }

S claramente no es skip, así que voy a probar con una asignación. Planteo la asignación:

res, x, y := E, F, G

¿Hace falta realmente? No, la postcondición sólo habla de res, así que sólo debo preocuparme por asignar a res:

res := E (E incógnita)

Ahora, derivemos: O sea, busquemos E tal que vale la terna, o sea, que vale

P $\Rightarrow$ wp.S.Q.

Suponemos P como hipótesis: $x = X \wedge y = Y$

Vemos la wp:

wp.(res := E).(res = X max Y)
 $\equiv$ { def. wp para la asignación }
E = X max Y
 $\equiv$ { **¿puedo elegir X max Y? no, porque X e Y no existen en el programa.**
Resolvemos eso usando la hipótesis: $x = X \wedge y = Y$ }
E = x max y
 $\equiv$ { **¿puedo elegir x max y? no, porque según el enunciado no puedo usar "max"**
puedo resolver esto haciendo un **análisis por casos** para resolver el max.
}

- Caso 1: **$x \geq y$** : (1er guarda del if)
 $\equiv$ { propiedad de max, si $x \geq y$, $x \max y = x$ }
E = x
 $\equiv$ { **elijo que E sea "x"** }

- True
- Caso 2: $x \leq y$ (también sirve $x < y$): (2da guarda del if)
 $\equiv \{ \text{prop. max} \}$
 $E = y$
 $\equiv \{ \text{elijo que } E \text{ sea "y"} \}$
 True

Acabo de intentar derivar una asignación, pero llegué a un análisis por casos, con dos elecciones distintas para E. Esto se traduce a un if, y el programa (con su terna) queda de la siguiente manera:

```
Var x, y, res : Int;
{ P: x = X  $\wedge$  y = Y } (X e Y variables de especificación)
if x  $\geq$  y  $\rightarrow$  res := x
[] x  $\leq$  y  $\rightarrow$  res := y
fi
{ Q: res = X max Y }
```

Observaciones:

- Las guardas de los casos deben ser expresiones válidas en lenguaje de programación.
- Los casos deben cubrir todas las posibilidades (siempre suponiendo la hipótesis P).

Otro ejemplo:

Sacado del problema de contar cuántos números divisibles por 6 tenemos entre 0 y un número N dado.

```
Const N : Int ;
Var numero_actual, resultado : Int ;
```

```
{ P: resultado =  $\langle$  N i :  $0 \leq i < \text{numero\_actual}$  : i mod 6 = 0  $\rangle \wedge \text{numero\_actual} \geq 0$  }
resultado, numero_actual := E, numero_actual + 1
{ Q: resultado =  $\langle$  N i :  $0 \leq i < \text{numero\_actual}$  : i mod 6 = 0  $\rangle$  }
```

Derivemos: Encontremos E tal que vale esta terna, o sea que vale $P \Rightarrow \text{wp.S.Q.}$

Supongamos P:

```
resultado =  $\langle$  N i :  $0 \leq i < \text{numero\_actual}$  : i mod 6 = 0  $\rangle \wedge \text{numero\_actual} \geq 0$ 
```

Y veamos la wp:

```
wp.(resultado, numero_actual := E, numero_actual + 1).Q
 $\equiv \{ \text{def wp. para :=} \}$ 
E =  $\langle$  N i :  $0 \leq i < \text{numero\_actual} + 1$  : i mod 6 = 0  $\rangle$ 
 $\equiv \{ \text{¿puedo elegir } E = \langle$  N i :  $0 \leq i < \text{numero\_actual} + 1$  : i mod 6 = 0  $\rangle$  ?
```

no, no es programable.

lógica y partición de rango de conteo,

podemos ya que el rango es no vacío, sabiendo que $\text{numero_actual} \geq 0$.

```
}
```

$$E = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle +$$

$$\langle N \ i : i = \text{numero_actual} : i \bmod 6 = 0 \rangle$$

$\equiv \{ \text{hipótesis: esta parte ya está calculada en la variable resultado} \}$

$$E = \text{resultado} + \langle N \ i : i = \text{numero_actual} : i \bmod 6 = 0 \rangle$$

$\equiv \{ \text{rango unitario para el conteo} \}$

$$E = \text{resultado} + (\text{numero_actual} \bmod 6 = 0 \rightarrow 1$$

$$\quad \quad \quad \square \text{ numero_actual} \bmod 6 \neq 0 \rightarrow 0$$

$$\quad \quad \quad)$$

$\equiv \{ \text{¿puedo elegir } E = \text{resultado} + (\text{numero_actual} \bmod 6 = 0 \rightarrow 1$

$\quad \quad \quad \square \text{ numero_actual} \bmod 6 \neq 0 \rightarrow 0$

$\quad \quad \quad) \text{ ? No puedo, porque no es una expresión válida en mi lenguaje de programación el análisis por casos.}$

para deshacerme de la expresión con análisis por casos, hago en análisis por casos en la derivación }

- Caso 1: **numero_actual mod 6 = 0** (es programable)
 $\equiv \{ \text{resuelvo la expresión por casos} \}$
 $E = \text{resultado} + 1$
 $\equiv \{ \text{elijo } \underline{E = \text{resultado} + 1} \}$
 True
- Caso 2: **numero_actual mod 6 ≠ 0** (es programable)
 $\equiv \{ \text{resuelvo la expresión por casos} \}$
 $E = \text{resultado} + 0$
 $\equiv \{ \text{arit.} \}$
 $E = \text{resultado}$
 $\equiv \{ \text{elijo } \underline{E = \text{resultado}} \}$
 True

Luego, acabamos de derivar el siguiente programa (con su terna):

$$\{ P: \text{resultado} = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle \wedge \text{numero_actual} \geq 0 \}$$

if numero_actual mod 6 = 0 $\rightarrow$

$$\quad \text{resultado, numero_actual} := \text{resultado} + 1, \text{numero_actual} + 1$$

$\square$ numero_actual mod 6 ≠ 0 $\rightarrow$

$$\quad \text{resultado, numero_actual} := \text{resultado}, \text{numero_actual} + 1$$

fi

$$\{ Q: \text{resultado} = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle \}$$

O, equivalentemente,

$$\{ P: \text{resultado} = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle \wedge \text{numero_actual} \geq 0 \}$$

if numero_actual mod 6 = 0 $\rightarrow$

$$\quad \text{resultado, numero_actual} := \text{resultado} + 1, \text{numero_actual} + 1$$

$\square$ numero_actual mod 6 ≠ 0 $\rightarrow$

$$\quad \text{numero_actual} := \text{numero_actual} + 1$$

fi

$$\{ Q: \text{resultado} = \langle N \ i : 0 \leq i < \text{numero_actual} : i \bmod 6 = 0 \rangle \}$$

Observación: Al derivar, encontré que la incógnita E es la misma variable que estaba asignando, esto es como no hacer nada así que se puede omitir esa asignación.

Observación: Este es sólo un escenario posible en el que pueden aparecer IF en derivaciones. En otras situaciones, tendremos que proponer IF's usando más creatividad.

Secuenciación (;)

Verificación con Terna de Hoare:

Tengo dos programas S y T secuenciados:

$$\{ P \} S ; T \{ Q \}$$

La correctitud de esta terna, va a depender de la correctitud de ternas planteadas sobre los dos programas: Si yo encuentro un predicado R tal que valen estas dos ternas:

$$\begin{aligned} \{ P \} S \{ R \} \wedge \\ \{ R \} T \{ Q \} \end{aligned}$$

$\begin{aligned} &\{ P \} \\ &S ; \\ &T \\ &\{ Q \} \\ \equiv & \\ &\text{Existe un predicado R tal que:} \\ &\quad \{ P \} S \{ R \} \wedge \\ &\quad \{ R \} T \{ Q \} \end{aligned}$

(R hace de postcondición para la primera terna, y de precondition para la segunda)

(R es un predicado intermedio que habla de los estados posibles luego de ejecutar S y antes de ejecutar T.)

Precondición más débil:

$$wp.(S ; T).Q \equiv wp.S.(wp.T.Q)$$

Ejemplo (práctico 6, ej 1e):

$$\begin{aligned} &\{ P: ??? \} \\ &a, x := x, y ; \quad // S ; \\ &y := a \quad // T \\ &\{ Q: x = B \wedge y = A \} \end{aligned}$$

Cuál es la $wp.(a, x := x, y ; y := a).Q$??? Veamos:

$$\begin{aligned} &wp.(a, x := x, y ; y := a).Q \\ \equiv &\{ \text{def. wp para la secuenciación: } wp.(S ; T).Q \equiv wp.S.(wp.T.Q) \} \\ &wp.(a, x := x, y).(wp.(y := a).Q) \end{aligned}$$

$\equiv \{ \text{primero aplicamos def. de wp para } := \text{ en la wp de adentro } \}$
 $\text{wp.}(a, x := x, y).(x = B \wedge a = A)$
 $\equiv \{ \text{def. wp para } := \text{ de nuevo } \}$
 $y = B \wedge x = A$

Esta es la wp, así que la terna queda:

$\{ P: y = B \wedge x = A \}$
 $a, x := x, y ;$
 $y := a$
 $\{ Q: x = B \wedge y = A \}$

¿Qué pasa si yo quiero demostrar esta Terna usando la “**Verificación con Terna de Hoare**”. Debo encontrar un predicado R tal que valgan estas dos ternas:

$\{ P: y = B \wedge x = A \}$
 $a, x := x, y ;$
 $\{ R \}$
 y
 $\{ R \} \text{ // si pongo wp.}(y := a).Q \text{ me aseguro que vale esta terna}$
 $y := a$
 $\{ Q: x = B \wedge y = A \}$

¿Qué puede ser R? Si yo hago que R sea $\text{wp.}(y := a).(x = B \wedge y = A)$, ya me aseguro la 2da terna por definición **de wp**. Sólo me quedaría demostrar la primer terna:

$\{ P: y = B \wedge x = A \}$
 $a, x := x, y ;$
 $\{ R: \text{wp.}(y := a).Q \}$

Y si nos fijamos bien, demostrar esta terna es demostrar:

$P \Rightarrow \text{wp.}(a, x := x, y).R$
 o sea
 $P \Rightarrow \underline{\text{wp.}(a, x := x, y).(\text{wp.}(y := a).Q)}$

Entonces, se justifica que esta dos wp anidadas sean de hecho la wp de la secuenciación.

La secuenciación en derivaciones

Tengo la terna:

$\{ P \} \leftarrow$ **precondición dada**
 $S \leftarrow$ **el programa que tengo que encontrar**
 $\{ Q \} \leftarrow$ **postcondición dada**

¿Puede ser que S sea una secuenciación?

Muchas veces lo vamos a ver por creatividad.

Ejemplo: Promedio de un arreglo de N elementos.

```
{ P: N > 0 }
S
{ Q: promedio =  $\langle \sum i : 0 \leq i < N : A.i \rangle / N$  }
```

¿Cómo sería el programa que resuelve este problema?

Primero hay que sumar todos los elementos del arreglo, **después** dividir por N. Esto no es otra cosa que una secuenciación de dos programas, con un predicado intermedio:

```
Const N : Int, A : array[0,N) of Float;
Var promedio, suma : Float;
{ P: N > 0 }
  S1 ; // acá calculo la suma
{ R: suma =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
  S2
{ Q: promedio =  $\langle \sum i : 0 \leq i < N : A.i \rangle / N$  }
```

Acabo de refinar mi programa S, convirtiéndolo en una secuenciación, y ahora tengo que hacer dos derivaciones: la de S1 (con su terna), y la de S2 (con su terna).

S1 va a ser un ciclo (ya lo hicimos), ya veremos cómo se deriva/demuestra.

S2 va a ser el siguiente programa: promedio := suma / N (**ejercicio: derivar esto!**)

Observación: Parecido a modularización!! Primero identificamos un problema accesorio (la suma), necesario para resolver el problema principal (el promedio).

Ciclo / Repetición (do)

Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.

Programa:

```
Const N : Int ;
Var numero_actual, resultado : Int ;
{ N ≥ 0 }
numero_actual , resultado := 0 , 0 ;
{ P: numero_actual = 0 ∧ resultado = 0 }  <-- precondición del do
do numero_actual ≤ N →
  if numero_actual mod 6 = 0 →
    numero_actual, resultado := numero_actual + 1, resultado + 1 ;
  [] numero_actual mod 6 ≠ 0 →
    numero_actual := numero_actual + 1
  fi
od
{ Q: resultado =  $\langle N \ i : 0 \leq i \leq N : i \bmod 6 = 0 \rangle$  }  <-- postcondición del do
```

Tenemos entonces una terna de la forma:

```
{ P }  
do B → // guarda  
    S // cuerpo del ciclo  
od  
{ Q }
```

donde S es el cuerpo del ciclo (un if en este caso).

¿Cómo demostramos esta terna? Tenemos que lograr conectar lógicamente la precondition con la postcondición. Sin embargo, no sabemos cuántas veces vamos a iterar el ciclo.

Supongamos que iteramos n veces. Ejecutar el ciclo es equivalente a ejecutar la siguiente secuenciación:

```
S ; S ; S ; S ; .... ; S (n veces)
```

O sea que queremos demostrar una terna:

```
{ P }  
    S ; S ; S ; S ; .... ; S (n veces)  
{ Q }
```

Recordando la terna de la secuenciación, ¿qué necesito para poder demostrar esto?

Necesitamos predicados intermedios que valen entre todas las secuenciaciones.

Nos conviene tener un único predicado intermedio para facilitar las pruebas. Llamemos por ahora R a ese predicado:

```
{ P }  
    skip  
{ R }  
    S ;  
{ R }  
    S ;  
{ R }  
    .... ;  
    S (n veces)  
{ R }  
    skip  
{ Q }
```

Algo importante que nos faltó considerar acá es la guarda. ¿En qué momentos vale y en qué momentos no vale?

```
{ P }  
    skip  
{ R ∧ B } // si voy a ejecutar S, es que la guarda dio True  
    S ;  
{ R ∧ B } // aca tambien  
    S ;  
{ R ∧ B } // aca tambien  
    .... ;  
    S (n veces)  
{ R ∧ ¬ B } // acá la guarda no vale, el ciclo termina  
    skip
```

$\{ Q \}$

Al plantearlo de esta manera, me sirve ya que las demostraciones que tengo que hacer se reducen a las siguientes:

$\{ P \} \text{ skip } \{ R \}$ // en este estado final puede valer B ó $\neg B$
 $\{ R \wedge B \} S \{ R \}$ // en este estado final puede valer B ó $\neg B$
 $\{ R \wedge \neg B \} \text{ skip } \{ Q \}$

El predicado R se llama "**invariante de ciclo**" así que por lo general vamos a denotarlo "I" o "INV":

$\{ P \} \text{ skip } \{ INV \}$ // "el invariante vale al **principio** del ciclo"
 $\{ INV \wedge B \} S \{ INV \}$ // "el cuerpo del ciclo preserva el invariante"
 $\{ INV \wedge \neg B \} \text{ skip } \{ Q \}$ // "el invariante implica la **postcondición** al terminar el ciclo"

¿Porqué se llama "invariante"? Porque es un predicado que se preserva a lo largo de todo el ciclo, y eso me permite conectar lógicamente la precondición con la postcondición.

Reescribamos las ternas skip usando su definición:

- i) $P \Rightarrow INV$ // "el invariante vale al **principio** del ciclo"
- ii) $\{ INV \wedge B \} S \{ INV \}$ // "el cuerpo del ciclo preserva el invariante"
- iii) $INV \wedge \neg B \Rightarrow Q$ // "el invariante implica la **postcondición** al terminar el ciclo"

Me falta además garantizar la terminación del ciclo:

- iv) "el ciclo termina" (ya lo veremos más adelante)

Resumiendo:

Verificación con Terna de Hoare:

 $\{ P \}$ $\underline{\text{do}} B \rightarrow$ S $\underline{\text{od}}$ $\{ Q \}$ $\equiv$

Existe invariante INV tal que:

i) $P \Rightarrow INV$

ii) $\{ INV \wedge B \} S \{ INV \}$

iii) $INV \wedge \neg B \Rightarrow Q$

iv) “el ciclo termina”

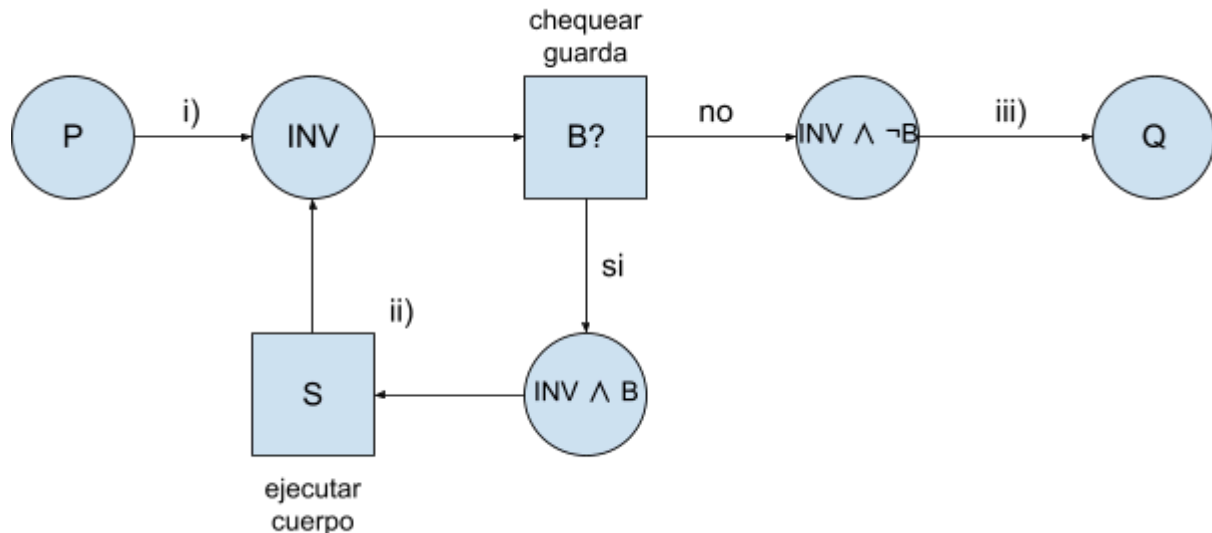
Precondición más débil: No la vemos.

Intuición del invariante: El invariante expresa el “resultado intermedio” o “resultado parcial” que se está calculando en el ciclo, es decir, la parte del problema que llevamos resuelta hasta ahora. Al terminar el ciclo, el “resultado intermedio” se convierte en el “resultado final” (gracias al requisito iii).

Figura: Este gráfico ilustra la ejecución completa de un ciclo.

$\{P\}$
do $B \rightarrow$
 $\{INV \wedge B\}$
 S
 $\{INV\}$
od
 $\{Q\}$

i) $P \Rightarrow INV$
 ii) $\{INV \wedge B\} S \{INV\}$
 iii) $INV \wedge \neg B \Rightarrow Q$



Ejemplo: Dado un número entero $N > 0$, queremos contar cuántos **números entre 0 y N** son múltiplos de 6.

Programa:

```

Const N : Int ;
Var numero_actual, resultado : Int ;
{ N ≥ 0 }
numero_actual , resultado := 0 , 0 ;
{ P: numero_actual = 0 ∧ resultado = 0 } <-- precondition del do
skip ; // por la condición i) "el invariante vale al principio"
{ INV }
do numero_actual ≤ N →
{ INV ∧ (numero_actual ≤ N) }
{ resultado = ??? }
if numero_actual mod 6 = 0 →
numero_actual, resultado := numero_actual + 1, resultado + 1 ;
[] numero_actual mod 6 ≠ 0 →
numero_actual := numero_actual + 1
fi
{ INV } // por la condición ii) "el invariante se preserva en el cuerpo"
od
{ INV ∧ ¬B }
skip // por la condición iii) "el invariante implica la post al terminar"
{ Q: resultado = ⟨ N i : 0 ≤ i ≤ N : i mod 6 = 0 ⟩ }
// postcondición del do (y también de todo el programa)

```

$B \equiv \text{numero_actual} \leq N$

$P \equiv \text{numero_actual} = 0 \wedge \text{resultado} = 0$

$Q \equiv \text{resultado} = \langle N \ i : 0 \leq i \leq N : i \bmod 6 = 0 \rangle$

Debo encontrar el invariante INV tal que vale lo siguiente:

- i) $P \Rightarrow INV$ (“el invariante vale al principio”)
- ii) $\{ INV \wedge B \} S \{ INV \}$ (“el cuerpo del ciclo preserva el invariante”)
- iii) $INV \wedge \neg B \Rightarrow Q$ (“el invariante garantiza la postcondición al terminar el ciclo”)

¿Qué puede ser INV?

- Puede ser True?
 - Vale i)? Sí.
 - Vale ii)? Sí (siempre que S termine)
 - **Vale iii)? $True \wedge \neg B \Rightarrow Q$?**
No! Falta muchísima información para poder garantizar Q.

Necesito sí o sí que el INV me diga algo sobre la variable resultado para que **valga iii**).

Apartado: Apreciaciones generales sobre los requisitos para el invariante: Tanto los requisitos i) y ii) son fáciles de satisfacer con un INV débil (incluso siempre valen con True):

i) $P \Rightarrow INV$ (a más débil INV, más fácil de que valga)

ii) $\{ INV \wedge B \} S \{ INV \}$ (lo mismo)

Pero el **requisito iii) me exige cierta fortaleza en el invariante** (un invariante **informativo**)

iii) $INV \wedge \neg B \Rightarrow Q$

(al aparecer del lado izquierdo del $\Rightarrow$ debe tener la fuerza suficiente para implicar a la postcondición Q)

El invariante sí o sí me debe decir algo sobre el resultado.

¿Qué podemos decir?

- **resultado ≥ 0** vale, pero no alcanza para implicar Q
- **resultado $\leq N+1$** vale pero tampoco alcanza.
- resultado = **“el resultado intermedio que tenemos calculado hasta el momento”**
= “hasta ahora sabemos la cantidad de divisibles por 6 entre 0 y
numero_actual - 1 (en esta iteración veremos qué pasa con numero_actual)”
= $\langle N \mid 0 \leq i \leq \text{numero_actual} - 1: i \bmod 6 = 0 \rangle$

Probemos con este invariante:

$INV \equiv \text{resultado} = \langle N \mid 0 \leq i \leq \text{numero_actual} - 1: i \bmod 6 = 0 \rangle$

Este invariante me representa el “resultado parcial” que yo estoy calculando.

Pregunta: ¿Cuándo este “resultado parcial” se convierte en un “resultado final” (caracterizado Q)? Cuando el ciclo termina, o sea, cuando la guarda se hace falsa (iii):

¿Vale?:

$INV \wedge \neg B \Rightarrow Q$

Veamos:

```

resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$  // INV
 $\wedge$  numero\_actual > N //  $\neg B$ 
 $\Rightarrow$  resultado =  $\langle N \mid 0 \leq i \leq N : i \bmod 6 = 0 \rangle$  // Q

```

¿me alcanzan las hipótesis para garantizar Q? Tendría que pasar que

numero_actual - 1 = N, o sea que

numero_actual = N + 1. ¿sabemos eso? casi, sabemos numero_actual > N, no vendría bien saber también que **numero_actual $\leq$ N + 1.**

¿Podremos agregarlo al invariante? Nuevo invariante:

```

INV  $\equiv$  resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$ 
 $\wedge$  numero\_actual  $\leq$  N + 1

```

Ahora sí vamos a tener que $INV \wedge \neg B \Rightarrow Q$, ya que:

```

resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$  // INV
 $\wedge$  numero\_actual  $\leq$  N + 1 // INV
 $\wedge$  numero\_actual > N //  $\neg B$ 
 $\Rightarrow$  resultado =  $\langle N \mid 0 \leq i \leq N : i \bmod 6 = 0 \rangle$  // Q

```

Fijarse que por 3ro excluido, numero_actual debe ser sí o sí N + 1.

Luego hemos encontrado un INV tal que vale el requisito iii).

Falta demostrar los requisitos i) y ii):

i) $P \Rightarrow INV$

o sea

numero_actual = 0 $\wedge$ resultado = 0

$\Rightarrow$

```

resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$ 
 $\wedge$  numero\_actual  $\leq$  N + 1

```

¿vale o no vale? Sí vale.

ii) $\{ INV \wedge B \} S \{ INV \}$

o sea:

```

{ resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$ 
 $\wedge$  numero\_actual  $\leq$  N + 1
 $\wedge$  numero\_actual  $\leq$  N
}
  if numero\_actual mod 6 = 0  $\rightarrow$ 
    numero\_actual, resultado := numero\_actual + 1, resultado + 1 ;
  [] numero\_actual mod 6  $\neq$  0  $\rightarrow$ 
    numero\_actual := numero\_actual + 1
  fi
{ resultado =  $\langle N \mid 0 \leq i \leq \text{numero\_actual} - 1 : i \bmod 6 = 0 \rangle$ 
 $\wedge$  numero\_actual  $\leq$  N + 1
}

```

¿Vale esta terna? Casi vale, en realidad al intentar demostrarla vamos a ver que necesitamos agregar al invariante una cosita más: “ $0 \leq \text{numero_actual}$ ”:

$$\text{INV} \quad \equiv \quad \text{resultado} = \langle N \mid 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle \\ \wedge \quad 0 \leq \text{numero_actual} \leq N + 1$$

(a esta demostración/derivación ya la hicimos la clase pasada al ver el if).

Conclusión: Este invariante me caracteriza el “resultado parcial” ya que me dice que la variable **resultado** guarda el conteo de los múltiplos de 6 entre 0 y $\text{numero_actual} - 1$.

Además me dice que numero_actual está entre 0 y $N+1$.

- Al inicio, numero_actual está en 0. (y $\text{resultado} = 0$, por lo que vale INV).
- En el cuerpo del ciclo, numero_actual se incrementa en 1.
- Al terminar el ciclo, numero_actual vale $N+1$.

Entonces, el INV me dice que la var. **resultado** guarda el conteo desde 0 y N , o sea que vale la postcondición.

=====

¿Qué puede ser INV? Proponemos:

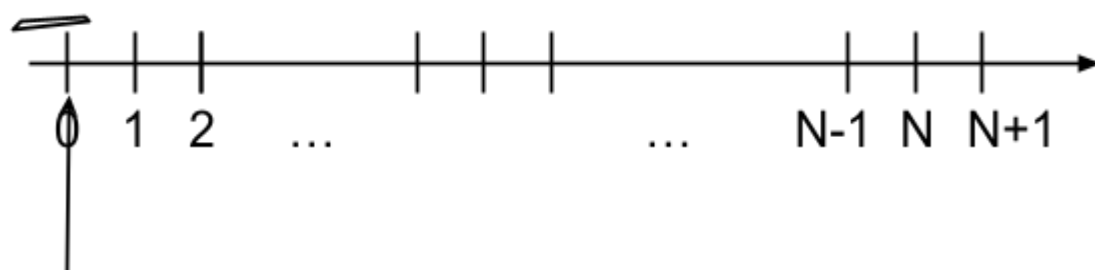
$$\text{INV} \quad \equiv \quad \text{resultado} = \langle N \mid 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle \\ \wedge \quad 0 \leq \text{numero_actual} \leq N + 1$$

Este invariante me caracteriza el “resultado parcial” ya que me dice que la variable **resultado** guarda el conteo de los múltiplos de 6 entre 0 y $\text{numero_actual} - 1$. Además me dice que numero_actual está entre 0 y $N+1$.

- Al inicio, numero_actual está en 0. (y $\text{resultado} = 0$, por lo que vale INV).
- En el cuerpo del ciclo, numero_actual se incrementa en 1.
- Al terminar el ciclo, numero_actual vale $N+1$.

Entonces, el INV me dice que la var. **resultado** guarda el conteo desde 0 y N , o sea que vale la postcondición.

resultado



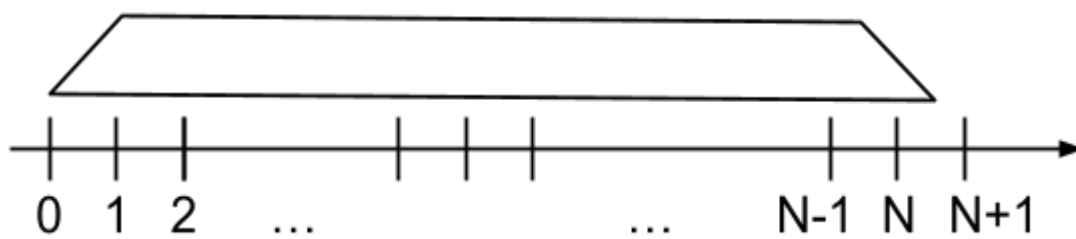
numero_actual

resultado



numero_actual

resultado



↑
numero_actual

=====

Demostración:

$B \equiv \text{numero_actual} \leq N$

$P \equiv \text{numero_actual} = 0 \wedge \text{resultado} = 0$

$Q \equiv \text{resultado} = \langle N \ i : 0 \leq i \leq N : i \bmod 6 = 0 \rangle$

$$\begin{aligned} \text{INV} \quad \equiv \quad & \text{resultado} = \langle N \ i : 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle \\ & \wedge \quad 0 \leq \text{numero_actual} \leq N + 1 \end{aligned}$$

i) El invariante vale al principio: $P \Rightarrow \text{INV}$

Suponemos P ($\text{numero_actual} = 0 \wedge \text{resultado} = 0$) como hipótesis y veamos INV :

INV
 $\equiv \{ \text{def. INV} \}$
 $\text{resultado} = \langle N \ i : 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle \wedge 0 \leq \text{numero_actual} \leq N + 1$
 $\equiv \{ \text{hip} \}$
 $0 = \langle N \ i : 0 \leq i \leq 0 - 1 : i \bmod 6 = 0 \rangle \wedge 0 \leq 0 \leq N + 1$
 $\equiv \{ \text{rango vacío del conteo} \}$
 $0 = 0 \wedge 0 \leq 0 \leq N + 1$
 $\equiv \{ \text{lógicas varias} \}$
 $0 \leq N + 1$
 $\equiv \{ \text{es True porque } N \geq 0 \text{ en todo el programa, es una } \underline{\text{hipótesis global}} \}$
(hipótesis global: todo lo que se diga en la precondition del programa entero sobre las constantes podemos asumir que vale siempre).
True

ii) $\{ \text{INV} \wedge B \} S \{ \text{INV} \}$

donde S es el cuerpo del ciclo. O sea:

```
{ INV ∧ B }
  if numero_actual mod 6 = 0 →
    numero_actual, resultado := numero_actual + 1, resultado + 1 ;
  □ numero_actual mod 6 ≠ 0 →
    numero_actual := numero_actual + 1
  fi
{ INV }
```

¿Cómo demostramos esto?

Vemos en el digesto para el if dos opciones: Usar directo la terna de hoare o traducir a wp. Es casi lo mismo (mejor usar directo la terna).

Observación: A esto ya lo hicimos en una clase anterior al derivar una terna para la asignación

$\text{numero_actual, resultado} := \text{numero_actual} + 1, E ;$
y llegamos a un análisis por casos igual al que tenemos ahora.

Les queda como ejercicio.

iii) $INV \wedge \neg B \Rightarrow Q$

Suponemos $INV \wedge \neg B$ como hipótesis:

$\neg B \equiv \text{numero_actual} > N$

$INV \equiv \text{resultado} = \langle N \mid 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle$
 $\wedge 0 \leq \text{numero_actual} \leq N + 1$

Si supongo estas hipótesis, entonces yo sé que:

1. $\text{numero_actual} > N \equiv \text{True}$
2. $\text{resultado} = \langle N \mid 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle \equiv \text{True}$
3. $0 \leq \text{numero_actual} \leq N + 1 \equiv \text{True}$

Veamos Q:

Q

$\equiv \{\text{def. Q}\}$

resultado = $\langle N \mid 0 \leq i \leq N : i \bmod 6 = 0 \rangle$

$\equiv \{ \text{por hip, sabemos que } \text{numero_actual} > N \text{ y que } \text{numero_actual} \leq N + 1, \text{ luego}$
 $\text{numero_actual} = N + 1, \text{ luego } N = \text{numero_actual} - 1 \}$

resultado = $\langle N \mid 0 \leq i \leq \text{numero_actual} - 1 : i \bmod 6 = 0 \rangle$

$\equiv \{ \text{por hip en INV} \}$

True

Observación: Fijarse que acá usamos todas las hip. salvo $0 \leq \text{numero_actual}$ (esta hace falta para demostrar ii).

Ejemplo: Suma de los elementos de un arreglo.

Const $N : \text{Int}, A : \text{Array}[0, N] \text{ of Int};$

Var $\text{pos}, \text{res} : \text{Int};$

$\{ P: N \geq 0 \}$

$\text{res}, \text{pos} := 0, 0;$

$\{ R: \text{res} = 0 \wedge \text{pos} = 0 \}$

do $\text{pos} < N \rightarrow$

$\{ \text{hasta acá ya sumé todas las posiciones desde 0 hasta pos } \underline{\text{no}} \text{ inclusive} \}$

$\text{res}, \text{pos} := \text{res} + A.\text{pos}, \text{pos} + 1$

od

$\{ Q: \text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle \}$

Invariante: Recordemos que expresa el “resultado intermedio”:

$INV \equiv \text{res} = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle \wedge 0 \leq \text{pos} \leq N$

Lo sacamos “de la galera”. Ahora vamos a demostrar que es correcto:

i) $R \Rightarrow INV$

Suponemos R como hip: $res = 0 \wedge pos = 0$.

Vemos INV:

INV

$\equiv \{ \text{????? (ejercicio)} \}$

True

ii) El invariante se preserva en el cuerpo del ciclo:

$\{ INV \wedge B \} S \{ INV \}$

donde S es el cuerpo del ciclo: $res, pos := res + A[pos], pos + 1$

Demostración: Usamos wp: Probamos que $INV \wedge B \Rightarrow wp.S.INV$.

Suponemos $INV \wedge B$ como hipótesis:

$INV \equiv res = \langle \sum i : 0 \leq i < pos : A[i] \rangle$

$\wedge 0 \leq pos \leq N$

$res = A[0] + \dots + A[pos-1]$

$B \equiv pos < N$

Veamos la wp:

$wp.(res, pos := res + A[pos], pos + 1).INV$

$\equiv \{ \text{def. wp para } := \}$

$res + A[pos] = \langle \sum i : 0 \leq i < pos + 1 : A[i] \rangle \wedge 0 \leq pos + 1 \leq N$

/// $A[0] + \dots + A[pos-1] + A[pos]$

$\equiv \{ \text{lógica y partición de rango} \}$

$res + A[pos] = \langle \sum i : 0 \leq i < pos : A[i] \rangle + \langle \sum i : i = pos : A[i] \rangle \wedge 0 \leq pos + 1 \leq N$

$\equiv \{ \text{rango unitario} \}$

$res + A[pos] = \langle \sum i : 0 \leq i < pos : A[i] \rangle + A[pos] \wedge 0 \leq pos + 1 \leq N$

$\equiv \{ \text{hip. me dice que esa sumatoria es res} \}$

$res + A[pos] = res + A[pos] \wedge 0 \leq pos + 1 \leq N$

$\equiv \{ \text{reflexividad} \}$

True $\wedge 0 \leq pos + 1 \leq N$

$\equiv \{ \text{neutro, y lógica} \}$

$0 \leq pos + 1 \wedge pos + 1 \leq N$

$\equiv \{ 0 \leq pos + 1 \text{ vale por hip } pos \geq 0. \}$

$pos + 1 \leq N$

$\equiv \{ pos + 1 \leq N \text{ vale por hip B: } pos < N \}$

True

Observación: en este punto usamos todas las hipótesis salvo $pos \leq N$ (se va a usar en iii).

iii) Al terminar, el invariante garantiza la post: $INV \wedge \neg B \Rightarrow Q$

Suponemos $INV \wedge \neg B$ como hipótesis y partimos de Q para llegar a True.

(ejercicio.)

La repetición en derivaciones

Tengo la terna:

{ P } ← **precondición dada**
S ← **el programa que tengo que encontrar**
{ Q } ← **postcondición dada**

¿Puede ser que **S** sea un ciclo, o deba contener uno? Podemos saber que **S** debe tener un ciclo si vemos que en la postcondición el estado tiene que tener calculada una expresión cuantificada con una cantidad indeterminada de términos que no estaba ya previamente calculada en la precondición.

Observación: una reflexión parecida hacíamos en funcional al decidir si necesitamos inducción / recursión.

Ejemplo: Acá sí hace falta un ciclo:

{ P: $N \geq 0$ **}**
S
{ Q: $\text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle$ **}**

Ejemplo: Acá no, porque en la precondición ya tengo resuelta gran parte de la sumatoria:

{ P: $N > 0 \wedge \text{res} = \langle \sum i : 0 \leq i < N-1 : A.i \rangle$ **}** // acá tengo sumado todo menos el $A.(N-1)$
S
{ Q: $\text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle$ **}** // acá tengo sumado todo

Acá alcanza con que **S** sea el siguiente programa:

$\text{res} := \text{res} + A.(N-1)$

Otro:

{ P: $N > 1 \wedge \text{res} = \langle \sum i : 0 \leq i < N-2 : A.i \rangle$ **}** // acá tengo sumado todo menos el $A.(N-2)$
S
{ Q: $\text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle$ **}** // acá tengo sumado todo

S es $\text{res} := \text{res} + A.(N-2) + A.(N-1)$

Otro:

{ P: $N > 1 \wedge M \leq N \wedge \text{res} = \langle \sum i : 0 \leq i < N - M : A.i \rangle$ **}**
// acá tengo sumado todo menos $A.(N-M) + \dots + A.(N-1)$, falta sumar M términos
S
{ Q: $\text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle$ **}** // acá tengo sumado todo

Sí hace falta un ciclo.

Derivación de ciclos: Técnicas para encontrar invariantes

Idea general

Tengo la terna:

{ P } ← **precondición dada**
S ← **el programa que tengo que encontrar**
{ Q } ← **postcondición dada**

Una vez que nos damos cuenta de que necesitamos un ciclo, debemos proponerlo y proponer su invariante (y la guarda). Para ello usaremos **técnicas para encontrar invariantes**. La técnica que use me va a dar un invariante INV y una guarda B (ya veremos cómo) tal que se garantiza el **requisito iii: $INV \wedge \neg B \Rightarrow Q$** .

Una vez que tengo el invariante y la guarda, tengo que proponer cómo va a quedar mi programa S.

Siempre lo vamos a proponer de la siguiente manera:

S va a ser una secuenciación:

- 1) una inicialización S1
- 2) ciclo de la forma: **do** B → S2 **od**.

O sea, S sería **S1 ; do B → S2 od** .

O sea tendremos el siguiente programa anotado:

```
{ P }  
  S1 ; // inicialización  
{ INV }  
  do B →  
    { INV ∧ B }  
    S2           // cuerpo del ciclo  
    { INV }  
  od  
{ Q }
```

Ahora tengo planteadas dos ternas de Hoare nuevas (dos subproblemas nuevos a resolver):

- 1) Inicialización (garantiza requisito i):
 { P }
 S1
 { INV }
- 2) Cuerpo del ciclo (garantiza requisito ii):
 { INV ∧ B }
 S2
 { INV }

S1 y S2 deben ser derivados usando las técnicas estándar de derivación.

Observación: S2 seguro no va a ser skip porque si no tengo un ciclo que no termina.

1ra técnica: Tomar términos de una conjunción

Ejemplo: Algoritmo de la división: “Dados dos números x e y , con $x \geq 0$ e $y > 0$, calcular cociente y resto de la división entera de x por y .” (No se puede usar div ni mod ni división real)

```
Const X, Y : Int ;
Var q, r : Int ;
{ P:  $X \geq 0 \wedge Y > 0$  }
S
{ Q:  $X = q * Y + r \wedge 0 \leq r \wedge r < Y$  }
       $Q_1 \quad \wedge \quad Q_2 \quad \wedge \quad Q_3$ 
```

Como todas las técnicas, esta se deriva del requisito iii):

$$\text{iii)} \quad \text{INV} \wedge \neg B \Rightarrow Q$$

Ahora si tengo que Q es una conjunción de varias cosas:

$$Q_1 \wedge Q_2 \wedge Q_3$$

Puedo elegir que una parte sea el INV y que otra parte sea $\neg B$. En ese caso tendría garantizado el requisito iii).

¿Qué me conviene elegir? Esta elección se hace por “creatividad”.

- Q_1 no puede ser $\neg B$ ni a palos, va seguro al invariante.
- Si elijo que Q_2 sea $\neg B$, me queda $B \equiv r < 0$. Voy a estar “ciclando” si mi resto es negativo. Raro.
- Si elijo que Q_3 sea $\neg B$, me queda $B \equiv r \geq Y$. Voy a estar “ciclando” si $r \geq Y$.

Capaz! Suena mejor que Q_2 . Probemos a ver si anda.

Elegimos:

$$\begin{aligned} \text{INV} &\equiv Q_1 \wedge Q_2 \equiv X = q * Y + r \wedge 0 \leq r \\ B &\equiv \neg Q_3 \equiv r \geq Y \end{aligned}$$

$$\text{¿Vale el requisito iii)?} \quad \text{INV} \wedge \neg B \Rightarrow Q$$

Obvio que vale, de hecho son equivalentes:

$$(Q_1 \wedge Q_2) \wedge \neg(\neg Q_3) \Rightarrow Q_1 \wedge Q_2 \wedge Q_3$$

Planteamos/refinamos el programa S:

```
Const X, Y : Int ;
Var q, r : Int ;
{ P:  $X \geq 0 \wedge Y > 0$  }
```

```

S1 ;           // inicialización
{ INV }
do r ≥ Y →
    { INV ∧ B }
    S2           // cuerpo del ciclo
    { INV }
od
{ Q: X = q * Y + r ∧ 0 ≤ r ∧ r < Y }

```

¿Qué falta para terminar de resolver el problema?

1) Inicialización: Encontrar S_1 tal que vale la terna:

```

{ P: X ≥ 0 ∧ Y > 0 }
S1 ;           // inicialización
{ INV: X = q * Y + r ∧ 0 ≤ r }

```

Skip seguro que no es. Asignación? Puede ser, la planteamos con incógnitas:

$q, r := E, F$

Derivamos la asignación: Usando wp, tengo que ver que $P \Rightarrow wp.S_1.INV$.

Supongamos P y veamos la wp:

```

wp.S1.INV
≡ { def. wp para := }
X = E * Y + X ∧ 0 ≤ F
≡ { elijo E = 1, F = X - Y, en ese caso vale la primera parte pero no vale 0 ≤ F
    elijo E = 0, F = X }
X = 0 * Y + X ∧ 0 ≤ X
≡ { arit. }
X = X ∧ 0 ≤ X
≡ { lógica e hip. }
True

```

Listo!! S1 es

$q, r := 0, X.$

2) Cuerpo del ciclo: Encontrar S_2 tal que vale la terna:

```

{ INV ∧ B : X = q * Y + r ∧ 0 ≤ r ∧ r ≥ Y }
S2           // cuerpo del ciclo
{ INV : X = q * Y + r ∧ 0 ≤ r }

```

¿Puede S_2 ser skip? **Nunca.**

¿Puede S_2 ser una asignación? Probemos:

$q, r := \underline{E}, \underline{F}$

¿Podemos conocer de antemano E o F ?

Queremos que el ciclo termine, la guarda dice $r \geq Y$, necesito que r se achique.

Replanteamos:

$q, r := \underline{E}, r - \underline{F}$

(ya prevemos que F es Y, pero tratemos de verlo en la derivación)

Derivamos: Suponemos la precondition:

$$\text{INV} \wedge B : X = q * Y + r \wedge 0 \leq r \wedge r \geq Y$$

Y vemos la wp:

wp.S₂.INV

$\equiv \{ \text{def. wp para } := \}$

$$X = \underline{E} * Y + (r - \underline{E}) \wedge \underline{0 \leq r - E}$$

$\equiv \{ \text{arit.} \}$

$$X = \underline{E} * Y + (r - \underline{E}) \wedge \underline{E \leq r}$$

$\equiv \{ \text{por la hip. } r \geq Y, \text{ pruebo eligiendo } \underline{E = Y} \}$

$$X = \underline{E} * Y + (r - Y) \wedge Y \leq r$$

$\equiv \{ \text{uso la hip.} \}$

$$X = \underline{E} * Y + (r - Y)$$

$\equiv \{ \text{despejo E} \}$

$$E = (X - r + Y) / Y \text{ (por acá no es porque no se puede usar la division)}$$

$\equiv \{ \text{reemplazo X por hip.} \}$

$$q * Y + r = \underline{E} * Y + (r - Y)$$

$\equiv \{ \text{despejo E de acá. paso 1} \}$

$$q * Y + r - (r - Y) = \underline{E} * Y$$

$\equiv \{ \text{despejo E de acá. paso 2} \}$

$$q * Y + Y = \underline{E} * Y$$

$\equiv \{ \text{despejo E de acá. paso 3} \}$

$$(q + 1) * Y = \underline{E} * Y$$

$\equiv \{ \text{despejo E de acá. paso 4} \}$

$$q + 1 = \underline{E}$$

$\equiv \{ \text{elijo } \underline{E = q + 1} \}$

True

Listo!! S₂ es $q, r := q + 1, r - Y$

Resultado final:

Const X, Y : Int ;

Var q, r : Int ;

{ P: $X \geq 0 \wedge Y > 0$ }

q, r := 0, X ;

do $r \geq Y \rightarrow$

 q, r := q + 1, r - Y

od

{ Q: $X = q * Y + r \wedge 0 \leq r \wedge r < Y$ }

Testing: Ejecutar a mano un ejemplo: X = 19, Y = 5. (q = 3, r = 4.)

iteración	sentencia	q	r	guarda ($r \geq Y$)
inicialización	q, r := 0, X	0	19	True

1	$q, r := q + 1, r - Y$	1	14	True
2	$q, r := q + 1, r - Y$	2	9	True
3	$q, r := q + 1, r - Y$	3	4	False

Luego, al dividir 19 por 5, el cociente es 3 y el resto es 4.

Otro ejemplo: Búsqueda lineal (p. 274 del libro)

2da técnica: Reemplazo de constantes por variables

La técnica más usada por lejos.

Ejemplo: Suma de los elementos de un arreglo:

```

Const N : Int, A : Array[0, N) of Int;
Var res : Int;
{ P: N ≥ 0 }
S
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }

```

Recordemos que necesitamos un ciclo ya que tenemos que calcular una expresión cuantificada con una cantidad indeterminada de términos:

$$\text{res} = \underline{A.0 + A.1 + \dots + A.(N-1)}$$

Acá nos gustaría tener control de la cantidad de términos calculados. Para ello lo que haremos es crear una nueva variable y reemplazar alguna de las constantes que determina la cantidad de términos por esta variable nueva.

En ejemplo:

$$Q \equiv \text{res} = \langle \sum i : 0 \leq i < \underline{N} : A.i \rangle$$

Probemos reemplazando la constante N por una nueva variable pos. Haciendo este reemplazo, el invariante que proponemos es:

$$\text{INV} \equiv \text{res} = \langle \sum i : 0 \leq i < N : A.i \rangle$$

Además, queremos que INV esté bien definido, y para eso necesitamos fortalecerlo con un predicado que restrinja los valores posibles para pos:

$$\text{INV} \equiv \text{res} = \langle \sum i : 0 \leq i < \text{pos} : A.i \rangle \wedge 0 \leq \text{pos} \leq N$$

(este es el invariante que habíamos dado ya)

(Obs: si $\text{pos} > N$, la sumatoria no estaría bien definida porque excede los límites del arreglo)

¡Falta la guarda B! Queremos que sea tal que vale el requisito iii):

$$\begin{array}{ccc} \text{INV} & & \wedge \neg B \Rightarrow Q \\ \text{res} = \langle \sum_{i=0}^{\text{pos}-1} A[i] \rangle \wedge 0 \leq \text{pos} \leq N & \wedge \neg B \Rightarrow & \text{res} = \langle \sum_{i=0}^{N-1} A[i] \rangle \\ 1 & 2 & 3 \end{array}$$

Q es casi la hip (1), sólo me falta que $\text{pos} = N$.

Entonces si elijo $\neg B \equiv \text{pos} = N$, ya estaría.

También funciona $\neg B \equiv \text{pos} \geq N$, ya que también tengo la hip. $\text{pos} \leq N$ (y luego $\text{pos} = N$)

Luego sirven ambas guardas:

- $B \equiv \text{pos} \neq N$
- $B \equiv \text{pos} < N$

Elegimos cualquiera. Por ejemplo la segunda: $\text{pos} < N$.

Planteamos/refinamos el programa S:

Const N : Int, A : Array[0, N) of Int;

Var res, pos : Int;

{ P: $N \geq 0$ }

S₁; // inicialización

{ INV }

do $\text{pos} < N \rightarrow$

{ INV $\wedge$ B }

S₂ // cuerpo del ciclo

{ INV }

od

{ Q: $\text{res} = \langle \sum_{i=0}^{N-1} A[i] \rangle$ }

Falta encontrar S₁ y S₂.

Inicialización: Encontrar S₁ tal que vale la terna:

{ P: $N \geq 0$ }

S₁; // inicialización

{ INV: $\text{res} = \langle \sum_{i=0}^{\text{pos}-1} A[i] \rangle \wedge 0 \leq \text{pos} \leq N$ }

Probamos con una asignación: $\text{res}, \text{pos} := E, F$.

Derivamos: Supongamos la precondition P: $N \geq 0$, y veamos la wp:

$$\begin{aligned} & \text{wp.S}_1.\text{INV} \\ & \equiv \{ \text{def. wp para } := \} \\ & E = \langle \sum_{i=0}^{F-1} A[i] \rangle \wedge 0 \leq F \leq N \\ & \equiv \{ \text{elijo } \underline{F=0}, \underline{E=0} \} \\ & 0 = \langle \sum_{i=0}^{-1} A[i] \rangle \wedge 0 \leq 0 \leq N \\ & \equiv \{ \text{pasos varios ...} \} \\ & \text{True} \end{aligned}$$

Cuerpo del ciclo: Encontrar S_2 tal que vale la terna:

$\{ INV \wedge B : res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos \leq N \wedge pos < N \}$
 S_2 // cuerpo del ciclo
 $\{ INV : res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos \leq N \}$

¿Qué es S_2 ? Probamos con una asignación:

$res, pos := E, F$

¿Puedo conocer de antemano alguna de las incógnitas? Está claro que $F = pos + 1$.

Replanteo:

$res, pos := E, pos + 1$

Derivación: Queda como ejercicio descubrir que me conviene elegir $E = res + A.pos$.

Resultado final:

```
Const N : Int, A : Array[0, N) of Int;  
Var res, pos : Int;  
{ P: N ≥ 0 }  
  res, pos := 0, 0 ; // inicialización  
  do pos < N →  
    res, pos := res + A[pos], pos + 1 // cuerpo del ciclo  
  od  
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
```

Ejercicio: Hacer testing de este programa.

Otro ejemplo: Exponenciación: Dados $X > 0$, e $Y \geq 0$, calcular X^Y .

```
Const X, Y : Int ;  
Var res : Int ;  
{ P : X > 0 ∧ Y ≥ 0 }  
  S  
{ Q: res =  $X^Y$  }
```

Acá en la postcondición tengo una multiplicación de una cantidad indeterminada de términos:

$res = X * X * \dots * X$ Y veces

Claramente necesitamos un ciclo. Cómo aplico reemplazo de constante por variable?

Reemplazamos Y por una variable nueva n.

Planteamos el invariable fortalecido:

$INV \equiv res = X^n \wedge 0 \leq n \leq Y$

$$B \equiv n \neq Y$$

De esta manera vale el requisito iii): $INV \wedge \neg B \Rightarrow Q$

Planteamos el programa como va quedando:

```

Const X, Y : Int ;
Var res , n : Int ;
{ P : X > 0  $\wedge$  Y  $\geq$  0 }
S1 ;
{ INV }
do n  $\neq$  Y  $\rightarrow$ 
    { INV  $\wedge$  B }
    S2
    { INV }
od
{ Q: res = XY }

```

Falta derivar S₁ (inicialización) y S₂ (cuerpo del ciclo).

Inicialización:

```

{ P : X > 0  $\wedge$  Y  $\geq$  0 }
S1 ;
{ INV: res = Xn  $\wedge$  0  $\leq$  n  $\leq$  Y }

```

¿Qué es S1? res, n := 1, 0 (**ejercicio: derivar/demostrar**)

Cuerpo del ciclo:

```

{ INV  $\wedge$  B: res = Xn  $\wedge$  0  $\leq$  n  $\leq$  Y  $\wedge$  n  $\neq$  Y }
S2
{ INV: res = Xn  $\wedge$  0  $\leq$  n  $\leq$  Y }

```

¿Qué es S2? Se puede derivar partiendo de: res, n := E, n+1

Resultado: res, n := res * X, n+1

Resultado final:

```

Const X, Y : Int ;
Var res , n : Int ;
{ P : X > 0  $\wedge$  Y  $\geq$  0 }
res, n := 1, 0 ;
do n  $\neq$  Y  $\rightarrow$ 
    res, n := res * X, n + 1
od
{ Q: res = XY }

```

Ejercicio: Hacer TESTING!!

Ejemplo: Factorial de un número N: Dado $N \geq 0$ quiero calcular el factorial de N.

```
Const N : Int;  
Var res : Int;  
{ P:  $N \geq 0$  }  
  S  
{ Q:  $res = N!$  }
```

¿Nos saldrá este programa a ojo? Queremos calcular:

$$1 * 2 * 3 * 4 * \dots * (N-1) * N$$

La idea es hacer un ciclo que recorra los números desde 1 hasta el N (en una variable n), e ir multiplicando en res esos números.

```
Const N : Int;  
Var res , n : Int;  
{ P:  $N \geq 0$  }  
  res , n := 1 , 1 ;  
  do  $n \leq N \rightarrow$   
    res , n := res * n , n + 1  
  od  
{ Q:  $res = N!$  }
```

La otra forma a ojo, recorriendo al revés:

```
Const N : Int;  
Var res , n : Int;  
{ P:  $N \geq 0$  }  
  res , n := 1 , N ;  
  do  $n \geq 1 \rightarrow$   
    res , n := res * n , n - 1  
  od  
{ Q:  $res = N!$  }
```

Ahora probemos derivando:

Usando la técnica de reemplazo de constante por variable, creamos una nueva variable n : Int, y proponemos:

$$INV \equiv res = n! \wedge 0 \leq n \leq N$$

$$B \equiv n \neq N \quad (\text{también andaría } n < N)$$

Replanteamos el programa:

```
Const N : Int;  
Var res : Int;  
{ P:  $N \geq 0$  }  
  S1 ;  
{ INV }  
  do  $n \neq N \rightarrow$   
    { INV  $\wedge$  B }
```

```

    S2
    { INV }
  od
{ Q: res = N! }

```

Inicialización:

```

{ P: N ≥ 0 }
S1 ;
{ INV: res = n! ∧ 0 ≤ E ≤ N }

```

S1 debe ser de la forma: $res, n := E, F$
 (F no puede ser 1 porque no puedo saber $1 \leq N$)
 Elijo **F = 0, E = 1** y sale todo bien (ejercicio: verificarlo con la wp)

Cuerpo del ciclo:

```

{ INV ∧ B: res = n! ∧ 0 ≤ n ≤ N ∧ n ≠ N }
S2
{ INV: res = n! ∧ 0 ≤ n ≤ N }

```

S2 debe ser de la forma: $res, n := E, n + 1$.

Derivemos: Supongamos como hipótesis **INV ∧ B**, y veamos la wp:

```

wp.(res, n := E, n + 1).INV
≡ { def. wp para := }
  E = (n+1)! ∧ 0 ≤ n+1 ≤ N
≡ { algebra (prop !) }
  E = n! * (n + 1) ∧ 0 ≤ n+1 ≤ N
≡ { hip. INV } ← FUNDAMENTAL A LA HORA DE DERIVAR UN CUERPO DE CICLO
  E = res * (n + 1) ∧ 0 ≤ n+1 ≤ N
≡ { elijo E = res * (n+1) }
  res * (n+1) = res * (n + 1) ∧ 0 ≤ n+1 ≤ N
≡ { lógica }
  0 ≤ n+1 ≤ N
≡ { 0 ≤ n+1 vale por hip. 0 ≤ n, n+1 ≤ N vale por hip n ≤ N ∧ n ≠ N }
  True

```

Listo! Resultado final:

```

Const N : Int;
Var res : Int;
{ P: N ≥ 0 }
  res, n := 1, 0 ;
  do n ≠ N →
    res, n := res * (n+1), n + 1
  od
{ Q: res = N! }

```

Es muy parecido pero no igual al que hicimos a ojo. Ambos andan bien igual.

Ejemplo: De nuevo suma de los elementos de un arreglo:

```
Const N : Int, A : Array[0, N) of Int;  
Var res : Int;  
{ P: N ≥ 0 }  
  S  
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle$ }
```

Esta vez derivamos distinto. Aplicamos cambio de constante por variable de la siguiente manera:

$$\text{INV} \equiv \text{res} = \langle \sum i : n \leq i < N : A.i \rangle \wedge 0 \leq n \leq N$$
$$B \equiv \underline{n \neq 0}$$

(acá reemplazamos la constante 0 por la variable nueva n)

Luego vale el requisito iii): $\text{INV} \wedge \neg B \Rightarrow \underline{Q}$

Replanteamos el programa:

```
Const N : Int, A : Array[0, N) of Int;  
Var res , n : Int;  
{ P: N ≥ 0 }  
  S1 ;  
{ INV }  
  do n ≠ 0 →  
    { INV ∧ B }  
    S2  
    { INV }  
  od  
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle$ }
```

Inicialización:

```
{ P: N ≥ 0 }  
  S1 ;  
{ INV: res =  $\langle \sum i : n \leq i < N : A.i \rangle \wedge 0 \leq n \leq N$  }
```

¿Qué es S1? De la forma: $\text{res}, n := E, F$.

Derivemos: Supongamos P como hip. y veamos la wp:

```
wp.(res, n := E, F).INV  
≡ { def wp para := }  
  E =  $\langle \sum i : F \leq i < N : A.i \rangle \wedge 0 \leq F \leq N$   
≡ { elijo F = N para forzar un rango vacío }  
  E =  $\langle \sum i : N \leq i < N : A.i \rangle \wedge 0 \leq N \leq N$ 
```

$\equiv \{ \text{elijo } E = 0 \text{ y hago más pasos} \}$
 True

Cuerpo del ciclo:

$\{ \text{INV} \wedge B: \text{res} = \langle \sum i: n \leq i < N: A.i \rangle \wedge 0 \leq n \leq N \wedge n \neq N \}$
 S2
 $\{ \text{INV}: \text{res} = \langle \sum i: n \leq i < N: A.i \rangle \wedge 0 \leq n \leq N \}$

Por intuición sabemos que S2 será de la forma: $\text{res}, n := E, n - 1$
 Derivemos: Supongamos como hip. $\text{INV} \wedge B$ y veamos la wp:

$\text{wp}(\text{res}, n := E, n - 1). \text{INV}$
 $\equiv \{ \text{def wp para } := \}$
 $E = \langle \sum i: n - 1 \leq i < N: A.i \rangle \wedge 0 \leq n - 1 \leq N$
 $\equiv \{ n - 1 \leq N \text{ vale ya que } n \leq N, 0 \leq n - 1 \text{ vale ya que por hip } n \geq 0 \text{ y además } n \neq 0$
 (o sea $n \geq 1$). }
 $E = \langle \sum i: n - 1 \leq i < N: A.i \rangle$
 $\equiv \{ \text{reescribimos rango por lógica} \}$

Ayudita: $n = 4, N = 8$: tenemos: $n - 1 \leq i < N: i \in \{3, 4, 5, 6, 7\}$
 queremos: $n \leq i < N: i \in \{4, 5, 6, 7\}$
 podemos aplicar esto: $n - 1 \leq i < N \equiv i = n - 1 \vee n \leq i < N$

$E = \langle \sum i: i = n - 1 \vee n \leq i < N: A.i \rangle$
 $\equiv \{ \text{part. rango} \}$
 $E = \langle \sum i: n \leq i < N: A.i \rangle + \langle \sum i: i = n - 1: A.i \rangle$
 $\equiv \{ \text{hip.} \}$
 $E = \text{res} + \langle \sum i: i = n - 1: A.i \rangle$
 $\equiv \{ \text{rango unitario} \}$
 $E = \text{res} + A.(n - 1)$
 $\equiv \{ \text{elijo } E = \text{res} + A.(n - 1) \}$
 True

Listo! Resultado final:

```
Const N : Int, A : Array[0, N) of Int;
Var res, n : Int;
  res, n := 0, N;
  do n ≠ 0 →
    res, n := res + A.(n - 1), n - 1
  od
{ Q: res = ⟨ ∑ i : 0 ≤ i < N : A.i ⟩ }
```

Este programa recorre el arreglo desde el final hacia adelante.

Ejemplo: Promedio de los elementos de un arreglo:

```
Const N : Int, A : Array[0, N) of Int;
```

```

Var res : Float;
{ P: N > 0 }
S
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle / N$  }

```

¿Cómo derivamos acá?

Opción 1: Algo que ya hicimos: Plantear una secuenciación de dos programas: primero calcular la suma y después dividir por N:

```

Const N : Int, A : Array[0, N) of Int;
Var res, sum : Float, Int ;
{ P: N > 0 }
S1 ; // esto es el programa que ya vimos (el que suma un arreglo)
{ R: sum =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
S2 // esto es una asignación res := sum / N
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle / N$  }

```

Opción 2: No planteamos la secuenciación y vamos derecho al ciclo usando reemplazo de constante por variable.

$$\begin{aligned} \text{INV} &\equiv \text{res} = \langle \sum i : 0 \leq i < n : A.i \rangle / N \quad \wedge \quad 0 \leq n \leq N \\ \text{B} &\equiv n \neq N \end{aligned}$$

Observación importante: Al hacer cambio de constante por variable, sólo se hace el cambio en el punto que me permite controlar la cantidad de términos de mi cuantificación (o sea, el rango). En el ejemplo, la otra mención de la constante N no debe ser reemplazada.

Replanteamos el programa:

```

Const N : Int, A : Array[0, N) of Int;
Var res : Float;
{ P: N > 0 }
S1 ; // res, n := 0, 0
{ INV: res =  $\langle \sum i : 0 \leq i < n : A.i \rangle / N \quad \wedge \quad 0 \leq n \leq N$  }
do n ≠ N →
  { INV ∧ B }
  S2 // res, n := res + A.n / N , n+1
  { INV }
od
{ Q: res =  $\langle \sum i : 0 \leq i < N : A.i \rangle / N$  }

```

Ejercicio: Derivar bien este programa.

Ciclos anidados

Ejemplo

Ejemplo (2025, practico 4, ejercicio 11): Veamos esta especificación:

```
Const N, A : array[0,N) of Int ;
Var res : Bool ;
{ P:  $N \geq 0$  }
S
{ Q:  $res = \langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }
```

¿Qué calcula este programa? “el programa verifica si cada elemento del arreglo es el factorial de su posición” (esto se calcula en la variable booleana r).

· · ¿Sale a ojo este programa? Necesitamos seguro un ciclo para recorrer los elementos del arreglo. Y una variable “pos” para las posiciones del arreglo.

```
Const N, A : array[0,N) of Int ;
Var res : Bool , fac : Int ;
{ P:  $N \geq 0$  }
res , pos := True , 0 ;
do pos < N → // también sirve: “pos < N  $\wedge$  res” (pero ya lo veremos)
    // acá quiero ver si A.pos = pos !
    res , pos := res  $\wedge$  (A.pos = pos ! ) , pos + 1 // casi!! esto estaría bien si el
                                                    // factorial fuera programable.
    // tengo que calcular acá el factorial de pos: pos !
    // usemos una variable nueva fac y planteamos la especificación del problema:
    T ; // este mini-programa T debe calcular el factorial de pos
    { fac = pos ! } // postcondición para T
    res , pos := res  $\wedge$  (A.pos = fac ) , pos + 1
od
{ Q:  $res = \langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }
```

Como el factorial no es programable, necesitamos una variable extra fac, y calcular fac = pos ! dentro del ciclo, en un nuevo ciclo:

```
Const N : Int, A : array[0, N) of Int ;
Var res : Bool,
    pos, fac, n : Int ;
{ P:  $N \geq 0$  }
res, pos := True , 0 ;
do pos < N →
    // acá calcular fac = pos !
    fac, n := 1, 0 ;
    do n  $\neq$  pos →
        fac, n := fac * (n+1) , n + 1
```



```

    od ;
    { fac = pos ! }
    res, pos := res  $\wedge$  (A.pos = fac) , pos + 1
od
{ Q: res =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Listo el programa a ojo!

A ver qué sale derivando:

```

Const N, A : array[0,N) of Int ;
Var res : Bool ;
{ P: N  $\geq$  0 }
S
{ Q: res =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Necesitamos un ciclo sí o sí. Aplicamos cambio de constante por variable. Reemplazamos la constante N por una nueva variable pos:

$$\begin{aligned} \text{INV} &\equiv \text{res} = \langle \forall i : 0 \leq i < \text{pos} : A.i = i! \rangle \wedge 0 \leq \text{pos} \leq N \\ B &\equiv \text{pos} \neq N \end{aligned}$$

Luego, vale $\text{INV} \wedge \neg B \Rightarrow Q$.

Replanteamos/refinamos:

```

Const N, A : array[0,N) of Int ;
Var res : Bool ;
{ P: N  $\geq$  0 }
S1 ;
{ INV }
do pos  $\neq$  N  $\rightarrow$ 
    { INV  $\wedge$  B }
    S2
    { INV }
od
{ Q: res =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Inicialización: S1 es res, pos := True, 0. (ejercicio)

Cuerpo del ciclo: Tenemos la siguiente terna:

```

{ INV  $\wedge$  B: res =  $\langle \forall i : 0 \leq i < \text{pos} : A.i = i! \rangle \wedge 0 \leq \text{pos} \leq N \wedge \text{pos} \neq N$  }
S2
{ INV: res =  $\langle \forall i : 0 \leq i < \text{pos} : A.i = i! \rangle \wedge 0 \leq \text{pos} \leq N$  }

```

Probemos con S2 de la forma: res, pos := E, pos + 1

Derivemos: Supongamos $\text{INV} \wedge B$ y veamos la wp:

$$\begin{aligned}
& wp.(res, pos := E, pos + 1).INV \\
& \equiv \{ \text{def. wp para } := \} \\
& E = \langle \forall i : 0 \leq i < pos + 1 : A.i = i! \rangle \wedge \underline{0 \leq pos + 1 \leq N} \\
& \equiv \{ \text{esta parte vale por hip } 0 \leq pos \leq N \wedge pos \neq N \text{ (ya lo vimos varias veces)} \} \\
& E = \langle \forall i : 0 \leq i < pos + 1 : A.i = i! \rangle \\
& \equiv \{ \text{lógica y partición de rango} \} \\
& E = \underline{\langle \forall i : 0 \leq i < pos : A.i = i! \rangle} \wedge \langle \forall i : i = pos : A.i = i! \rangle \\
& \equiv \{ \text{hip.} \} \\
& E = res \wedge \langle \forall i : i = pos : A.i = i! \rangle \\
& \equiv \{ \text{rango unitario} \} \\
& E = res \wedge A.pos = pos! \\
& \equiv \{ \text{si ! fuera programable, acá podría elegir } E = res \wedge A.pos = pos! \}
\end{aligned}$$

Acá nos trabamos, porque “pos !” no es programable. Nos hubiera venido bien tener una hipótesis adicional que me diga que hay una variable (llamémosla “fac”) en la que está calculado “pos !”.

Una cosa así:

```

{ INV ∧ B ∧ fac = pos ! }
  res, pos := E, pos + 1
{ INV }

```

Para esto, necesitaría tener un programa antes S3 cuya postcondición es “INV ∧ B ∧ fac = pos !”.

Antes teníamos:

```

{ INV ∧ B }
  S2
{ INV }

```

Ahora lo replanteamos/refinamos de la siguiente manera:

```

{ INV ∧ B }
  S3;
{ INV ∧ B ∧ fac = pos ! }
  res, pos := E, pos + 1
{ INV }

```

(acá S2 es “S3 ; res, pos := E, pos + 1”, y no cambiamos ni pre ni post)

Ahora tenemos dos nuevos problemas:

Cierre del cuerpo del ciclo: res, pos := E, pos + 1

(son los mismos pasos que hicimos hasta trabarnos, pero nos destrabamos con la nueva hipótesis y terminamos eligiendo $E = res \wedge (A.pos = fac)$)

Ciclo anidado: S3, con la terna:

$$\{ P_2: INV \wedge B \}$$

$$S_3;$$

$$\{ Q_2: INV \wedge B \wedge fac = pos! \}$$

¿Qué es S3? Necesito un ciclo para calcular $pos! = 1 * 2 * 3 * \dots * (pos - 1) * pos$.

Observaciones:

- Acá, $INV \wedge B$ valen al principio y deben valer al final. No debo tocar en S3 ninguna de las variables involucradas en $INV \wedge B$ (pos, res).
- O sea, a los ojos del programa S3, **pos y res pueden ser vistas como constantes.**

Para derivar este nuevo ciclo, aplicamos la técnica de cambio de constante por variable, cambiando la “constante” pos por una nueva variable n:

$$INV_2 \equiv INV \wedge B \wedge fac = n! \wedge 0 \leq n \leq pos$$

$$B_2 \equiv n \neq pos$$

Luego, vale el requisito iii): $INV_2 \wedge B_2 \Rightarrow Q_2$

Replanteamos/refinamos S3:

$$\{ P_2: INV \wedge B \}$$

$$S_4;$$

$$\{ INV_2 \}$$

$$\underline{\text{do}} \ n \neq pos \rightarrow$$

$$\quad \{ INV_2 \wedge B_2 \}$$

$$\quad S_5$$

$$\quad \{ INV_2 \}$$

$$\underline{\text{od}}$$

$$\{ Q_2: INV \wedge B \wedge fac = pos! \}$$

Ahora tenemos dos nuevos subproblemas:

Inicialización 2: La terna es:

$$\{ P_2: INV \wedge B \}$$

$$S_4;$$

$$\{ INV_2 \}$$

S₄ va a ser de la forma $fac, n := E, F$
(recordemos: no tocamos ni pos ni res.)

Ya se que **E = 1, y F = 0**. Demostremos en lugar de derivar:
Supongamos la pre P₂ y veamos la wp:

$$wp.(fac, n := 1, 0).INV_2$$

$$\equiv \{ \text{def. wp para } := \}$$

$$\begin{aligned}
& \underline{INV \wedge B} \wedge 1 = 0! \wedge 0 \leq 0 \leq pos \\
& \equiv \{ INV \wedge B \text{ ya valen por hip. } P_2 \} \\
& 1 = 0! \wedge 0 \leq 0 \leq pos \\
& \equiv \{ 1 = 0! \text{ por arit. } \} \\
& 0 \leq 0 \leq pos \\
& \equiv \{ pos \geq 0 \text{ por hip } INV \} \\
& \text{True}
\end{aligned}$$

Cuerpo del ciclo 2: La terna es:

$$\begin{aligned}
& \{ INV_2 \wedge B_2 \} \\
& S_5 \\
& \{ INV_2 \}
\end{aligned}$$

S_5 va a ser de la forma: $fac, n := E, n + 1$.

Derivemos: Supongamos $\underline{INV}_2 \wedge B_2$ y veamos la wp:

$$\begin{aligned}
& wp.(fac, n := E, n + 1).INV_2 \\
& \equiv \{ \text{def. wp para } := \} \\
& \underline{INV \wedge B} \wedge E = (n + 1)! \wedge 0 \leq n + 1 \leq pos \\
& \equiv \{ \text{vale por hip } INV_2 \} \\
& E = (n + 1)! \wedge \underline{0 \leq n + 1 \leq pos} \\
& \equiv \{ \text{por hip. } 0 \leq n \leq pos \text{ (parte de } INV_2), n \neq pos \text{ (} B_2 \text{)} \} \\
& E = (n + 1)! \\
& \equiv \{ \text{prop !} \} \\
& E = \underline{n!} * (n + 1) \\
& \equiv \{ \text{hip. } INV_2 \} \\
& E = fac * (n + 1) \\
& \equiv \{ \text{elijo } \underline{E = fac * (n + 1)} \} \\
& \text{True}
\end{aligned}$$

Listo el ciclo anidado, luego el cuerpo del ciclo S2 queda así:

$$\begin{aligned}
& \{ INV \wedge B \} \\
& fac, n := 1, 0; \\
& \underline{do} \ n \neq pos \rightarrow \\
& \quad fac, n := fac * (n+1), n + 1 \\
& \underline{od} \\
& \quad res, pos := E, pos + 1 \\
& \{ INV \}
\end{aligned}$$

Y el programa completo queda así:

```

Const N, A : array[0,N) of Int ;
Var res : Bool, pos, fac, n : Int ;

```

```

{ P:  $N \geq 0$  }
  res, pos := True, 0 ;
  do pos  $\neq$  N  $\rightarrow$ 
    fac, n := 1, 0 ;
    do n  $\neq$  pos  $\rightarrow$ 
      fac, n := fac * (n+1) , n + 1
    od
    res, pos := r  $\wedge$  (A.pos = fac), pos + 1
  od
{ Q: res =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Observación: Quedó igual que el que hicimos a ojo!

Otro ejemplo

Ejemplo: Considere la siguiente especificación:

```

Const N : Int, A : array[0, N) of Int ;
Var r : Int ;
{ P:  $N \geq 0$  }
  S
{ Q: r =  $\langle N i, j : 0 \leq i < j < N : A.i = A.j \rangle$  }

```

¿Qué calcula este programa? Calcula la cantidad de veces que dos posiciones distintas del arreglo contienen el mismo valor.

Ejemplo con testing: $N = 5$. $A = [1, 2, 1, 1, 2]$.

0 1 2 3 4

El resultado va a ser $r = \text{????}$.

Podemos hacer la lectura operacional:

El rango es : $(i, j) \in \{ (0, 1), \underline{(0, 2)}, \underline{(0, 3)}, (0, 4),$
 $(1, 2), (1, 3), \underline{(1, 4)}$
 $\underline{(2, 3)}, (2, 4)$
 $(3, 4) \}$

El resultado va a ser 4.

Derivación: Necesitamos un ciclo. Aplicamos reemplazo de constante por variable.

Reemplazamos la constante N por una nueva variable n:

```

INV  $\equiv r = \langle N i, j : 0 \leq i < j < n : A.i = A.j \rangle \wedge 0 \leq n \leq N$ 
B    $\equiv n \neq N$ 

```

Replanteamos el ciclo:

Const $N : \text{Int}$, $A : \text{array}[0, N) \text{ of } \text{Int}$;

Var $r, n : \text{Int}$;

{ $P: N \geq 0$ }

S1 ;

{ INV }

do $n \neq N \rightarrow$

{ $\text{INV} \wedge B$ }

S2

{ INV }

od

{ $Q: r = \langle N \mid i, j : 0 \leq i < j < N : A.i = A.j \rangle$ }

Inicialización: S1 es $r, n := 0, 0$ (ejercicio)

Cuerpo del ciclo: Probamos que S2 sea la asignación: $r, n := E, n + 1$.

Derivemos: Supongamos $\text{INV} \wedge B$, y veamos la wp:

$\text{wp}.(r, n := E, n + 1).\text{INV}$

$\equiv \{ \text{def. wp para } := \}$

$E = \langle N \mid i, j : 0 \leq i < j < n + 1 : A.i = A.j \rangle \wedge \underline{0 \leq n+1 \leq N}$

$\equiv \{ \text{por hip. (ya sabemos hacerlo)} \}$

$E = \langle N \mid i, j : \underline{0 \leq i < j < n + 1} : A.i = A.j \rangle$

$\equiv \{ \text{lógica:}$

$0 \leq i < j < n + 1$

es lo mismo que :

$0 \leq i < j \quad \wedge \quad j < n + 1$

que es lo mismo que :

$0 \leq i < j \quad \wedge \quad (j < n \vee j = n)$

y distribuimos:

$\underline{(0 \leq i < j \quad \wedge \quad j < n)} \vee (0 \leq i < j \quad \wedge \quad j = n)$

que es lo mismo que:

$(0 \leq i < j < n) \vee (0 \leq i < j \quad \wedge \quad j = n)$

}

$E = \langle N \mid i, j : (0 \leq i < j < n) \vee (0 \leq i < j \quad \wedge \quad j = n) : A.i = A.j \rangle$

$\equiv \{ \text{part. de rango} \}$

$E = \langle \underline{N \mid i, j : 0 \leq i < j < n : A.i = A.j} \rangle + \langle N \mid i, j : 0 \leq i < j \quad \wedge \quad j = n : A.i = A.j \rangle$

$\equiv \{ \text{hip.} \}$

$E = r + \langle N \mid i, j : 0 \leq i < j \quad \wedge \quad j = n : A.i = A.j \rangle$

$\equiv \{ \text{elim. de variable } j \}$

$E = r + \underline{\langle N \mid i : 0 \leq i < n : A.i = A.n \rangle}$

Nos trabamos, no hay forma de elegir E programable, necesito una nueva hip y un ciclo anidado para calcularla. Replanteo S2 de la siguiente manera:

{ $\text{INV} \wedge B$ }

S3 ;

{ $\text{INV} \wedge B \wedge r2 = \langle N \mid i : 0 \leq i < n : A.i = A.n \rangle$ }

$$r, n := r + r2, n + 1$$

$$\{ INV \}$$

(r2 cuenta cuantos elementos anteriores son iguales a A.n)

Falta derivar el ciclo anidado S3:

$$\{ P_2: INV \wedge B \}$$

$$S3;$$

$$\{ Q_2: INV \wedge B \wedge r2 = \langle N \ i: 0 \leq i < n : A.i = A.n \rangle \}$$

Para S3, n y r son constantes. Esto es un ciclo, aplicamos cambio de constante por variable, reemplazando la “constante” n por la variable m:

$$INV_2 \equiv INV \wedge B \wedge r2 = \langle N \ i: 0 \leq i < m : A.i = A.n \rangle \wedge 0 \leq m \leq n$$

$$B_2 \equiv m \neq n$$

Por supuesto vale el requisito iii).

Observación: solo hacer el reemplazo en el rango!!!!

Replanteamos/refinamos S3:

$$\{ P_2 \}$$

$$S4;$$

$$\{ INV_2 \}$$

$$\text{do } m \neq n \rightarrow$$

$$\quad \{ INV_2 \wedge B_2 \}$$

$$\quad S5$$

$$\quad \{ INV_2 \}$$

$$\{ Q_2 \}$$

Inicialización del ciclo anidado: S4 es $r2, m := 0, 0$ (ejercicio: demostrar!!)

Cuerpo del ciclo anidado: S5 es de la forma: $r2, m := E, m + 1$.

Derivemos:

Al derivar me voy a encontrar con un análisis por casos:

- Si $A.m = A.n$: Debo elegir $E = r2 + 1$
- Si $A.m \neq A.n$: Debo elegir $E = r2$.

Con esto estaría todo, y el programa final final quedaría:

```
Const N : Int, A : array[0, N) of Int ;
Var r, n, r2, m : Int ;
{ P: N ≥ 0 }
r, n := 0, 0 ;
do n ≠ N →
  r2, m := 0, 0;
  do m ≠ n →
    if A.m = A.n → r2, m := r2 + 1, m + 1
    [] A.m ≠ A.n → m := m + 1
```

```

    fi
  od
  r, n := r + r2, n + 1
od
{ Q: r =  $\langle N \ i, j : 0 \leq i < j < N : A.i = A.j \rangle$  }

```

Ejercicio: Testear con el arreglo de ejemplo que vimos.

$N = 5$. $A = [1, 2, 1, 1, 2]$.

0 1 2 3 4

Testeamos centrandonos en el if. Cada fila corresponde a una ejecución del if:

vez q paso por el if	n (va de 0 hasta N-1)	m (va de 0 hasta n-1)	r	r2	A.n = A.m?
1	1	0	0	0	False
2	2	0	0	0	True
3	2	1	0	1	False
4	3	0	1	0	
5	3	1			
6	3	2			
7	4	0			
8	...				

Ejercicio: completar!

Fortalecimiento de invariantes

Ejemplo

Ejemplo (2025, practico 4, ejercicio 11): Dado un arreglo $a : \text{array}[0, N)$ of Num con $N \geq 0$ determinar si sus elementos son iguales al factorial de la posición.

```

Const N, A : array[0,N) of Int ;
Var r : Bool ;
{ P:  $N \geq 0$  }
S
{ Q:  $r = \langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Recordemos el algoritmo con ciclos anidados:

```

Const N, A : array[0,N) of Int ;
Var r : Bool, pos, fac, n : Int ;
{ P:  $N \geq 0$  }
  r, pos := True, 0 ;
  do pos  $\neq$  N  $\rightarrow$ 

```

```

    fac, n := 1, 0 ;
    do n ≠ pos →
        fac, n := fac * (n+1) , n + 1
    od ;
    r, pos := r ∧ (A.pos = fac), pos + 1
od
{ Q: r =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Ejemplo: Tengo un arreglo de 1000 elementos. En la ejecución, debo chequear que

$$A.998 = 998! = 1 * 2 * 3 \dots * 998$$

y también debo chequear en la iteración siguiente:

$$A.999 = 999! = 1 * 2 * 3 \dots * 998 * 999$$

¿Podemos hacer esto más eficiente? Sí, podemos recordar el factorial que habíamos calculado en la iteración anterior y sólo actualizarlo con el multiplicando que falta (999 o sea "pos").

Veamos si nos sale a ojo:

```

Const N, A : array[0,N) of Int ;
Var r : Bool, pos, fac, n : Int ;
{ P: N ≥ 0 }
    r, pos, fac := True, 0, 1 ;
    do pos ≠ N →
        fac, n := 1, 0 ;
        do n ≠ pos →
            fac, n := fac * (n+1) , n + 1
        od ;
        r, pos := r ∧ (A.pos = fac), pos + 1 ;
        fac := fac * pos
    od
{ Q: r =  $\langle \forall i : 0 \leq i < N : A.i = i! \rangle$  }

```

Parece que salió. (Faltaría testear: **ejercicio!**)

Ahora probemos derivando: El proceso empieza igual que antes: reemplazo de constante por variable para obtener:

$$\begin{aligned}
 \text{INV} &\equiv r = \langle \forall i : 0 \leq i < \text{pos} : A.i = i! \rangle \wedge 0 \leq \text{pos} \leq N \\
 \text{B} &\equiv \text{pos} \neq N
 \end{aligned}$$

Garantizando el requisito iii): $\text{INV} \wedge \neg \text{B} \Rightarrow \text{Q}$.

Inicialización: igual que antes obtenemos “ $r, pos := \text{True}, 0$ ”.

Cuerpo del ciclo: Los mismos pasos que antes:

{ INV $\wedge$ B }

S_2

{ INV }

Probamos con S_2 siendo “ $r, pos := E, pos + 1$ ” y derivamos:

Suponemos INV $\wedge$ B y vemos la wp:

wp.($r, pos := E, pos + 1$).INV

$\equiv \{ \text{def wp para } := \}$

$E = \langle \forall i : 0 \leq i < pos + 1 : A.i = i! \rangle \wedge 0 \leq pos + 1 \leq N$

$\equiv \{ \text{pasos varios usando varias hip.} \}$

$E = r \wedge (A.pos = pos !)$

Acá nos trabamos. Necesitamos una hipótesis adicional. Que una nueva variable es igual a la parte que no es programable:

$fac = pos !$

- **Antes:** lo habíamos resuelto replanteando S_2 como una secuenciación de un ciclo con la asignación, y el ciclo me garantiza esta hipótesis que necesito.
- **Ahora:** Vamos a probar si funciona directamente agregar al invariante que ya tenía la nueva hipótesis que necesito:

Vamos a probar fortaleciendo el invariante:

INV' $\equiv$ INV $\wedge$ fac = pos !

Este invariante reemplaza al original por lo que debo replantear todo el programa de nuevo: (el replanteo incluye declarar la nueva variable fac y reemplazar todo INV por INV').

Const N, A : array[0,N) of Int ;

Var r : Bool, pos, fac ; Int ;

{ P: $N \geq 0$ }

S1 ;

{ INV' }

do pos $\neq$ N $\rightarrow$

{ INV' $\wedge$ B }

S2

{ INV' }

od

{ Q: $r = \langle \forall i : 0 \leq i < N : A.i = i! \rangle$ }

Inicialización de nuevo: La que tenía ya no vale porque la post. INV' es más fuerte que antes, debo además asignar algo a fac. Podemos plantear:

$r, fac, pos := E, F, 0$

y derivar (ejercicio!).

O directamente se puede elegir $E = \text{True}$, $F = 1$ y demostrar.

Cuerpo del ciclo de nuevo: Tenemos la siguiente terna:

$\{ \text{INV}' \wedge B \}$

S2

$\{ \text{INV}' : r = \langle \forall i : 0 \leq i < \text{pos} : A.i = i! \rangle \wedge 0 \leq \text{pos} \leq N \wedge \text{fac} = \text{pos}! \}$

Vamos a probar con S2 de la forma: $r, \text{fac}, \text{pos} := E, F, \text{pos} + 1$.

Supongamos la **hip. $\text{INV}' \wedge B$** y veamos la wp:

$\text{wp.}(r, \text{fac}, \text{pos} := E, F, \text{pos} + 1). \text{INV}'$

$\equiv \{ \text{def. wp para } := \}$

$\underline{E = \langle \forall i : 0 \leq i < \text{pos} + 1 : A.i = i! \rangle \wedge 0 \leq \text{pos} + 1 \leq N \wedge F = (\text{pos} + 1)!}$

$\equiv \{ \text{mismos pasos que antes} \}$

$E = r \wedge (A.\text{pos} = \underline{\text{pos}}!) \wedge F = (\text{pos} + 1)!$

$\equiv \{ \text{hip. nueva} \}$

$E = r \wedge (A.\text{pos} = \text{fac}) \wedge F = (\text{pos} + 1)!$

$\equiv \{ \text{elijo } \underline{E = r \wedge (A.\text{pos} = \text{fac})} \}$

$F = (\text{pos} + 1)!$

$\equiv \{ \text{prop. } ! \}$

$F = (\text{pos} + 1) * \text{pos}!$

$\equiv \{ \text{hip. nueva de nuevo} \}$

$F = (\text{pos} + 1) * \text{fac}$

$\equiv \{ \text{elijo } \underline{F = (\text{pos} + 1) * \text{fac}} \}$

True

Perfecto!! El programa final queda:

Const N, A : array[0,N) of Int ;

Var r : Bool, pos, fac, n : Int ;

{ P: $N \geq 0$ }

$r, \text{fac}, \text{pos} := \text{True}, 1, 0 ;$

do $\text{pos} \neq N \rightarrow$

$r, \text{fac}, \text{pos} := r \wedge (A.\text{pos} = \text{fac}), \text{fac} * (\text{pos} + 1), \text{pos} + 1 ;$

od

{ Q: $r = \langle \forall i : 0 \leq i < N : A.i = i! \rangle$ }

Casi lo mismo pero mejor.

Análisis de complejidad

Tengo un arreglo de 1000 elementos.

¿Cuántas iteraciones hacía el algoritmo con ciclos anidados?

1 -> 1

$2 \rightarrow 1 * 2$
 $3 \rightarrow 1 * 2 * 3$
 ...
 $1000 \rightarrow 1 * 2 * 3 * \dots * 1000$

Esto tiene forma de triángulo $N * N / 2 = N^2 / 2$ que es aprox. la cantidad de iteraciones. Es lo que llamamos complejidad cuadrática.

¿Cuántas iteraciones hace el algoritmo con fortalecimiento? Aproximadamente N. Es lo que llamamos complejidad lineal. Mucho mejor que cuadrática.

¿Qué pasó?

- Derivando el cuerpo del ciclo me trabo porque hay una parte no programable.
- Necesito una nueva variable y una hipótesis que me diga que la variable vale esa parte no programable.
- Dos opciones:
 - Replantear S2 para que tenga un ciclo anidado. **Funciona siempre.**
 - Fortalecer el invariante y replantear el ciclo principal. **Funciona a veces pero da un programa más eficiente.**

Ejemplo (fibonacci)

```

Const N : Int ;
Var r : Int ;
{ P: N ≥ 0 }
S
{ Q: r = fib.N }
    
```

donde la función fib está definida como:

```

fib.0 ≐ 0
fib.1 ≐ 1
fib.(n+2) ≐ fib.n + fib.(n+1)
    
```

Derivación: Necesitamos un ciclo (hay que calcular una función recursiva!). Aplicamos técnica de cambio de constante por variable. Creamos variable nueva n y decimos:

```

INV ≡ r = fib.n ∧ 0 ≤ n ≤ N
B ≡ n ≠ N
    
```

Luego vale requisito iii).

Replanteo el programa:

```

Const N : Int ;
Var r : Int ;
{ P: N ≥ 0 }
S1
    
```

```

{ INV }
do n ≠ N →
  { INV ∧ B }
  S2
  { INV }
od
{ Q: r = fib.N }

```

Inicialización: Será: $r, n := 0, 0$. (ejercicio: demostrar!)

Cuerpo del ciclo: S2 será de la forma: $r, n := E, n+1$.

Derivamos: Supongamos $INV \wedge B$ y veamos la wp:

```

wp.(r, n := E, n+1).INV
≡ { def. wp para := }
  E = fib.(n+1) ∧ 0 ≤ n+1 ≤ N
≡ { burocracia: vale por hip varias }
  E = fib.(n+1)

```

Acá nos trabamos de entrada ya que no podemos aplicar definición de fib (si $n = 0$, aplica el 2do caso base y si $n > 0$ aplica el caso recursivo).

Necesitamos una nueva hipótesis que nos diga que una nueva variable vale fib.(n+1).

Probamos fortaleciendo el invariante:

```

INV' ≡ INV ∧ r2 = fib.(n+1)
      ≡ r = fib.n ∧ r2 = fib.(n+1) ∧ 0 ≤ n ≤ N

```

Inicialización de nuevo: Ahora va a ser: $r, r2, n := 0, 1, 0$ (ejercicio: demostrar!)

Cuerpo del ciclo de nuevo: Ahora va a ser: $r, r2, n := E, F, n+1$.

Derivamos: Supongamos $INV' \wedge B$ y veamos la wp:

```

wp.(r, n := E, n+1).INV'
≡ { def. wp para := }
  E = fib.(n+1) ∧ F = fib.((n+1)+1) ∧ 0 ≤ n+1 ≤ N
≡ { burocracia }
  E = fib.(n+1) ∧ F = fib.((n+1)+1)
≡ { hip. nueva }
  E = r2 ∧ F = fib.((n+1)+1)
≡ { elijo E = r2 }
  F = fib.((n+1)+1)
≡ { arit }
  F = fib.(n+2)
≡ { def. fib }
  F = fib.n + fib.(n+1)
≡ { hip. para r y r2 }
  F = r + r2
≡ { elijo F = r + r2 }
  True

```

Listo! El programa queda:

```
Const N : Int ;  
Var r, r2, n : Int ;  
{ P: N ≥ 0 }  
  r, r2, n := 0, 1, 0 ;  
  do n ≠ N →  
    r, r2, n := r2 , r + r2, n + 1  
  od  
{ Q: r = fib.N }
```

Esto anda. **Ejercicio:** testear!!

Terminación de ciclos: Función de cota

Tenemos la siguiente terna de Hoare:

$$\{ P \} \text{ **do** } B \rightarrow S \text{ **od** } \{ Q \}$$

Teníamos vistos tres requisitos que deben valer para que valga la terna:

Existe invariante INV:

- i) $P \Rightarrow INV$
- ii) $\{ INV \wedge B \} S \{ INV \}$
- iii) $INV \wedge \neg B \Rightarrow Q$

Nos falta un requisito importante: La demostración de que el ciclo termina siempre.

Vamos a introducir el concepto de **función de cota**. Como el invariante, la función de cota es una cosa que inventamos para demostrar que el ciclo es correcto, pero **no es parte del programa ni del lenguaje de programación**.

La función de cota es una función que me calcula un número entero a partir de mi estado (o sea a partir del valor de las variables y constantes en un punto determinado de la ejecución). Llamaremos t : Estado $\rightarrow$ Int a la función de cota.

Para poder demostrar que el ciclo termina, vamos a tener que demostrar dos cosas acerca de la función de cota en relación al ciclo:

- iv.a) Si estoy en el ciclo, la cota es ≥ 0 .
 $INV \wedge B \Rightarrow t \geq 0$

Equivalentemente, si la cota es < 0 , entonces el ciclo termina
(version contrarecíproca).

$$INV \wedge t < 0 \Rightarrow \neg B$$

- iv.b) La cota se achica en cada ejecución del cuerpo del ciclo.

Formalmente,

$$\{ INV \wedge B \wedge t = T \} \text{ // fijo el valor la cota antes de ejecutar el cuerpo}$$

(usando la variable de especificación T)

S // cuerpo del ciclo
 $\{ INV \wedge t < T \}$ // al terminar, la cota vale menos de lo que valía antes

Si lo logramos demostrar i) y ii), sabremos que **el ciclo termina siempre**.

¿Porque? No importa cuánto valga la cota, por ii), se va a achicar siempre que se ejecute el cuerpo del ciclo. Luego, sí o sí, en algún momento se va a hacer negativa, y por i), sabemos que si la cota es negativa el ciclo termina.

Ejemplo: Suma de los elementos de un arreglo:

Aplicamos cambio de constante por variable de la siguiente manera:

$INV \equiv res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos \leq N$
 $B \equiv pos \neq N$

```
Const N : Int, A : Array[0, N) of Int;
Var res, pos : Int;
{ P: N ≥ 0 }
  res, pos := 0, 0 ; // inicialización
  { INV }
  do pos < N →
    { INV ∧ B }
    res, pos := res + A[pos], pos + 1
    { INV }
  od
{ Q: res = ⟨ ∑ i : 0 ≤ i < N : A.i ⟩ }
```

Demostremos que este ciclo termina.

Deducción de la cota

La función de cota t debe cumplir:

iv.b) La cota decrece:

$\{ INV \wedge B \wedge t = T \}$
 $res, pos := res + A[pos], pos + 1$
 $\{ INV \wedge t < T \}$

Acá, sabemos que pos crece en el cuerpo del ciclo, luego la cota podría ser de la forma:

$t = \text{"algo"} - pos$

iv.a) $INV \wedge t < 0 \Rightarrow \neg B$,

o sea:

$(res = \langle \sum i : 0 \leq i < pos : A.i \rangle \wedge 0 \leq pos \leq N) \wedge \underline{t < 0} \Rightarrow \underline{pos \geq N}$

$t < 0$
 $\equiv \text{"algo"} - \text{pos} < 0$
 $\equiv \text{"algo"} < \text{pos}$
 $\equiv \text{"algo"} + 1 \leq \text{pos}$
 O sea, debo elegir "algo" tal que $\text{"algo"} + 1 \leq \text{pos} \Rightarrow \text{pos} \geq N$ (o sea $N \leq \text{pos}$)
 Si elijo que "algo" + 1 sea N, ya estaría, o sea "algo" = N - 1.

En fin, luego de toda esta deducción, la función de cota que me sirve es:

$$t = (N - 1) - \text{pos}$$

Otra cota que también sirve:

$$t = N - \text{pos}$$

(sale a partir de ver iv.a como $\text{INV} \wedge B \Rightarrow t \geq 0$).

Demostración formal de la cota

Quedémosnos con la cota $t = N - \text{pos}$.

Faltaría hacer la demostración **formal de todo**.

Según el digesto, debemos demostrar:

iv.a) $\text{INV} \wedge B \Rightarrow t \geq 0$

Demostración: Suponemos $\text{INV} \wedge B$ y vemos:

$t \geq 0$
 $\equiv \{ \text{def. } t \}$
 $N - \text{pos} \geq 0$
 $\equiv \{ \text{arit.} \}$
 $N \geq \text{pos}$
 $\equiv \{ \text{hip. del INV} \}$
 True

iv.b) $\{ \text{INV} \wedge B \wedge t = T \} \text{res, pos} := \text{res} + A.n, \text{pos} + 1 \{ t < T \}$

Demostración: Suponemos como hip $\text{"INV} \wedge B \wedge t = T"$ y vemos la wp:

$\text{wp.}(\text{res, pos} := \text{res} + A.n, \text{pos} + 1). (t < T)$
 $\equiv \{ \text{def. } t \}$
 $\text{wp.}(\text{res, pos} := \text{res} + A.n, \text{pos} + 1). (N - \text{pos} < T)$
 $\equiv \{ \text{def. wp} \}$
 $N - (\text{pos} + 1) < T$
 $\equiv \{ \text{arit.} \}$
 $N - \text{pos} - 1 < T$
 $\equiv \{ \text{por hip, } T = t = N - \text{pos} \}$
 $N - \text{pos} - 1 < N - \text{pos}$
 $\equiv \{ \text{arit.} \}$
 $-1 < 0$
 $\equiv \{ \text{logica} \}$
 True

Observaciones:

- Siempre que recorramos un arreglo de izquierda a derecha, la cota va a tener esta misma forma.
- Demostrar los requisitos de la cota formalmente es un poco burocrático para lo obvios que son.
- Muchas veces para la cota admitiremos explicaciones con palabras en lugar de demostraciones formales.

Ejemplo: Suma de los elementos de un arreglo de otra forma:

Const N : Int, A : Array[0, N) of Int;

Var res, n : Int;

res, n := 0, N ;

do n ≠ 0 →

res, n := res + A.(n-1) , n - 1

od

{ Q: res = $\langle \sum i : 0 \leq i < N : A.i \rangle$ }

Aplicamos cambio de constante por variable de la siguiente manera:

INV $\equiv$ res = $\langle \sum i : n \leq i < N : A.i \rangle \wedge 0 \leq n \leq N$

B $\equiv$ n ≠ 0

La función de cota es:

- t = N - n NO ANDA porque crece en el cuerpo del ciclo.
- t = n ???

Recordemos requisitos:

iv.a) Si estoy en el ciclo, t ≥ 0. t = n funciona!!! (ver INV: 0 ≤ n)

iv.b) t se achica con el cuerpo del ciclo. obvio que se achica porque se asigna n := n - 1.

CON ALGO ASÍ ALCANZA PARA PARCIAL/RECUPERATORIO.

Conclusión: t = n funciona.

Pregunta: ¿Hay otra función de cota que vale?

¿Vale la función de cota t = n + 344565? Si! Valen tanto iv.a) como iv.b).

Observación:

- La función de cota no es única.
- En particular, si tengo una función de cota t correcta, también valen como función de cota todas las funciones t' = t + C (a donde C > 0 es una constante cualquiera).
-

Ejemplo: Algoritmo de la división

Const X, Y : Int ;

Var q, r : Int ;

{ P: X ≥ 0 ∧ Y > 0 }

q, r := 0, X ;

```

do r ≥ Y →
    q, r := q + 1, r - Y
od
{ Q: X = q * Y + r ∧ 0 ≤ r ∧ r < Y }

```

$INV \equiv X = q * Y + r \wedge 0 \leq r$

La función de cota es: $t = r$???

iv.a) Si estoy en el ciclo, $r \geq 0$? Sí, por INV.

iv.b) r se achica en el cuerpo del ciclo? Sí, ya que asignamos $r := r - Y$ con $Y > 0$.

Luego, **$t = r$** es una función de cota correcta y el ciclo termina.

Observación:

- Mirar especialmente la guarda y el cuerpo del ciclo para determinar la cota. (el INV también ayuda)
- Otra cota para el algoritmo de la división podría ser de la forma $t = \text{"algo"} - q$ (ya que q crece).

Ejemplo (practico 6, ejercicio 2f):

```

{ True }
r := N ;
do r ≠ 0 →
    if r < 0 → r := r + 1    // r es negativo, por ejemplo de -10 pasa a -9.
    [] r > 0 → r := r - 1    // r es positivo, por ejemplo de 13 pasa a 12.
fi
od
{ r = 0 }

```

Este programa arranca con r en N y va hacia el 0, luego en el cuerpo del ciclo el valor absoluto de r (o sea su distancia al 0) se achica siempre.

¿Cuál es el invariante?

$INV \equiv ???$

¿Cuál es la función de cota?

$t = N - |r|$ // $|r|$ se achica siempre,
 // entonces me conviene sumarlo no restarlo.

Mejor probemos:

$t = |r|$

iv.a) $INV \wedge B \Rightarrow |r| \geq 0$??? Vale siempre por def de valor absoluto.

iv.b) Se achica? Si, ya lo pensamos. Ejercicio: demostrar formalmente (hay una terna de un **if**).

Ejercicio: Hacer testing pero además calculando el valor de la función de cota en cada iteración del ciclo.

Problemas de bordes

[Más material acá \(2021-12-01 Consulta\)](#)

[Y acá \(2021-12-14 Consulta\)](#)

Ejercicio con segmentos iniciales

Enunciado: Parecido a la función psum pero con arreglos en lugar de listas:

“Dado un arreglo con N enteros, decidir si todos los segmentos iniciales del arreglo suman ≥ 0 .”

Especificación:

```
Const N : Int, A : array[0, N) of Int;  
Var res : Bool;  
{ P : N  $\geq$  0 }  
  S  
{ Q : res =  $\langle \forall i : 0 \leq i \leq N : \text{psum}.i \geq 0 \rangle$  }  
  || psum.i =  $\langle \sum j : 0 \leq j < i : A.j \rangle$  ||
```

Derivación:

Necesitamos un ciclo. Usando la técnica de reemplazo de constante por variable proponemos:

$$\text{INV} \equiv \text{res} = \langle \forall i : 0 \leq i \leq \text{pos} : \text{psum}.i \geq 0 \rangle \wedge 0 \leq \text{pos} \leq N$$
$$B \equiv \text{pos} < N$$

donde pos es una nueva variable de tipo Int.

De esta manera tenemos asegurado $\text{INV} \wedge \neg B \Rightarrow Q$.

El programa será de la forma:

```
Const N, A : array[0,N) of Int ;  
Var res : Bool, pos : Int ;  
{ P: N  $\geq$  0 }  
  S1 ;  
{ INV }  
  do pos  $\neq$  N  $\rightarrow$ 
```

```

    { INV ∧ B }
      S2
    { INV }
  od
{ Q }

```

Cuerpo del ciclo (S2): Proponemos una asignación de la forma $res, pos := E, pos+1$.

Suponemos $INV \wedge B$ como hipótesis y vemos la wp:

```

wp.(res, pos := E, pos+1).INV
≡ { Def. wp }
  E =  $\langle \forall i : 0 \leq i \leq pos+1 : psum.i \rangle \wedge \underline{0 \leq pos+1 \leq N}$ 
≡ { hipótesis  $0 \leq pos$  y  $pos < N$  }
  E =  $\langle \forall i : 0 \leq i \leq pos+1 : psum.i \rangle$ 
≡ { partición de rango y rango unitario }
  E =  $\langle \forall i : 0 \leq i \leq pos : psum.i \rangle \wedge ( psum.(pos+1) \geq 0 )$ 
≡ { hipótesis }
  E =  $res \wedge ( psum.(pos+1) \geq 0 )$ 
≡ { def. psum }
  E =  $res \wedge ( \langle \sum j : 0 \leq j < pos+1 : A.j \rangle \geq 0 )$ 
≡ { part. de rango y rango unitario }
  E =  $res \wedge ( \langle \sum j : 0 \leq j < pos : A.j \rangle + A.pos \geq 0 )$ 

```

Nos trabamos! No podemos elegir una expresión “programable” para E por la sumatoria.

Fortalecemos el invariante:

$$INV' \equiv INV \wedge sum = \langle \sum j : 0 \leq j < pos : A.j \rangle$$

Cuerpo del ciclo de nuevo (S2): Proponemos una asignación de la forma $res, sum, pos := E, F, pos+1$.

Suponemos $INV' \wedge B$ como hipótesis y vemos la wp:

```

wp.(res, sum, pos := E, F, pos+1).INV'
≡ { Def. wp y mismos pasos ya hechos }
  E =  $res \wedge ( \langle \sum j : 0 \leq j < pos : A.j \rangle + A.pos \geq 0 ) \wedge F = \underline{\langle \sum j : 0 \leq j < pos+1 : A.j \rangle}$ 
≡ { part. de rango y rango unitario }
  E =  $res \wedge ( \langle \sum j : 0 \leq j < pos : A.j \rangle + A.pos \geq 0 ) \wedge F = \underline{\langle \sum j : 0 \leq j < pos : A.j \rangle} + A.pos$ 
≡ { hipótesis nueva dos veces }
  E =  $res \wedge ( sum + A.pos \geq 0 ) \wedge F = sum + A.pos$ 
≡ { elijo convenientemente }
  True

```

Inicialización: Ejercicio. Resultado: $res, sum, pos := True, 0, 0$.

Resultado final:

```

Const N : Int, A : array[0, N) of Int;
Var r : Int;
{P : N ≥ 0}

```

```

res, sum, pos := True, 0, 0 ;
do pos < N →
    res, sum, pos := res ∧ ( sum + A.pos ≥ 0 ), sum + A.pos, pos + 1
od
{ Q : res = (∀ i : 0 ≤ i ≤ N : psum.i ≥ 0 ) }

```

¿Qué pasa si fortalecemos mal?

Volvemos atrás:

- Cuerpo del ciclo (S2): Proponemos una asignación de la forma $res, pos := E, pos+1$.

Suponemos $INV \wedge B$ como hipótesis y vemos la wp:

```

wp.(res, pos := E, pos+1).INV
≡ { ... }
E = res ∧ ( ⟨ ∑ j : 0 ≤ j < pos+1 : A.j ⟩ ≥ 0 )

```

Fortalecemos así (**mala idea**):

$$INV' \equiv INV \wedge sum = \langle \sum j : 0 \leq j < pos+1 : A.j \rangle$$

Sabemos que $0 \leq pos \leq N$, así que cuando $pos = N$ la sumatoria “sum” queda:

$$A.0 + A.1 + \dots + A.(N-1) + A.N$$

¡Mal! Se va de los límites del arreglo.

Si derivamos el cuerpo del ciclo de nuevo:

- Cuerpo del ciclo de nuevo (S2): Proponemos una asignación de la forma $res, sum, pos := E, F, pos+1$.

Suponemos $INV' \wedge B$ como hipótesis y vemos la wp:

```

wp.(res, sum, pos := E, F, pos+1).INV'
≡ { Def. wp y mismos pasos ya hechos }
E = res ∧ ( ⟨ ∑ j : 0 ≤ j < pos+1 : A.j ⟩ ≥ 0 ) ∧ F = ⟨ ∑ j : 0 ≤ j < pos+2 : A.j ⟩
≡ { part. de rango y rango unitario }
E = res ∧ ( ⟨ ∑ j : 0 ≤ j < pos : A.j ⟩ + A.(pos+1) ≥ 0 ) ∧
F = ⟨ ∑ j : 0 ≤ j < pos+1 : A.j ⟩ + A.(pos+1)
≡ { hipótesis nueva dos veces }
E = res ∧ ( sum + A.(pos+1) ≥ 0 ) ∧ F = sum + A.(pos+1)
≡ { elijo convenientemente }
True

```

Resultado:

```

do pos < N →
    res, sum, pos := res ∧ ( sum + A.(pos+1) ≥ 0 ), sum + A.(pos+1), pos + 1
od

```

¡Mal! En la última iteración del ciclo $pos = N - 1$ y $A.(pos+1) = A.N$ se va de los límites del arreglo.

Ejercicio con segmentos arbitrarios

Segmento de suma máxima

Especificación:

```
Const N : Int, A : array[0, N) of Int;  
Var r : Int;  
{P : N ≥ 0}  
S  
{Q : r = ⟨Max p, q : 0 ≤ p ≤ q ≤ N : sum.p.q⟩  
  |[sum.p.q = ⟨∑ i : p ≤ i < q : A.i⟩]| }
```

¿Qué calcula este programa? Considera todos los segmentos posibles del arreglo (incluso los segmentos vacíos, porque puede ser $p = q$). p y q funcionan como índices de comienzo y fin del segmento: p cerrado y q abierto. Ejemplo: si $(p, q) = (6, 9)$ estamos hablando de la suma: $A.6 + A.7 + A.8$. De todos los segmentos se calcula la suma y se toma el máximo.

Derivación: VER EN LIBRO CÁLCULO DE PROGRAMAS.

Resultado final:

```
Const N : Int, A : array[0, N) of Int;  
Var r : Int;  
{P : N ≥ 0}  
  r, r2, n := 0, 0, 0 ;  
  do n ≠ N →  
    r, r2, n := r max ( r2 + A.n ) max 0 , ( r2 + A.n ) max 0 , n+1 ;  
  od  
{Q : r = ⟨Max p, q : 0 ≤ p ≤ q ≤ N : sum.p.q⟩  
  |[sum.p.q = ⟨∑ i : p ≤ i < q : A.i⟩]| }
```

Ejercicio: Testear que efectivamente esto está calculando el segmento de suma máxima.

Función de cota: $t = N - n$

(la misma que tenemos siempre que recorremos un arreglo de izquierda a derecha)

Segmento de suma máxima sin Segmentos vacíos

En esta versión más simple del problema de “Segmento de suma máxima” no consideraremos segmentos vacíos. Observar en la especificación que dice $p < q$ en lugar de $p \leq q$.

Especificación:

Const N : Int, A : array[0, N) of Int;
 Var res : Int;
 {P : N ≥ 0}
 S
 {Q : res = ⟨Max p, q : 0 ≤ p < q ≤ N : sum.p.q⟩
 [[sum.p.q = ⟨∑ i : p ≤ i < q : A.i⟩]] }

Derivación:

Necesitamos un ciclo. Usando la técnica de reemplazo de constante por variable proponemos:

INV ≡ res = ⟨Max p, q : 0 ≤ p < q ≤ pos : sum.p.q⟩ ∧ 0 ≤ pos ≤ N

B ≡ pos < N

donde pos es una nueva variable de tipo Int.

De esta manera tenemos asegurado INV ∧ ¬B ⇒ Q.

El programa será de la forma:

Const N, A : array[0,N) of Int ;

Var res : Int, pos : Int ;

{ P: N ≥ 0 }

S1 ;

{ INV }

do pos < N →

{ INV ∧ B }

S2

{ INV }

od

{ Q }

Cuerpo del ciclo (S2): Proponemos una asignación de la forma:

res, pos := E, pos + 1

Suponemos INV ∧ B como hipótesis y vemos la wp:

wp.(res, pos := E, pos+1).INV

≡ { def. wp }

E = ⟨Max p, q : 0 ≤ p < q ≤ pos + 1 : sum.p.q⟩ ∧ 0 ≤ pos + 1 ≤ N

≡ { por hipótesis 0 ≤ pos ≤ N y pos < N }

E = ⟨Max p, q : 0 ≤ p < q ≤ pos + 1 : sum.p.q⟩

≡ { lógica }

E = ⟨Max p, q : 0 ≤ p < q ∧ (q ≤ pos ∨ q = pos + 1) : sum.p.q⟩

≡ { lógica: distributividad }

E = ⟨Max p, q : (0 ≤ p < q ≤ pos) ∨ (0 ≤ p < q ∧ q = pos + 1) : sum.p.q⟩

≡ { partición de rango }

E = ⟨Max p, q : 0 ≤ p < q ≤ pos : sum.p.q⟩ max

⟨Max p, q : 0 ≤ p < q ∧ q = pos + 1 : sum.p.q⟩

≡ { hipótesis }

E = res max ⟨Max p, q : 0 ≤ p < q ∧ q = pos + 1 : sum.p.q⟩

≡ { eliminación de variable q }

E = res max ⟨Max p : 0 ≤ p < pos + 1 : sum.p.(pos + 1)⟩

$\equiv \{ \text{no fortalecer todavía!! este Max considera todos los segmentos no vacíos terminados en la posición } p \text{ inclusive (A.p) , por lo que tenemos un problema de borde.} \}$

Faltan más pasos: lógica

}

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} \vee p = \text{pos} : \text{sum.p.}(\text{pos} + 1) \rangle$

$\equiv \{ \text{partición de rango y rango unitario} \}$

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.}(\text{pos} + 1) \rangle \text{ max } \underline{\text{sum.pos.}(\text{pos} + 1)}$

$\equiv \{ \text{def. sum} \}$

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.}(\text{pos} + 1) \rangle \text{ max } \langle \underline{\sum i : \text{pos} \leq i < \text{pos} + 1 : A.i} \rangle$

$\equiv \{ \text{rango unitario: } i = \text{pos} \}$

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} : \underline{\text{sum.p.}(\text{pos} + 1)} \rangle \text{ max } A.\text{pos}$

$\equiv \{ \text{def. sum} \}$

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} : \langle \underline{\sum i : p \leq i < \text{pos} + 1 : A.i} \rangle \rangle \text{ max } A.\text{pos}$

$\equiv \{ \text{lógica} \}$

$E = \text{res max } \langle \text{Max } p : 0 \leq p < \text{pos} : \langle \underline{\sum i : p \leq i < \text{pos} \vee i = \text{pos} : A.i} \rangle \rangle \text{ max } A.\text{pos}$

$\equiv \{ \text{partición de rango y rango unitario} \}$

$E = \text{res max } \langle \underline{\text{Max } p : 0 \leq p < \text{pos} : \langle \sum i : p \leq i < \text{pos} : A.i \rangle + A.\text{pos}} \rangle \text{ max } A.\text{pos}$

$\equiv \{ \text{distributividad de + con Max, y deshacer definición de sum} \}$

$E = \text{res max } (\langle \underline{\text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.pos}} \rangle + A.\text{pos}) \text{ max } A.\text{pos}$

Acá nos detenemos. Este Max ya no incluye al elemento A.pos, ya que considera todos los segmentos no vacíos terminados en la posición pos-1:

$$\begin{array}{l} A.0 + A.1 + \dots + A.(\text{pos}-2) + A.(\text{pos}-1) \\ A.1 + \dots + A.(\text{pos}-2) + A.(\text{pos}-1) \\ \dots \\ A.(\text{pos}-2) + A.(\text{pos}-1) \\ A.(\text{pos}-1) \end{array}$$

Por lo tanto podemos fortalecer sin problemas.

$\text{INV}' \equiv \text{INV} \wedge \text{aux} = \langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.pos} \rangle$

Cuerpo del ciclo de nuevo (S2): Proponemos una asignación de la forma

$\text{res, aux, pos} := E, F, \text{pos} + 1.$

Suponemos $\text{INV}' \wedge B$ como hipótesis y vemos la wp:

$\text{wp.}(\text{res, aux, pos} := E, F, \text{pos} + 1).\text{INV}'$

$\equiv \{ \text{def. wp} \}$

$E = \langle \underline{\text{Max } p, q : 0 \leq p < q \leq \text{pos} + 1 : \text{sum.p.q}} \rangle \wedge 0 \leq \text{pos} + 1 \leq N \wedge$

$F = \langle \text{Max } p : 0 \leq p < \text{pos} + 1 : \text{sum.p.}(\text{pos} + 1) \rangle$

$\equiv \{ \text{mismos pasos que hicimos antes} \}$

$E = \text{res max } (\langle \underline{\text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.pos}} \rangle + A.\text{pos}) \text{ max } A.\text{pos} \wedge$

$F = \langle \text{Max } p : 0 \leq p < \text{pos} + 1 : \text{sum.p.}(\text{pos} + 1) \rangle$

$\equiv \{ \text{hipótesis nueva} \}$

$E = \text{res max } (\text{aux} + A.\text{pos}) \text{ max } A.\text{pos} \wedge$

$F = \langle \text{Max } p : 0 \leq p < \text{pos} + 1 : \text{sum.p.}(\text{pos} + 1) \rangle$

$\equiv \{ \text{elegimos } E = \text{res max } (\text{aux} + A.\text{pos}) \text{ max } A.\text{pos} \}$

$F = \langle \underline{\text{Max } p : 0 \leq p < \text{pos} + 1 : \text{sum.p.}(\text{pos} + 1)} \rangle$

$\equiv \{ \text{a estos pasos ya los hicimos también, lo hago rápido: } \}$
 $F = \langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.}(\text{pos} + 1) \rangle \text{ max A.pos}$
 $\equiv \{ \text{separación de término en sum} \}$
 $F = \langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.pos} + \text{A.pos} \rangle \text{ max A.pos}$
 $\equiv \{ \text{distributividad} \}$
 $F = (\langle \text{Max } p : 0 \leq p < \text{pos} : \text{sum.p.pos} \rangle + \text{A.pos}) \text{ max A.pos}$
 $\equiv \{ \text{hipótesis nueva} \}$
 $F = (\text{aux} + \text{A.pos}) \text{ max A.pos}$
 $\equiv \{ \text{elijo } F = (\text{aux} + \text{A.pos}) \text{ max A.pos} \}$
 True

Inicialización (S1): Queda como ejercicio. Obtenemos:
 $\text{res, aux, pos} := -\text{infinito}, -\text{infinito}, 0$

Resultado final:

Const N : Int, A : array[0, N) of Int;
 Var res, aux, pos : Int;
 $\{P : N \geq 0\}$
 $\text{res, aux, pos} := -\text{infinito}, -\text{infinito}, 0 ;$
do $n < N \rightarrow$
 $\text{res, aux, pos} := \text{res max (aux + A.pos) max A.pos,}$
 $(\text{aux} + \text{A.pos}) \text{ max A.pos,}$
 $\text{pos} + 1$
od
 $\{Q : \text{res} = \langle \text{Max } p, q : 0 \leq p < q \leq N : \text{sum.p.q} \rangle$
 $|| \text{sum.p.q} = \langle \sum i : p \leq i < q : \text{A.i} \rangle || \}$

Ejercicio: Testear que efectivamente esto está calculando el segmento de suma máxima.

Función de cota: $t = N - n$

(la misma que tenemos siempre que recorremos un arreglo de izquierda a derecha)

Especificaciones con segmentos de arreglo

¿Cómo escribimos cuantificadores sobre segmentos de arreglo?

Solemos usar dos variables cuantificadas p, q para representar el segmento:

$$\text{A.p, A.(p+1), ... , A.(q-1)}$$

Es decir, el segmento va desde la posición “p” hasta la posición “q - 1”.
(observar que no incluye la posición q, es cerrado-abierto: “[p, q)”.

La cuantificación sería de la siguiente forma:

$$\langle \oplus p, q : R.p.q \wedge \dots : T.p.q \rangle$$

donde R es un predicado que indica el tipo de segmento:

$0 \leq p \leq q \leq N$: segmentos arbitrarios (o sea, todos los segmentos posibles)

$0 \leq p < q \leq N$: segmentos no vacíos

$0 < p \leq q \leq N$: segmentos no iniciales (no prefijos)

$0 \leq p \leq q < N$: segmentos no finales (no sufijos)

$0 < p \leq q < N$: segmentos interiores (no iniciales ni finales)

$0 < p < q < N$: segmentos interiores no vacíos

$p = 0 \wedge 0 \leq q \leq N$: segmentos iniciales (se puede eliminar p, queda: $0 \leq q \leq N$)

$0 \leq p \leq N \wedge q = N$: segmentos finales (se puede eliminar q, queda: $0 \leq p \leq N$)

TODO: PARTICIONES DE RANGO CON SEGMENTOS Y CON PARES DE ELEMENTOS

Temas complementarios

Terminación anticipada de ciclos

Volvemos al ejercicio de [esta sección](#):

El programa era:

Const N : Int, A : array[0, N) of Int;

Var r : Int;

{P : N ≥ 0}

res, sum, pos := True, 0, 0 ;

do pos < N →

res, sum, pos := res ∧ (sum + A.pos ≥ 0), sum + A.pos, pos + 1

od

{ Q : res = (∀ i : 0 ≤ i ≤ N : psum.i ≥ 0) }

Notamos que al primer False podemos terminar sin necesidad de recorrer el resto del arreglo. Nueva guarda:

$$B' \equiv \text{pos} < N \wedge \text{res}$$

(sólo quiero continuar en el ciclo si res es True)

¿Qué debo demostrar al introducir esta modificación?

Lugares donde aparece la guarda:

- Requisito 2:

Ya habíamos demostrado:

{ INV ∧ B } S1 { INV } (donde S1 es el cuerpo del ciclo)

¿Vale con B'?:

{ INV ∧ B' } S1 { INV }

Respuesta: Sí, si vale una terna, también vale cuando fortalecemos su precondition.

El conjunto de estados iniciales admitidos por $INV \wedge B'$ está incluido en el conjunto admitido por $INV \wedge B$.

- Requisito 3:

Ya habíamos demostrado:

$$INV \wedge \neg B \Rightarrow Q$$

¿Vale con B' ?:

$$INV \wedge \neg B' \Rightarrow Q$$

¡No necesariamente!

Veamos:

$$INV \wedge \neg B' \Rightarrow Q$$

$$\equiv INV \wedge \neg(B \wedge res) \Rightarrow Q$$

$$\equiv INV \wedge (\neg B \vee \neg res) \Rightarrow Q$$

$$\equiv (INV \wedge \neg B) \vee (INV \wedge \neg res) \Rightarrow Q$$

$$\equiv (INV \wedge \neg B \Rightarrow Q) \wedge (INV \wedge \neg res \Rightarrow Q)$$

La primera " $INV \wedge \neg B \Rightarrow Q$ " ya la habíamos demostrado.

Falta demostrar la segunda: $INV \wedge \neg res \Rightarrow Q$.

Solamente hace falta demostrar:

$$INV \wedge \neg B' \Rightarrow Q.$$

Ahora, sabemos que:

$$INV \wedge \neg B' \Rightarrow Q \equiv (INV \wedge \neg B \Rightarrow Q) \wedge (INV \wedge \neg res \Rightarrow Q)$$

(ejercicio: demostrar)

Y también sabemos que vale " $INV \wedge \neg B \Rightarrow Q$ ".

Luego sólo hace falta demostrar:

$$INV \wedge \neg res \Rightarrow Q$$

Demostración: Queremos demostrar $INV \wedge \neg res \Rightarrow Q$.

Hipótesis 1: $INV \equiv res = \langle \forall i : 0 \leq i \leq n : psum.i \geq 0 \rangle \quad \wedge \quad 0 \leq n \leq N \wedge \dots$

Hipótesis 2: $\neg res$

Suponemos verdaderas estas dos hipótesis. Y vemos Q:

Q

$$\equiv \{ \text{def. Q} \}$$

$$res = \langle \forall i : 0 \leq i \leq N : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{hip. 2} \}$$

$$False = \langle \forall i : \underline{0 \leq i \leq N} : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{lógica (ver abajo explicación), sabiendo que } 0 \leq n \leq N \text{ (hip. 1)} \}$$

$$False = \langle \forall i : 0 \leq i \leq n \vee n < i \leq N : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{part. rango} \}$$

$$False = \langle \underline{\forall i : 0 \leq i \leq n : psum.i \geq 0} \rangle \wedge \langle \forall i : n < i \leq N : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{hip. 1} \}$$

$$False = res \wedge \langle \forall i : n < i \leq N : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{hip. 2 de nuevo} \}$$

$$False = \underline{False} \wedge \langle \forall i : n < i \leq N : psum.i \geq 0 \rangle$$

$$\equiv \{ \text{lógica: absorbente de } \wedge \}$$

$$False = False$$

$$\equiv \{ \text{lógica} \}$$

True

$i \in \{0, 1, 2, \dots, N\} \quad (0 \leq i \leq N)$

$i \in \{0, 1, 2, \dots, n\} \quad (0 \leq i \leq n)$

$i \in \{n+1, n+2, \dots, N\} \quad (n < i \leq N) \text{ ó } (n+1 \leq i \leq N)$

El programa finalmente queda:

Const N : Int, A : array[0, N) of Int;

Var r : Int;

{P : N ≥ 0}

res, sum, pos := True, 0, 0 ;

do pos < N ∧ res →

res, sum, pos := res ∧ (sum + A.pos ≥ 0), sum + A.pos, pos + 1

od

{ Q : res = (∀ i : 0 ≤ i ≤ N : psum.i ≥ 0) }

Traducción a C

Ver Apéndice B del libro [Cálculo de Programas](#).

Ejercicios

[practico3.pdf](#)

[practico4.pdf](#)

Resoluciones de ejercicios

Ejercicio de final con varias cosas interesantes

- Problema con pares de elementos de arreglo
- Problema de “borde”
- Análisis por casos en el cuerpo (if)
- Dos fortalecimientos

Consideremos la siguiente especificación:

Const N: Int, A : array[0, N) of Int;

Var r : Int;

{ P: N ≥ 0 }

S

{ Q : r = (∑ i, j : 0 ≤ i < j < N ∧ A.i * A.j > 0 : A.i * A.j) }

Este programa calcula la suma de todas formas posibles de multiplicar dos elementos distintos del arreglo, siempre que ese producto sea > 0 .

Ejemplo (testing): Para el arreglo $A = [3, -2, 1, 0, -2]$, el resultado es:

$$3 * 1 + (-2) * (-2) = 7$$

¿Programa a ojo? Se podría hacer con ciclos anidados, el ciclo de afuera para fijar un elemento, el ciclo de adentro para fijar el otro elemento, y un if para ver si el producto es > 0 .

Derivación: Necesitamos un ciclo. Aplicamos cambio de constante por variable:

$$\text{INV} \equiv r = \langle \sum i, j : 0 \leq i < j < n \wedge A.i * A.j > 0 : A.i * A.j \rangle \wedge 0 \leq n \leq N$$

$$B \equiv n \neq N$$

La cota va a ser $t = N - n$ (ya sabemos que vamos a recorrer el arreglo de izq. a der.).

El programa será de la forma:

Const N: Int, A : array[0, N) of Int;

Var r : Int;

{ P: $N \geq 0$ }

S1

{ INV }

do $n \neq N \rightarrow$

{ INV $\wedge$ B }

S2

{ INV }

od

{ Q: $r = \langle \sum i, j : 0 \leq i < j < N \wedge A.i * A.j > 0 : A.i * A.j \rangle$ }

Cuerpo del ciclo: Será de la forma: $r, n := E, n + 1$.

Sup. INV $\wedge$ B y veamos la wp:

$\text{wp.}(r, n := E, n + 1). \text{INV}$

$\equiv \{ \text{def. wp} \}$

$E = \langle \sum i, j : 0 \leq i < j < n+1 \wedge A.i * A.j > 0 : A.i * A.j \rangle \wedge 0 \leq n+1 \leq N$

$\equiv \{ \text{hip. } 0 \leq n \leq N \text{ y } n \neq N \}$

$E = \langle \sum i, j : 0 \leq i < j < n+1 \wedge A.i * A.j > 0 : A.i * A.j \rangle$

$\equiv \{ \text{reescribimos: } 0 \leq i < j < n+1 \equiv 0 \leq i < j \wedge (j < n \vee j = n) \}$
distribuimos y partimos rango }

$E = \langle \sum i, j : 0 \leq i < j \wedge j < n \wedge A.i * A.j > 0 : A.i * A.j \rangle +$
 $\langle \sum i, j : 0 \leq i < j \wedge j = n \wedge A.i * A.j > 0 : A.i * A.j \rangle$

$\equiv \{ \text{hip. para } r \}$

$E = r + \langle \sum i, j : 0 \leq i < j \wedge j = n \wedge A.i * A.j > 0 : A.i * A.j \rangle$

$\equiv \{ \text{eliminación } j = n \}$

$E = r + \langle \sum i : 0 \leq i < n \wedge A.i * \underline{A.n} > 0 : A.i * \underline{A.n} \rangle$

// aca si quisiera podría ya plantear ciclo anidado, y obtener el mismo programa que
// había pensado a ojo.

// pero quiero fortalecer

// no puedo fortalecer como está, porque para calcular esto no puedo **reusar**
 // lo que se calculó en la iteración anterior.
 // me doy cuenta porque esta cuenta depende de **A.n**, y es un elemento que no
 // habíamos visto nunca antes en la iteraciones anteriores.
 // para poder fortalecer, debo poder **deshacerme de las menciones a A.n**.

$\equiv \{ \text{distributividad} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge \underline{A.i * A.n > 0} : A.i \rangle * A.n$$

$\equiv \{ \text{para poder deshacerme de este A.n, puedo recurrir a la regla de los signos:$

$$a * b > 0 \equiv (a > 0 \wedge b > 0) \vee (a < 0 \wedge b < 0)$$

$\}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge ((A.i > 0 \wedge \underline{A.n > 0}) \vee (A.i < 0 \wedge \underline{A.n < 0})) : A.i \rangle * A.n$$

$\equiv \{ \text{"A.n > 0" y "A.n < 0" no dependen de i, tienen un valor de verdad fijo. Podemos preguntar por su valor de verdad con un if!!! } }$

- Caso 1: **A.n > 0**: ($\leftarrow$ debe ser programable ya que es la guarda del if)

$\equiv \{ \text{sustituyo los valores de verdad} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge ((A.i > 0 \wedge \text{True}) \vee (A.i < 0 \wedge \text{False})) : A.i \rangle * A.n$$

$\equiv \{ \text{simplifico por lógica} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge A.i > 0 : A.i \rangle * A.n$$

Ahora sí me trabo pero puedo fortalecer: $s = \langle \sum i : 0 \leq i < n \wedge A.i > 0 : A.i \rangle$

¿Qué es s? s es la sumatoria de los elementos positivos vistos hasta el momento.

- Caso 2: **A.n < 0**: ($\leftarrow$ debe ser programable ya que es la guarda del if)

$\equiv \{ \text{sustituyo los valores de verdad} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge ((A.i > 0 \wedge \text{False}) \vee (A.i < 0 \wedge \text{True})) : A.i \rangle * A.n$$

$\equiv \{ \text{simplifico por lógica} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge A.i < 0 : A.i \rangle * A.n$$

Ahora también me trabo pero puedo fortalecer:

$$t = \langle \sum i : 0 \leq i < n \wedge A.i < 0 : A.i \rangle$$

¿Qué es t? t es la sumatoria de los elementos negativos vistos hasta el momento.

- Caso 3: **A.n = 0**: ($\leftarrow$ debo cubrir todas las posibilidades, tengo tres guardas)

$\equiv \{ \text{sustituyo los valores de verdad} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge ((A.i > 0 \wedge \text{False}) \vee (A.i < 0 \wedge \text{False})) : A.i \rangle * A.n$$

$\equiv \{ \text{simplifico por lógica} \}$

$$E = r + \langle \sum i : 0 \leq i < n \wedge \text{False} : A.i \rangle * A.n$$

$\equiv \{ \text{rango vacío} \}$

$$E = r + 0 * A.n$$

$\equiv \{ \text{elijo E = r} \}$

True

Nuevo invariante:

$$\text{INV}' \equiv \text{INV} \wedge s = \langle \sum i : 0 \leq i < n \wedge A.i > 0 : A.i \rangle$$

$$\wedge t = \langle \sum i : 0 \leq i < n \wedge A.i < 0 : A.i \rangle$$

Cuerpo del ciclo de nuevo:

Sup. $\text{INV}' \wedge B$ y vemos la wp:

$wp.(r, s, t, n := E, F, G, n+1).INV'$
 $\equiv \{ \text{def. wp para } := \}$
 $E = \langle \sum i, j: 0 \leq i < j < n+1 \wedge A.i * A.j > 0 : A.i * A.j \rangle \wedge 0 \leq n+1 \leq N$
 $\wedge F = \langle \sum i: 0 \leq i < n+1 \wedge A.i > 0 : A.i \rangle \wedge G = \langle \sum i: 0 \leq i < n+1 \wedge A.i < 0 : A.i \rangle$
 $\equiv \{ \text{partición de rango e hipotesis} \}$
 $E = \langle \sum i, j: 0 \leq i < j < n+1 \wedge A.i * A.j > 0 : A.i * A.j \rangle \wedge 0 \leq n+1 \leq N$
 $\wedge F = s + \langle \sum i: i = n \wedge A.i > 0 : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge A.i < 0 : A.i \rangle$
 $\equiv \{ \text{Leibniz: reemplazo } i \text{ por } n \text{ ya que } i = n, \text{ pero eliminar } i \}$
 $E = \langle \sum i, j: 0 \leq i < j < n+1 \wedge A.i * A.j > 0 : A.i * A.j \rangle \wedge 0 \leq n+1 \leq N$
 $\wedge F = s + \langle \sum i: i = n \wedge A.n > 0 : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge A.n < 0 : A.i \rangle$
 $\equiv \{ \text{ahora sí, repito los pasos que hice antes hasta el análisis por casos} \}$
 $E = r + \langle \sum i: 0 \leq i < n \wedge ((A.i > 0 \wedge \underline{A.n > 0}) \vee (A.i < 0 \wedge \underline{A.n < 0})) : A.i \rangle * A.n$
 $\wedge F = s + \langle \sum i: i = n \wedge \underline{A.n > 0} : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge \underline{A.n < 0} : A.i \rangle$

- Caso 1: $A.n > 0$:
 $\equiv \{ \text{sustituyo valores de verdad} \}$
 $E = r + \langle \sum i: 0 \leq i < n \wedge A.i > 0 : A.i \rangle * A.n \wedge$
 $F = s + \langle \sum i: i = n \wedge \text{True} : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge \text{False} : A.i \rangle$
 $\equiv \{ \text{hip para } s \}$
 $E = r + s * A.n \wedge$
 $F = s + \langle \sum i: i = n \wedge \text{True} : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge \text{False} : A.i \rangle$
 $\equiv \{ \text{el 1ro es rango unitario, y el 2do es rango vacío} \}$
 $E = r + s * A.n \wedge F = s + A.n \wedge G = t$
 $\equiv \{ \underline{\text{elijo convenientemente}} \}$
True
- Caso 2: $A.n < 0$:
 $\equiv \{ \text{pasos similares al caso anterior} \}$
 $E = r + t * A.n \wedge F = s \wedge G = t + A.n$
 $\equiv \{ \underline{\text{elijo convenientemente}} \}$
True
- Caso 3: $A.n = 0$:
 $\equiv \{ \text{sustituyo valores de verdad} \}$
 $E = r + \langle \sum i: 0 \leq i < n \wedge ((A.i > 0 \wedge \underline{\text{False}}) \vee (A.i < 0 \wedge \underline{\text{False}})) : A.i \rangle * A.n$
 $\wedge F = s + \langle \sum i: i = n \wedge \underline{\text{False}} : A.i \rangle \wedge G = t + \langle \sum i: i = n \wedge \text{False} : A.i \rangle$
 $\equiv \{ \text{todos rangos vacíos} \}$
 $E = r \wedge F = s \wedge G = t$
 $\equiv \{ \underline{\text{elijo convenientemente}} \}$
True

Listo el cuerpo del ciclo!!! Es un if con asignaciones adentro.

Inicialización: $r, s, t, n := 0, 0, 0, 0$ (todos rangos vacíos de sumatorias)

Resultado final:

Const N: Int, A : array[0, N) of Int;

Var r : Int;


```

{ P:  $N \geq 0$  }
r, s, t, n := 0, 0, 0, 0 ;
do  $n \neq N \rightarrow$ 
    if  $A.n > 0 \rightarrow r, s, \cancel{t}, n := r + s * A.n, s + A.n, \cancel{t}, n+1$ 
     $\square$   $A.n < 0 \rightarrow r, \cancel{s}, t, n := r + t * A.n, \cancel{s}, t + A.n, n+1$ 
     $\square$   $A.n = 0 \rightarrow \cancel{r}, \cancel{s}, \cancel{t}, n := \cancel{r}, \cancel{s}, \cancel{t}, n+1$ 
    if
od
{ Q:  $r = \langle \sum i, j : 0 \leq i < j < N \wedge A.i * A.j > 0 : A.i * A.j \rangle$  }

```

Conclusión: A ojo nos salió con dos ciclos anidados, pero gracias a la derivación pudimos encontrar una forma de hacer fortalecimiento que salga mucho más eficiente con un solo ciclo.

Ejercicio: Testear con el arreglo de ejemplo que vimos, chequear que da 7.

Referencias

[Cálculo de programas](#)

Javier Blanco, Silvina Smith, Damián Barsotti
ISBN 978-950-33-0642-0. Año 2009.

Hoare, C. A. R. (October 1969). "[An axiomatic basis for computer programming](#)".
Communications of the ACM. 12 (10): 576–580.

Edsger Dijkstra: The Humble Programmer (PDF file, 473kB)
<http://cs.utexas.edu/~EWD/ewd03xx/EWD340.PDF>

Hoare 1996, "How Did Software Get So Reliable Without Proof?"
<https://gwern.net/doc/math/1996-hoare.pdf>

Dijkstra, Edsger W. (March 1968). "Letters to the editor: Go to statement considered harmful" (PDF). Communications of the ACM. 11 (3): 147–148. doi:10.1145/362929.362947. S2CID 17469809.
<https://www.cs.utexas.edu/~EWD/ewd02xx/EWD215.PDF>

Apéndice

Ejercicios

[practico1.pdf](#)

[practico2.pdf](#)

[practico3.pdf](#)

[practico4.pdf](#)

Digestos

Digesto de Funciones de Listas y Propiedades

[listas.pdf](#)

Digesto de Cuantificadores y Cálculo Proposicional

[digesto.pdf](#)

Cuantificador de Conteo N

[conteo.pdf](#)

Digesto para la Programación Imperativa

[imperativo.pdf](#)

Videos

Dictado completo del teórico 2022:

https://www.youtube.com/playlist?list=PLSddYh_b7LP8a0mdrpx1-3M9S9Grnz1pQ

Contenido extra I: Clases prácticas y consultas

[Pizarras 2021](#)

[Grabaciones 2021](#)

Funcional / Cuantificación general (2021)

- 2021-08-26 Práctico (práctico 1)
- 2021-08-31 Práctico - Sala 2 (práctico 1, práctico 2: sum_cuad, iga)
- 2021-09-02 Práctico - Sala 2 (práctico 2: cuantos, busca, creciente)
- 2021-09-09 Práctico (práctico 2: prod_suf, cos, minimo)
- 2021-09-14 Práctico (sum_ant, esCuad)
- 2021-09-16 Práctico (segmentos de lista)
- 2021-09-23 Práctico (segmentos de lista)
- 2021-09-28 Práctico

Imperativo (2021)

- 2021-10-12 Práctico
- 2021-10-14 Práctico
- 2021-10-19 Práctico
- 2021-10-21 Práctico
- 2021-10-26 Práctico
- 2021-10-28 Práctico
- 2021-11-02 Práctico

☰ 2021-11-04 Práctico

☰ 2021-11-09 Práctico

☰ 2021-11-11 Práctico

Clases de consulta (2021)

☰ 2021-12-01 Consulta

☰ 2021-12-14 Consulta

Clases de consulta (2023)

[CONSULTA ALGORITMOS 1 - 4/12/2023](#)

[CONSULTA ALGORITMOS 1 - 19/12/2023](#)

[CONSULTA ALGORITMOS 1 - 22/2/2024](#)

Clases de consulta (2024)

[CONSULTA ALGORITMOS 1 - 16/12/2024](#) ([Recording](#))

Clases de consulta (2025)

[CONSULTA ALGORITMOS 1 - 17/09/2025](#) ([Recording](#))

[CONSULTA ALGORITMOS 1 - 28/10/2025](#) ([Recording](#))

[CONSULTA ALGORITMOS 1 - 10/11/2025](#) ([Recording](#))

Exámenes parciales (2016 – 2019)

https://wiki.cs.famaf.unc.edu.ar/doku.php?id=algo1:2019-2#exámenes_anos_anteriores

- 2016 (2do cuat.):
 - 1er parcial
 - 2do parcial
 - Recuperatorio 1er parcial (con solución ej. 3)

- Recuperatorio 2do parcial (con solución ej. 3)
- 2017 (2do cuat.):
 - 1er parcial
 - 2do parcial
 - Recuperatorio 1er parcial
 - Recuperatorio 2do parcial
- 2018 (2do cuat.):
 - 1er parcial
 - 2do parcial
 - Recuperatorio 1er parcial
 - Recuperatorio 2do parcial
- 2019 (2do cuat.):
 - 1er parcial
 - 2do parcial: tema A, tema B
 - Recuperatorio 1er parcial
 - Recuperatorio 2do parcial

Solución 1er parcial turno tarde (2024)

EJERCICIO 1a.

Hago inducción en xs. Espero encontrar una definición de la forma:

$f.[] \doteq ???$

$f.(x \blacktriangleright xs) \doteq ???$

Empiezo con el caso inductivo por las dudas haya que generalizar.

Caso inductivo:

$$\begin{aligned}
 & f.(x \blacktriangleright xs) \\
 = & \{ \text{especificación} \} \\
 & \langle \sum i : 0 \leq i < \#(x \blacktriangleright xs) : (\#(x \blacktriangleright xs) - i) * (x \blacktriangleright xs)!i \rangle \\
 = & \{ \text{def. \# y lógica} \} \\
 & \langle \sum i : i = 0 \vee 1 \leq i < \#xs + 1 : (\#xs + 1 - i) * (x \blacktriangleright xs)!i \rangle \\
 = & \{ \text{part. de rango y rango unitario} \} \\
 & (\#xs + 1 - 0) * (x \blacktriangleright xs)!0 + \langle \sum i : 1 \leq i < \#xs + 1 : (\#xs + 1 - i) * (x \blacktriangleright xs)!i \rangle \\
 = & \{ \text{def. ! y aritmética} \} \\
 & (\#xs + 1) * x + \langle \sum i : 1 \leq i < \#xs + 1 : (\#xs + 1 - i) * (x \blacktriangleright xs)!i \rangle \\
 = & \{ \text{cambio de variable } i \rightarrow i + 1 \} \\
 & (\#xs + 1) * x + \langle \sum i : 1 \leq i + 1 < \#xs + 1 : (\#xs + 1 - (i + 1)) * (x \blacktriangleright xs)!(i + 1) \rangle \\
 = & \{ \text{aritmética en rango} \} \\
 & (\#xs + 1) * x + \langle \sum i : 0 \leq i < \#xs : (\#xs + 1 - (i + 1)) * (x \blacktriangleright xs)!(i + 1) \rangle \\
 = & \{ \text{aritmética y def ! en término} \} \\
 & (\#xs + 1) * x + \langle \sum i : 0 \leq i < \#xs : (\#xs - i) * xs!i \rangle \\
 = & \{ \text{H.I.} \} \\
 & (\#xs + 1) * x + f.xs
 \end{aligned}$$

Pude llegar a la H.I. sin problemas!

Caso base:

$f.[]$
 $= \{ \text{especificación} \}$
 $\langle \sum i : 0 \leq i < \#[] : (\#[] - i) * []!i \rangle$
 $= \{ \text{def. } \# \}$
 $\langle \sum i : 0 \leq i < 0 : (\#[] - i) * []!i \rangle$
 $= \{ \text{lógica y rango vacío} \}$
 0

Resultado final:

$f.[] \doteq 0$
 $f.(x \blacktriangleright xs) \doteq (\#xs + 1) * x + f.xs$

EJERCICIO 1b.

$f.[1,2,0,4,8] \rightarrow (\#[2,0,4,8] + 1) * 1 + f.[2,0,4,8]$
 $\rightarrow 5 * 1 + f.[2,0,4,8]$
 $\rightarrow 5 * 1 + (\#[0,4,8] + 1) * 2 + f.[0,4,8]$
 $\rightarrow 5 * 1 + 4 * 2 + f.[0,4,8]$
 $\rightarrow 5 * 1 + 4 * 2 + (\#[4,8] + 1) * 0 + f.[4,8]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + f.[4,8]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + (\#[8] + 1) * 4 + f.[8]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + 2 * 4 + f.[8]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + 2 * 4 + (\#[] + 1) * 8 + f.[]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + 2 * 4 + 1 * 8 + f.[]$
 $\rightarrow 5 * 1 + 4 * 2 + 3 * 0 + 2 * 4 + 1 * 8 + 0$
 $\rightarrow 5 + 8 + 0 + 8 + 8 + 0$
 $\rightarrow 29$

EJERCICIO 2.

Hago inducción en xs. Espero encontrar una definición de la forma:

$P.[] \doteq ???$
 $P.(x \blacktriangleright xs) \doteq ???$

Empiezo con el caso inductivo por las dudas haya que generalizar.

Caso inductivo:

$P.(x \blacktriangleright xs)$
 $= \{ \text{especificación} \}$
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$
 $= \{ \text{lógica (neutro } \wedge) \}$
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge \text{True} : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$
 $= \{ \text{lógica (3ro excluído)} \}$
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge (as = [] \vee as \neq []) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$
 $= \{ \text{lógica (distributividad)} \}$
 $\langle \exists as, b, bs : (x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = []) \vee (x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as \neq []) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$
 $= \{ \text{part. rango} \}$
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = [] : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$

$$\begin{aligned}
& \forall \langle \exists as, b, bs : x \triangleright xs = as ++ (b \triangleright bs) \wedge as \neq [] : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{2do cuantificador: cambio de variable } as \rightarrow a \triangleright as \} \\
& \dots \forall \langle \exists a, as, b, bs : x \triangleright xs = (a \triangleright as) ++ (b \triangleright bs) \wedge a \triangleright as \neq [] : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{def. ++ y propiedades de listas} \} \\
& \dots \forall \langle \exists a, as, b, bs : x = a \wedge xs = as ++ (b \triangleright bs) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{elim. variable } a \} \\
& \dots \forall \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{def. sum} \} \\
& \dots \forall \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : x + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle
\end{aligned}$$

Me trabo. No puedo llegar a la H.I. Se cancela la derivación por inducción!
Debo generalizar. Especifico:

$$gP.n.xs = \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : n + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle$$

Ahora, gP generaliza a P, por lo que P se puede derivar de la siguiente manera:

$$\begin{aligned}
& P.xs \\
& = \{ \text{especificación de P} \} \\
& \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{aritmética} \} \\
& \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : 0 + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{especificación de gP} \} \\
& gP.0.xs
\end{aligned}$$

Luego, podemos definir:

$$P.xs \doteq gP.0.xs$$

Falta derivar gP. Lo hago por inducción en xs.

Caso base:

$$\begin{aligned}
& gP.[] \\
& = \{ \text{especificación} \} \\
& \langle \exists as, b, bs : [] = as ++ (b \triangleright bs) : n + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{prop. de listas} \} \\
& \langle \exists as, b, bs : [] = as \wedge \text{False} : n + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{lógica y RANGO VACIO!} \} \\
& \text{False}
\end{aligned}$$

Caso inductivo:

$$\begin{aligned}
& gP.(x \triangleright xs) \\
& = \{ \text{mismos pasos que con P pero todo con el "n +"} \} \\
& \dots \forall \langle \exists as, b, bs : xs = as ++ (b \triangleright bs) : n + x + \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& = \{ \text{H.I.} \} \\
& \dots \forall gP.(n+x).xs \\
& = \{ \text{retorno 1er cuantificador pero con el "n +"} \}
\end{aligned}$$

$$\begin{aligned}
& \langle \exists as, b, bs : x \triangleright xs = as ++ (b \triangleright bs) \wedge as = [] : \text{sum.as} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& \vee gP.(n+x).xs \\
& = \{ \text{elim. variable as} \} \\
& \langle \exists b, bs : x \triangleright xs = [] ++ (b \triangleright bs) : \text{sum.[]} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& \vee gP.(n+x).xs \\
& = \{ \text{prop. listas} \} \\
& \langle \exists b, bs : x = b \wedge xs = bs : \text{sum.[]} \leq b \wedge \text{sum.bs} \leq b \rangle \\
& \vee gP.(n+x).xs \\
& = \{ \text{rango unitario} \} \\
& (\text{sum.[]} \leq x \wedge \text{sum.xs} \leq x) \vee gP.(n+x).xs \\
& = \{ \text{def. sum} \} \\
& (0 \leq x \wedge \text{sum.xs} \leq x) \vee gP.(n+x).xs
\end{aligned}$$

Resultado final:

$P.xs \doteq gP.0.xs$

$gP.[] \doteq \text{False}$

$gP.(x \triangleright xs) \doteq (0 \leq x \wedge \text{sum.xs} \leq x) \vee gP.(n+x).xs$

Solución 2do parcial turno tarde (2024)

Const N : Int, A : array [0, N) of Int;

Var res : Bool;

{ P: N ≥ 0 }

S

{ Q: res = $\langle \forall i : 0 \leq i \leq N : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle$ }

a) A = [-2, 5, 7, 3]. N = 4

Rango: $i \in \{0, 1, 2, 3, 4\}$

Términos:

$\langle \sum j : 0 \leq j < 0 : A.j \rangle < 2^0$

$\wedge \langle \sum j : 0 \leq j < 1 : A.j \rangle < 2^1$

$\wedge \langle \sum j : 0 \leq j < 2 : A.j \rangle < 2^2$

$\wedge \langle \sum j : 0 \leq j < 3 : A.j \rangle < 2^3$

$\wedge \langle \sum j : 0 \leq j < 4 : A.j \rangle < 2^4$

$\equiv \{ \text{resuelvo sumatorias y potencias} \}$

$0 < 1$

$\wedge A.0 < 2$

$\wedge A.0 + A.1 < 4$

$\wedge A.0 + A.1 + A.2 < 8$

$\wedge A.0 + A.1 + A.2 + A.3 < 16$

$\equiv \{ \text{resuelvo sumas} \}$

$0 < 1$

$\wedge -2 < 2$

$\wedge 3 < 4$

$\wedge 10 < 8$

$\wedge 13 < 16$

$\equiv \{ \text{resuelto} < \}$
 True
 $\wedge \text{True}$
 $\wedge \text{True}$
 $\wedge \text{False}$
 $\wedge \text{True}$
 $\equiv \{ \text{resuelto} \wedge \}$
 False

b) **Derivación:** Necesitamos un ciclo. Usamos la técnica de reemplazo de constante por variable. Obtenemos:

$\text{INV} \equiv \text{res} = \langle \forall i : 0 \leq i \leq \text{pos} : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle \wedge 0 \leq \text{pos} \leq N$
 $B \equiv \text{pos} < N$

El programa tendrá la siguiente estructura:

Const N : Int, A : array [0, N) of Int;
 Var res : Bool, pos : Int;
 { P }
 S_0
 { INV }
do $n < N \rightarrow$
 $\{ \text{INV} \wedge B \}$
 S_1
 $\{ \text{INV} \}$
od
 { Q }

Cuerpo del ciclo: Probamos con S_1 de la forma **res, pos := E, pos + 1**. Suponemos $\text{INV} \wedge B$ como hipótesis y vemos la wp:

$\text{wp}(\text{res, pos} := E, \text{pos} + 1). \text{INV}$
 $\equiv \{ \text{def. wp} \}$
 $E = \langle \forall i : 0 \leq i \leq \text{pos} + 1 : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle \wedge \underline{0 \leq \text{pos} + 1 \leq N}$
 $\equiv \{ \text{vale por hip. } 0 \leq \text{pos} \leq N \text{ y } B (\text{pos} < N) \}$
 $E = \langle \forall i : \underline{0 \leq i \leq \text{pos} + 1} : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle$
 $\equiv \{ \text{lógica} \}$
 $E = \langle \forall i : 0 \leq i \leq \text{pos} \vee i = \text{pos} + 1 : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle$
 $\equiv \{ \text{partición de rango y rango unitario} \}$
 $E = \langle \underline{\forall i : 0 \leq i \leq \text{pos} : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i} \rangle \wedge \langle \sum j : 0 \leq j < \text{pos} + 1 : A.j \rangle < 2^{(\text{pos}+1)}$
 $\equiv \{ \text{hipótesis} \}$
 $E = \text{res} \wedge \langle \underline{\sum j : 0 \leq j < \text{pos} + 1 : A.j} \rangle < 2^{(\text{pos}+1)}$
 $\equiv \{ \text{lógica, part. de rango y rango unitario} \}$
 $E = \text{res} \wedge \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle + A.\text{pos} < \underline{2^{(\text{pos}+1)}}$
 $\equiv \{ \text{álgebra} \}$
 $E = \text{res} \wedge \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle + A.\text{pos} < 2^{\text{pos}} * 2$

Acá nos trabamos pero podemos fortalecer:

$$INV' \equiv INV \wedge \text{sum} = \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle \wedge \text{pow} = 2^{\text{pos}}$$

Cuerpo del ciclo de nuevo: Ahora S_1 será de la forma **res, sum, pow, pos := E, F, G, pos**

+ 1. Suponemos $INV' \wedge B$ como hipótesis y vemos la wp:

$$\begin{aligned} & wp.(\text{res, sum, pow, pos} := E, F, G, \text{pos} + 1).INV' \\ & \equiv \{ \text{def. wp} \} \\ & E = \langle \forall \dots \rangle \wedge 0 \leq \text{pos} + 1 \leq N \wedge F = \langle \sum j : 0 \leq j < \text{pos} + 1 : A.j \rangle \wedge G = 2^{\text{pos} + 1} \\ & \equiv \{ \text{mismos pasos} \} \\ & E = \text{res} \wedge \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle + A.\text{pos} < 2^{\text{pos}} * 2 \wedge F = \dots \wedge G = \dots \\ & \equiv \{ \text{hipótesis nuevas} \} \\ & E = \text{res} \wedge \text{sum} + A.\text{pos} < \text{pow} * 2 \wedge F = \dots \wedge G = \dots \\ & \equiv \{ \text{elijo } E = \text{res} \wedge \text{sum} + A.\text{pos} < \text{pow} * 2 \} \\ & F = \langle \sum j : 0 \leq j < \text{pos} + 1 : A.j \rangle \wedge G = 2^{\text{pos} + 1} \\ & \equiv \{ \text{en F: lógica, part. de rango y rango unitario. en G: álgebra} \} \\ & F = \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle + A.\text{pos} \wedge G = 2^{\text{pos}} * 2 \\ & \equiv \{ \text{hipótesis nuevas} \} \\ & F = \text{sum} + A.\text{pos} \wedge G = \text{pow} * 2 \\ & \equiv \{ \text{elijo } F = \text{sum} + A.\text{pos} \text{ y } G = \text{pow} * 2 \} \\ & \text{True} \end{aligned}$$

Inicialización: S_0 será de la forma **res, sum, pow, pos := E, F, G, H**. Suponemos P como hipótesis y vemos la wp:

$$\begin{aligned} & wp.(\text{res, sum, pow, pos} := E, F, G, H).INV' \\ & \equiv \{ \text{def. wp} \} \\ & E = \langle \forall i : 0 \leq i \leq H : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle \\ & \wedge 0 \leq H \leq N \wedge F = \langle \sum j : 0 \leq j < H : A.j \rangle \wedge G = 2^H \\ & \equiv \{ \text{elijo } H = 0 \} \\ & E = \langle \forall i : 0 \leq i \leq 0 : \langle \sum j : 0 \leq j < i : A.j \rangle < 2^i \rangle \\ & \wedge 0 \leq 0 \leq N \wedge F = \langle \sum j : 0 \leq j < 0 : A.j \rangle \wedge G = 2^0 \\ & \equiv \{ \text{rango unitario (i = 0)} \} \\ & E = \langle \sum j : 0 \leq j < 0 : A.j \rangle < 2^0 \\ & \wedge 0 \leq 0 \leq N \wedge F = \langle \sum j : 0 \leq j < 0 : A.j \rangle \wedge G = 2^0 \\ & \equiv \{ \text{rangos vacíos y álgebra} \} \\ & E = 0 < 1 \wedge 0 \leq 0 \leq N \wedge F = 0 \wedge G = 1 \\ & \equiv \{ \text{elijo } E = \text{True}, F = 0 \text{ y } G = 1 \} \\ & 0 \leq 0 \leq N \\ & \equiv \{ \text{hipótesis} \} \\ & \text{True} \end{aligned}$$

Cota: La cota es $t = N - \text{pos}$.

1. Vale $N - \text{pos} \geq 0$ ya que el invariante dice $\text{pos} \leq N$.
2. Decrece con el cuerpo del ciclo ya que pos se incrementa en 1 y N es constante.

Resultado final:

Const $N : \text{Int}$, $A : \text{array}[0, N) \text{ of Int}$;
Var $\text{res} : \text{Bool}$, $\text{pos, sum, pow} : \text{Int}$;

```

{ P }
res, sum, pow, pos := True, 0, 1, 0;
do pos < N →
    res, sum, pow, pos := res ∧ (sum + A.pos < pow * 2), sum + A.pos, pow * 2, pos + 1
od
{ Q }

```

Variante: Otro fortalecimiento posible:

$$INV' \equiv INV \wedge \text{sum} = \langle \sum j : 0 \leq j < \text{pos} : A.j \rangle \wedge \text{pow} = 2^{\text{pos} + 1}$$

El resultado final en este caso es (los cambios en **negrita** y subrayado):

```

Const N : Int, A : array [0, N) of Int;
Var res : Bool, pos, sum, pow : Int;
{ P }
res, sum, pow, pos := True, 0, 2, 0;
do pos < N →
    res, sum, pow, pos := res ∧ (sum + A.pos < pow), sum + A.pos, pow * 2, pos + 1
od
{ Q }

```

c) Fortalecemos la guarda de la siguiente manera:

$$\begin{aligned}
 B' &\equiv B \wedge \text{res} \\
 &\equiv \text{pos} < N \wedge \text{res}
 \end{aligned}$$

Debemos demostrar que $INV' \wedge \neg \text{res} \Rightarrow Q$. Suponemos $INV' \wedge \neg \text{res}$ como hipótesis y vemos Q:

$$\begin{aligned}
 \underline{\text{res}} &= \langle \forall i : 0 \leq i \leq N : \dots \rangle \\
 &\equiv \{ \text{hip. } \neg \text{res} \} \\
 \text{False} &= \langle \forall i : \underline{0 \leq i \leq N} : \dots \rangle \\
 &\equiv \{ \text{lógica} \} \\
 \text{False} &= \langle \forall i : 0 \leq i \leq \text{pos} \vee \text{pos} < i \leq N : \dots \rangle \\
 &\equiv \{ \text{part. de rango} \} \\
 \text{False} &= \langle \forall i : 0 \leq i \leq \text{pos} : \dots \rangle \wedge \langle \forall i : \text{pos} < i \leq N : \dots \rangle \\
 &\equiv \{ \text{hipótesis INV} \} \\
 \text{False} &= \text{res} \wedge \langle \forall i : \text{pos} < i \leq N : \dots \rangle \\
 &\equiv \{ \text{hip. } \neg \text{res} \} \\
 \text{False} &= \text{False} \wedge \langle \forall i : \text{pos} < i \leq N : \dots \rangle \\
 &\equiv \{ \text{absorbente y más lógica} \} \\
 &\text{True}
 \end{aligned}$$

Solución examen final 17/12/2024

Ejercicio 1:

$p.xs = \langle \exists as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle$

a) Por inducción en xs.

Paso inductivo:

$p.(x \blacktriangleright xs)$
= { esp. }
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) : \dots \rangle$
= { 3ro excluido }
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge (as = [] \vee as \neq []) : \dots \rangle$
= { distrib. y part. de rango }
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = [] : \dots \rangle \vee$
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as \neq [] : \dots \rangle$
= { en el segundo cuantificador: cambio de variable $as \rightarrow a \blacktriangleright as$ }
 $\dots \vee \langle \exists a, as, b, bs : \underline{x \blacktriangleright xs = (a \blacktriangleright as) ++ (b \blacktriangleright bs) \wedge a \blacktriangleright as \neq []} :$
 $b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle$
= { propiedades de listas }
 $\dots \vee \langle \exists a, as, b, bs : x = a \wedge xs = as ++ (b \blacktriangleright bs) :$
 $b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle$
= { eliminación de variable a }
 $\dots \vee \langle \exists as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle$
= { Hipótesis Inductiva }
 $\dots \vee p.xs$
= { volvemos al primer cuantificador: }
 $\langle \exists as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = [] : \dots \rangle \vee p.xs$
= { eliminación de variable as }
 $\langle \exists b, bs : \underline{x \blacktriangleright xs = [] ++ (b \blacktriangleright bs)} : b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle \vee p.xs$
= { propiedades de listas }
 $\langle \exists b, bs : x = b \wedge xs = bs : b = \langle \sum i : 0 \leq i < \#bs \wedge (bs!i) \bmod 2 = 1 : bs!i \rangle \rangle \vee p.xs$
= { eliminación de variable (x) + rango unitario (xs) }
 $x = \langle \underline{\sum i : 0 \leq i < \#xs \wedge (xs!i) \bmod 2 = 1 : xs!i} \rangle \vee p.xs$
= { modularización!! introduzco nueva función especificada por:
 $sumImpar.xs = \langle \sum i : 0 \leq i < \#xs \wedge (xs!i) \bmod 2 = 1 : xs!i \rangle$
}
 $x = sumImpar.xs \vee p.xs$

Caso base:

$p.[]$
= { esp. }
 $\langle \exists as, b, bs : [] = as ++ (b \blacktriangleright bs) : \dots \rangle$
= { prop. listas }
 $\langle \exists as, b, bs : False : \dots \rangle$
= { rango vacío }
False

Resultado parcial:

$$p.(x \blacktriangleright xs) \doteq x = \text{sumImpar}.xs \vee p.xs$$

Derivación de sumImpar: SE LAS DEBO

Resultado final:

p.[] $\doteq$ False

$$p.(x \blacktriangleright xs) \doteq x = \text{sumImpar}.xs \vee p.xs$$

$$\text{sumImpar.}[] \doteq 0$$

$$\text{sumImpar.}(x \blacktriangleright xs) \doteq (x \bmod 2 = 0 \rightarrow \text{sumImpar}.xs \\ \quad \square x \bmod 2 = 1 \rightarrow x + \text{sumImpar}.xs \\ \quad)$$

b) Con la especificación:

$p.[4, 1, 0, 3] = (4 = 1 + 3) \vee (1 = 3) \vee (0 = 3) \vee (3 = 0)$
 $= \text{True} \vee \text{False} \vee \text{False} \vee \text{False}$
 $= \text{True}$

```
p[0] = (0 == 0)
      = True
```

$$p.[2, 1] = (2 = 1) \vee (1 = 0) \\ = \text{False}$$

Ejercicio 2:

```
Const N : Int, A : array[0, N) of Int;
```

```
Var r : Int;
```

$$\{P : N \geq 0\}$$

S

$$\{Q:r=\langle \exists i:0\leq i<N\wedge i\text{ mod }2=0:A.i=\langle \sum j:0\leq j<i\wedge j\text{ mod }2=1:A.j\rangle\rangle\}$$

a) $A = [7, -3, 0, 5, 2]$

$i \in \{0, 2, 4\}$:

- $i = 0: A.0 = 0 \quad \equiv \quad 7 = 0 \quad \equiv \text{False}$
- $i = 2: A.2 = A.1 \quad \equiv \quad 0 = -3 \quad \equiv \text{False}$
- $i = 3: A.4 = A.1 + A.3 \quad \equiv \quad 2 = (-3) + 5 \quad \equiv \text{True}$

Resultado final: $\text{False} \vee \text{False} \vee \text{True} \equiv \text{True}.$

b) Necesitamos un ciclo. Por reemplazo de constante por variable:

$$\text{INV} \equiv \quad r = \langle \exists i : 0 \leq i < \text{pos} \wedge i \bmod 2 = 0 : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle$$

$$\wedge \quad 0 \leq \text{pos} \leq N$$

$$B \equiv \text{pos} \neq N$$

Luego, $\text{INV} \wedge \neg B \Rightarrow Q$.

Cuerpo del ciclo: Suponemos $\text{INV} \wedge B$ y vemos la wp:

$$\begin{aligned} & \text{wp.}(r, \text{pos} := E, \text{pos} + 1). \text{INV} \\ \equiv & \{ \text{def. wp} \} \\ & E = \langle \exists i : 0 \leq i < \text{pos} + 1 \wedge i \bmod 2 = 0 : \dots \rangle \\ & \wedge \underline{0 \leq \text{pos} + 1 \leq N} \\ \equiv & \{ \text{vale por hip. } 0 \leq \text{pos} + 1 \leq N \text{ y } \text{pos} \neq N \} \\ & E = \langle \exists i : \underline{0 \leq i < \text{pos} + 1} \wedge i \bmod 2 = 0 : \dots \rangle \\ \equiv & \{ \text{lógica} \} \\ & E = \langle \exists i : (0 \leq i < \text{pos} \vee i = \text{pos}) \wedge i \bmod 2 = 0 : \dots \rangle \\ \equiv & \{ \text{distrib. y part. de rango} \} \\ & E = \langle \underline{\exists i : 0 \leq i < \text{pos} \wedge i \bmod 2 = 0 : \dots} \rangle \\ & \vee \langle \exists i : i = \text{pos} \wedge i \bmod 2 = 0 : \dots \rangle \\ \equiv & \{ \text{Hipótesis} \} \\ & E = r \vee \langle \exists i : \underline{i = \text{pos} \wedge i \bmod 2 = 0} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\ \equiv & \{ \text{Leibniz} \} \\ & E = r \vee \langle \exists i : i = \text{pos} \wedge \underline{\text{pos} \bmod 2 = 0} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\ & \bullet \text{ Caso 1: } \text{pos} \bmod 2 = 0 \\ & \equiv \{ \text{reemplazo} \} \\ & E = r \vee \langle \exists i : i = \text{pos} \wedge \text{True} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\ & \equiv \{ \text{rango unitario} \} \\ & E = r \vee A.\text{pos} = \underline{\langle \sum j : 0 \leq j < \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle} \\ & \text{Acá me trabo! Debo fortalecer.} \\ & \bullet \text{ Caso 2: } \text{pos} \bmod 2 \neq 0 \text{ (o lo que es lo mismo: } \text{pos} \bmod 2 = 1) \\ & \equiv \{ \text{reemplazo} \} \\ & E = r \vee \langle \exists i : i = \text{pos} \wedge \text{False} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\ & \equiv \{ \text{lógica y rango vacío} \} \\ & E = r \vee \text{False} \\ & \equiv \{ \text{elijo } E = r \} \\ & \text{True} \end{aligned}$$

Fortalecimiento:

$$\text{INV}' \equiv \text{INV} \wedge \text{sum} = \langle \sum j : 0 \leq j < \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle$$

Cuerpo del ciclo de nuevo: Suponemos $\text{INV}' \wedge B$ y vemos la wp:

$$\begin{aligned} & \text{wp.}(r, \text{sum}, \text{pos} := E, F, \text{pos} + 1). \text{INV}' \\ \equiv & \{ \text{def. wp} \} \\ & E = \langle \exists i : 0 \leq i < \text{pos} + 1 \wedge i \bmod 2 = 0 : \dots \rangle \\ & \wedge \underline{0 \leq \text{pos} + 1 \leq N} \\ & \wedge F = \langle \sum j : 0 \leq j < \text{pos} + 1 \wedge j \bmod 2 = 1 : A.j \rangle \\ \equiv & \{ \text{mismos pasos que antes hasta Leibniz} \} \\ & E = r \vee \langle \exists i : i = \text{pos} \wedge \text{pos} \bmod 2 = 0 : \dots \rangle \\ & \wedge F = \underline{\langle \sum j : 0 \leq j < \text{pos} + 1 \wedge j \bmod 2 = 1 : A.j \rangle} \\ \equiv & \{ \text{lógica y partición de rango} \} \end{aligned}$$

$$\begin{aligned}
& E = r \vee \langle \exists i : i = \text{pos} \wedge \text{pos} \bmod 2 = 0 : \dots \rangle \\
& \wedge F = \langle \sum j : 0 \leq j < \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle + \langle \sum j : j = \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle \\
& \equiv \{ \text{hipótesis} \} \\
& E = r \vee \langle \exists i : i = \text{pos} \wedge \text{pos} \bmod 2 = 0 : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\
& \wedge F = \text{sum} + \langle \sum j : j = \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle \\
& \equiv \{ \text{Leibniz} \} \\
& E = r \vee \langle \exists i : i = \text{pos} \wedge \text{pos} \bmod 2 = 0 : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\
& \wedge F = \text{sum} + \langle \sum j : j = \text{pos} \wedge \text{pos} \bmod 2 = 1 : A.j \rangle \\
& \bullet \text{ Caso 1: pos mod 2 = 0:} \\
& \quad \equiv \{ \text{"pos mod 2 = 0" es True y "pos mod 2 = 1" es False} \} \\
& \quad E = r \vee \langle \exists i : i = \text{pos} \wedge \text{True} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\
& \quad \wedge F = \text{sum} + \langle \sum j : j = \text{pos} \wedge \text{False} : A.j \rangle \\
& \quad \equiv \{ \text{rango unitario en el 1ro, rango vacío en el 2do} \} \\
& \quad E = r \vee A.\text{pos} = \langle \sum j : 0 \leq j < \text{pos} \wedge j \bmod 2 = 1 : A.j \rangle \\
& \quad \wedge F = \text{sum} + 0 \\
& \quad \equiv \{ \text{hipótesis, neutro} + \} \\
& \quad E = r \vee A.\text{pos} = \text{sum} \\
& \quad \wedge F = \text{sum} \\
& \quad \equiv \{ \text{elijo } \underline{E = r \vee A.\text{pos} = \text{sum}} \text{ y } \underline{F = \text{sum}} \} \\
& \quad \text{True} \\
& \bullet \text{ Caso 2: pos mod 2 } \neq 0: \\
& \quad \equiv \{ \text{"pos mod 2 = 0" es False y "pos mod 2 = 1" es True} \} \\
& \quad E = r \vee \langle \exists i : i = \text{pos} \wedge \text{False} : A.i = \langle \sum j : 0 \leq j < i \wedge j \bmod 2 = 1 : A.j \rangle \rangle \\
& \quad \wedge F = \text{sum} + \langle \sum j : j = \text{pos} \wedge \text{True} : A.j \rangle \\
& \quad \equiv \{ \text{rango vacío en el 1ro, rango unitario en el 2do} \} \\
& \quad E = r \vee \text{False} \\
& \quad \wedge F = \text{sum} + A.\text{pos} \\
& \quad \equiv \{ \text{elijo } \underline{E = r} \text{ y } \underline{F = \text{sum} + A.\text{pos}} \} \\
& \quad \text{True}
\end{aligned}$$

Inicialización: SE LAS DEBO. DA:

r, sum, pos := False, 0, 0

Resultado final:

Const N : Int, A : array[0, N) of Int;

Var r : Int;

{ P : N ≥ 0 }

r, sum, pos := False, 0, 0;

do n ≠ N →

if pos mod 2 = 0 →

r, sum, pos := r ∨ (A.pos = sum), sum, pos + 1

fi pos mod 2 = 1 →

r, sum, pos := r, sum + A.pos, pos + 1

fi

od

{ Q : r = ⟨ ∃ i : 0 ≤ i < N ∧ i mod 2 = 0 : A.i = ⟨ ∑ j : 0 ≤ j < i ∧ j mod 2 = 1 : A.j ⟩ ⟩ }

c) Nueva guarda: B' ≡ B ∧ ¬ r.

Demostración: Probar que $INV \wedge \neg(\neg r) \Rightarrow Q$. SE LAS DEBO.

Calendario sugerido

1ra clase: 2022-08-23

Preliminares

Cuantificación General

2da clase: 2022-08-23

Cuantificación General

Repaso: Sintaxis

Repaso: Lectura operacional

Ejemplos

A1: Rango vacío

A2: Rango unitario

A3: Partición de rango

3ra clase: 2022-08-30

Cuantificación General

Repaso

A4: Regla del término

A5: Término constante

A6: Distributividad

4ta clase: 2022-09-01

Cuantificación General

Repaso

A7: Anidado

T1: Cambio de variable

T2: Eliminación de variable

T3: Rango unitario y condición

T11: Leibniz 2

A8: Conteo

5ta clase: 2022-09-06

Cuantificación General

Repaso

Reglas adicionales para el conteo

Ejercicio 15b

Programación funcional

Introducción

... hasta Demostración

6ta clase: 2022-09-08

Programación funcional

Repaso: Introducción

Derivación

Testing

El lenguaje de programación funcional

7ma clase: 2022-09-13

Programación funcional

Repaso

Derivación

Modularización

8va clase: 2022-09-15

Programación funcional

Repaso: Modularización

Esquemas de inducción

Generalización

9na clase: 2022-09-22

Programación funcional

Repaso: Generalización

Segmentos de lista

10ma clase: 1ER PARCIAL

11va clase: 2022-09-29

Programación imperativa

Introducción

... hasta "Ejemplo: contar múltiplos de 6"

12va clase: 2022-10-04

Programación imperativa

Repaso

Arreglos

Lenguaje completo

Anotaciones de programa

Especificación

13va clase: 2022-10-06

Programación imperativa

Repaso

Especificación

Ternas de Hoare

14va clase: 2022-10-11

Programación imperativa

Repaso

Precondición más débil (weakest precondition)

15va clase: 2022-10-18

Programación imperativa

Repaso

Derivación de programas imperativos

Skip

Asignación

16va clase: 2022-10-20

Programación imperativa

Repaso

Derivación de programas imperativos

Condicional (if)

Secuenciación (;)

17va clase: 2022-10-22

Programación imperativa

Repaso

Derivación de programas imperativos

Ciclo / Repetición (do)

18va clase: 2022-10-27

Programación imperativa

Repaso

Derivación de ciclos: Técnicas para encontrar invariantes

Idea general

19va clase: 2022-11-01

Programación imperativa

Repaso

Derivación de ciclos: Técnicas para encontrar invariantes

1ra técnica: Tomar términos de una conjunción

2da técnica: Reemplazo de constantes por variables

... hasta 2do ejemplo

20va clase: 2022-11-03

Programación imperativa

Repaso

Derivación de ciclos: Técnicas para encontrar invariantes

2da técnica: Reemplazo de constantes por variables

Ejemplos

21va clase: 2022-11-08

Programación imperativa

Repaso

Derivación de ciclos: Técnicas para encontrar invariantes

Ciclos anidados

Ejemplos

22va clase: 2022-11-10

Programación imperativa

Repaso

Derivación de ciclos: Técnicas para encontrar invariantes

Fortalecimiento de invariantes

Ejemplos

23va clase: 2DO PARCIAL

24va clase:

Programación imperativa

Repaso

Terminación de ciclos: Función de cota

Problemas de bordes